A quiet space. Built on reliable foundations.

Kuro: complete research, current decisions, architecture audit and execution draft

Prepared for Max · 26 September 2026

2025–2026 market and monetization research
Seven architecture and planning audit lanes
Client, server, database, workers and music sources

Start with MAX_BRIEF.pdf.
This document contains the detailed reasoning, evidence, screen contracts and execution prompt.

Current decisions supersede earlier proposals.
Animation stays within welcome/authentication.
No application or hosted state was changed.
Static call inventories are not runtime measurements.
Implementation prompt remains a draft until design agreement.

Contents

Kuro — current decisions and architecture audit 3

Kuro — comprehensive execution prompt 12

Detailed screen/data contracts 18

Client call and lifecycle audit 27

Clubs, social and availability audit 34

Backend functions and ingestion audit 39

SQL, schema and modularity audit 48

Operational jobs audit 53

Music and metadata source research 58

Retained market and current-product evidence 62

Kuro — current decisions and architecture audit

26 September 2026 · Prepared for Max · Research and planning only

Read this first

The direction is a quiet personal space for anime and manga, with fast tracking, relevant updates and discovery through deliberately joined clubs. The approved animation belongs to welcome/authentication only. The earlier Home, first-login, agenda, Collection and email mockups were rejected; neither those layouts nor the later moving-mountains Home are an implementation reference.

This update combines the user's latest decisions with seven independent audit lanes: client requests, clubs/social/provider requests, Edge Functions, SQL/schema, operational workers, metadata/music, and the design contract. The complete companion review retains the earlier 2025–2026 market and monetization research, including its source register and access limitations. This is qualitative research, not a representative survey of what “most people” want.

No app code, hosted data, feature flags, schedules, credentials or release state were changed. Existing local AuthView edits remain untouched. Static call-site coverage is not proof of every request a running app makes; device traces and representative query plans remain implementation prerequisites.

A. Product decisions that now govern the plan

Decision	Required behavior	Status
Welcome/authentication	Preserve the owner-selected GitHub welcome, commit b943e457fd2b611bafa6a52dcbbada417d5b727a, including the no-TV composition. Carry its animation into authentication only.	Accepted direction; exact form layout still to design
Navigation	Retain the top selector and swipe interaction. Launch into Home. Search belongs to Discover.	Accepted; exact destination order and Browse/Concierge transition remain to settle
Anime / Manga	Remember one persistent choice and use it in requests, cache keys, labels, progress and discovery. Do not download both media solely to hide one.	Accepted
Home	Show current watching/reading, relevant new releases/seasons, club activity, personal recommendations and small useful snippets.	Accepted content; order and first-screen balance still need visual agreement
Discover	Find something new through search, browsing, filters and editorial routes.	Accepted; reuse existing useful Browse/Search functionality
Collection	Visual shelf, quick progress, clear Watching/Reading, Planned and Completed states; secondary Paused/Dropped; optional personal named shelves.	Accepted direction; shelf mechanics below are proposals
Shelf membership	Several shelves may contain one title; deleting a shelf preserves tracking history. Search covers unloaded records too.	Recommended implementation of accepted optional shelves
Clubs and profiles	Relationships arise only from shared clubs. Profiles show identity by default; activity appears only when enabled and authorized.	Accepted; identity fields and consent migration need design
Taste	Optional and secondary until its usefulness is demonstrated.	No approval to foreground or remove it
Detail music	Research season/episode-associated music, metadata and efficient ingestion.	Requested investigation; OP/ED vs soundtrack vs playback unresolved
Feedback and color	Satisfying, restrained motion and haptics; investigate cover-derived accents.	Ideas to test; no approval of colored statuses or constant animation
Monetization	Keep tracking, discovery, history, import/export and correction free. Useful optional commerce or disclosed partner benefits may be tested.	Accepted constraint; no partner or revenue model approved

Recommended navigation for review is Home → Discover → Collection → Clubs within the existing top swipe pattern. Taste and Concierge can be secondary Discover tools, but that placement is not yet a final owner decision. Club identity can remain independent of media choice; club title activity should respect the choice or explicitly identify cross-media content.

The Home first-screen proposal is a compact “what changed for you” area beside an immediately available continue/update action. It must not become either a giant discovery poster or a long activity dashboard. Test returning users, an empty account, an imported large library and a person using only manga. The evidence does not

establish one universal rail order.

B. What is built, enabled and still missing

The source stack is SwiftUI with @Observable and Supabase PostgreSQL, Edge Functions, Storage and Realtime. No MongoDB implementation was found in the inspected app/backend/scripts. “Manga” is a media domain in the same PostgreSQL system; a separate database is not justified by the request.

Capability	Evidence	Consequence
Existing top swipe pager, Discover/Browse/Search, list grid/rows, progress, clubs, title social activity	Current source; 93 app Swift files	Improve ownership and information hierarchy rather than replacing the stack
Taste, social and club feature flags	Live snapshot enabled at 100%	Availability is not proof of useful recommendations, correct privacy behavior or installed-client parity
Personalized New To You, realm discovery rails, provider availability	Live rollout 0%; provider availability dataset empty in preceding exact-count check	Built/staged; do not promise these to users or enable them blindly
Home as a separate personal destination	Proposed, not the current default screen	Requires its own scoped data contract and design
Personal named shelves and identity-first member profiles	Additional product work	Status filters and club rails do not already satisfy these requirements
Release support	Anime next-airing fields, upcoming query and reminders exist	Existing schedule data is too sparse/stale to promise a complete release calendar
Manga next-chapter schedule	No future timestamps in preceding live check; bulk mapping explicitly sets these fields to null	Chapter counts/catalogue rows are not a reliable official release schedule
Music song/version/episode model	Not established in current ingestion	Separate source and data model needed, subject to scope and permissions
GitHub welcome upload	Exact branch/commit verified in earlier review	Not proof that TestFlight, public App Store or the installed phone contains it

At 21:14 UTC on 25 September, exact aggregate checks found five future anime next-airing timestamps, none within seven days, no future episode rows, and no manga next-chapter timestamps. These are missing-coverage/freshness signals, not proof nothing is releasing. Later PostgreSQL catalogue estimates differ from those exact counts; they are approximate and have not been substituted for them.

C. Audit coverage and evidence strength

The evidence folder contains separate CSV registers for the core client, social extensions, Edge Functions, operational scripts and latest definitions of 49 literal iOS RPC names. Registers include request sites, callers, wrappers, external HTTP and subscription setup where relevant. Their totals must not be added together as unique requests or requests per session. Dynamic endpoints and runtime execution are explicitly bounded by each lane's coverage statement.

Read-only hosted inspection collected database metadata, indexes, selected function definitions, performance-advisor results, normalized statement statistics and sanitized cron summaries. Sixteen selected deployed function bodies match the latest local definitions after comments/whitespace normalization. That sample does not establish complete schema, migration checksum, Edge Function or runtime parity.

Evidence labels in the findings below:

- **Observed live:** a current metadata/statistics result, with a timestamp and interpretation limit.
- **Confirmed source:** a call graph or implementation visible in the inspected checkout; activation/runtime effect may be unverified.
- **Risk to reproduce:** a plausible failure requiring a targeted device/integration test.
- **Proposal:** a change and acceptance criterion, not implemented behavior.

D. Highest-priority findings and decisions

A01 — A simple list update triggers much more work than its visible change

Confirmed source. SupabaseService+UserLists.swift:109–115 awaits full user-list refresh, separate anime/manga Collection pages and the combined Collection feed after a successful upsert; anime additionally schedules reminders and fetches upcoming titles. The full-list path at lines 232–243 reads both user tables without pagination or an explicit projection. A server maximum-row setting could also make the apparent full snapshot incomplete. Existing in-flight deduplication helps, but does not eliminate the separate downstream work.

Decision: use one canonical mutation result to reconcile shared progress/status locally, invalidate only affected pages/sections, and refresh bounded data when needed. Do not refresh both media just because they share a service. Preserve Undo and error recovery. First reproduce the prior audit's rating-scale mismatch: some source paths send 100-scale values while the live list constraints accept 1–10.

Acceptance: instrument +1, Undo, rapid repeated edits and rejected writes. A successful basic progress action must not reload complete anime and manga lists or the mixed feed. Ratings/notes must survive unrelated progress changes; stale responses must not overwrite newer edits.

A02 — Mounted pages and weak cache ownership can create invisible work

Confirmed source. The pager retains previously mounted pages and neighbors. Mounting is therefore not proof of visibility. Some caches store results but do not avoid future reads; availability keys omit language parameters; broad prefetch task slots can cancel another surface's request. Club membership with a successful empty result is repeatedly treated as not loaded. Collection prefetches friend counts that its custom cards do not display.

Decision: preserve top swiping, but give loaders explicit active-surface, account and media scope. Use one keyed in-flight request per resource, cache empty results, batch visible IDs, and avoid decoration requests without a visible consumer. Keep public catalogue data separate from private list/social state.

Acceptance: cold/warm page changes, fast swiping, foreground return and switching Anime/Manga do not duplicate identical active requests. Old results cannot populate the new scope. Optional feature-off paths issue no associated requests.

A03 — Account reset and optimistic rollback need correctness checks

Confirmed source / risk to reproduce. resetUserState clears several arrays but not all derived list lookups, club/social/provider caches or in-flight work. Club cleanup is partly tied to view disappearance; the general sign-out realtime cleanup targets a different channel. Collection refresh error handling can restore an older snapshot after a newer request has begun. These findings do not establish a demonstrated cross-account disclosure.

Decision: centralize identity-epoch cancellation and private-state cleanup, including scheduled local airing reminders; guard all response and rollback commits by account and request generation. Keep public metadata caches only where no private decoration is embedded.

Acceptance: sign out/account switch during a slow request, failed refresh and active club subscription; no previous account's list, membership, activity, seen-state or preferences appears in the next account. Leaving a club or disabling sharing invalidates cached access immediately.

A04 — Live recommendation maintenance is a material background workload

Observed live, 22:16 UTC on 25 September. realm-similar-drive-5m completed 287 runs in the preceding 24 hours, mean 25.89 seconds, maximum 442.59 seconds. The daily realm-tier refresh took 133.57 seconds. Since statement-statistics reset on 4 August, the similar rebuild accumulated 15,060 calls averaging 25.54 seconds. These are job/function durations, not Home latency or a monetary bill.

The current live similar builder synchronizes seed state, recursively reconstructs franchise components and refreshes entry points before checking whether its stale-title batch is empty. This was deliberately implemented to keep franchise data fresh. An advisory lock already prevents overlapping execution; removing it or merely moving the early return would be an incomplete fix.

Decision: separate change-driven franchise maintenance from bounded similarity work. Preserve freshness/version dependencies, overlap protection and the last valid serving snapshot. Measure quiet/active queue costs and internal query plans before choosing the implementation. Review daily re-staling and realm refreshes as part of the same dependency graph.

Acceptance: an unchanged empty queue avoids whole-catalogue franchise recomputation; changed relations/cuts still update franchise entry points within the agreed freshness budget. Reader correctness and serving indexes remain intact.

Statement statistics are normalized aggregates; this server tracks top-level statements, not every nested statement. Cumulative scanned tuples and temporary bytes cannot be presented as per-request or daily totals. PostgreSQL statistics documentation

A05 — An idle local worker can still issue 1,000 diagnostic title calls

Confirmed source; installation/activation unverified. The relations worker calls its coverage report even with an empty queue. Defaults inspect 500 anime and 500 manga using individual ladder RPCs, plus relation counts and a paginated relation-table read. Its installer specifies a 15-minute cadence. That implies up to 96,000 ladder checks/day only if this exact configuration runs continuously; it is not measured live traffic.

Decision: separate work polling from diagnostic reporting; cache or batch the coverage summary, run it on changes or an explicit lower cadence, and document one supported scheduler per pipeline. Local dashboards that read local JSON are not database polling and should not be incorrectly removed.

Acceptance: idle worker call traces contain only bounded queue/lease checks; full reports run at their intended cadence. Confirm installed path and environment before changing any running job.

A06 — Import names and dispatch success overstate freshness

Confirmed source + observed live schedule. The anime/manga bulk imports request popularity-sorted catalogue pages even when track names imply airing/recent processing. Four live import jobs run hourly, including jobs whose names contain “weekly.” Cron's successful HTTP dispatch does not prove Edge Function completion, persisted updates or fresh release dates.

Some import checkpoints are saved only at run end; per-title failures may be caught while the page cursor advances. Lock acquisition fails open on some RPC errors. Episodes/relations can be deleted and then rewritten in separate operations; interrupted refreshes can leave partial data. A recommendation-edge importer retries HTTP

5xx indefinitely at a fixed interval. See the Edge and operational appendices for exact sites and qualifications.

Decision: define distinct initial backfill, current-title refresh and changed-source reconciliation jobs. Use explicit status/update filters where supported and authorized, durable per-item outcomes, bounded retry/deadline rules and distributed leases that fail safely. Replace a valid snapshot only after the replacement is ready. Keep anime episode freshness separate from manga chapter/edition release evidence.

Acceptance: forced provider failure, timeout, 429 and database 5xx leave resumable progress and last-known-good rows; no endless retry or silent skipped failed title. Report last successful source update, age and unresolved coverage—not merely a green cron dispatch.

A07 — Search and schema reproducibility deserve targeted fixes, not a database rewrite

Confirmed source / selected live parity. Search computes normalized title blobs, text vectors and similarities that do not align with some existing expression indexes. Friend-count SQL casts legacy text user IDs to UUID. Collection already has useful keyset pagination and matching indexes; retain those foundations. Performance-advisor output includes nine unindexed foreign keys, eight repeated-auth-evaluation RLS warnings and a primary-key warning on the live realm-tier table. The 105 unused-index notices are not permission to delete indexes, especially for disabled or low-traffic features.

The migration chain has replay gaps: an early dependency on a view whose CREATE exists in legacy SQL, a late baseline, and an object-kind mismatch around realm-tier. Sixteen sampled function bodies match live, but version-list equality cannot prove a fresh database can be recreated.

Decision: establish reproducible scratch/staging schema and a dependency map; migrate legacy identifiers carefully; align predicates and indexes using representative plans. Prioritize RLS/foreign-key work by actual hot path and write/delete behavior. Do not add MongoDB, duplicate anime/manga services, or drop indexes based only on advisor labels.

Acceptance: supported empty-database setup succeeds in isolation; selected current schemas/functions/indexes match the intended deployment; representative small/large-library search, counts and paging meet measured budgets without changing result semantics. Supabase query optimization guidance

A08 — Club realtime can pay for a refresh that the view does not consume

Confirmed source / risk to reproduce. Three of five club stream declarations lack club filters; RLS may still constrain delivered events. All streams debounce into a forced full bundle read. Own mutations may also explicitly refresh. The detail view stores its own bundle while realtime refresh updates the service cache; a visible consumer of that changed cache was not found. Failed subscription setup may leave the same-club guard set.

Decision: one observable club state owner and one subscription lifecycle, scoped to authorized club/resource changes. Deduplicate own mutation echoes, remove obsolete chat-triggered work if no supported consumer remains, and make successful refreshes reach displayed state. Do not equate stream declarations with independent queries.

Acceptance: one remote reaction updates the visible counts without a full unrelated reload; own reaction is not applied twice; unsubscribe, reconnect, backgrounding, permission loss and failed subscribe recover predictably.

A09 — Optional provider, detail and recommendation work needs bounds

Confirmed source. Availability can perform repeated status checks plus duplicate legacy/V2 prefetches; it remains rollout 0%. Entity-work pages have no explicit stable pagination. Credits fetch full entity descriptions before showing small previews. Recommendation fallbacks can repeat a secondary rail; parsing parallelizes candidates/variants without a shared concurrency ceiling. Taste failures can remain queued past the comment's stated retry count. Image slot waiting swallows cancellation and may repeatedly yield while all slots remain busy.

Decision: keep detail first paint small; page entity work; reuse a single availability result with a full scope/locale cache key; cap and deduplicate recommendation/provider work; stop retries under an explicit policy. Distinguish unavailable, failed and valid-empty states.

Acceptance: optional enrichment never blocks tracking or title identity, cannot loop forever, and is not fetched when hidden/disabled. Test cancellation and partial failures at concurrency limits. No provider rollout is part of this audit.

A10 — Data semantics must be corrected before they power richer screens

Confirmed source and a targeted live view check. The RAG cache key omits media type although its retrieval query uses it. Some ingest paths derive an episode total from aired-so-far or a manga total from the maximum indexed chapter; these are not equivalent. Undo can mark an import cancelled despite individual restore failures and does not guard every later-edited field. Account deletion includes a global orphan-club scan and an unpaginated storage listing; completeness requires an isolated test.

At 22:28 UTC on 25 September, the live `user_lists` view was confirmed to project `rating * 10` as score, null volume progress/start/completion dates, and a constant false `private` field for both media. Those projected placeholders are not actual user history or a consent model. The constant does not itself prove public access; RLS and RPC authorization remain separate. The missing CREATE VIEW in the migration chain must reproduce this live contract intentionally rather than adopting a guessed projection.

Decision: fix scope keys, version-safe apply/Undo and explicit progress/history/consent semantics before exposing them in Home, profiles, exports or recommendations. Scope account cleanup to affected resources and verify every page is removed in tests. Do not fill unknown history or dates with fabricated values.

Acceptance: identical Anime/Manga queries cannot cross-serve cached results; ongoing totals remain unknown where appropriate; Undo preserves subsequent rating/notes edits and reports partial failures; isolated deletion fixtures prove completeness without scanning unrelated clubs.

E. More value from the data already available

Information	Current verified position	Product use and next decision
Cover color	Imported for both media and decoded in models; source search found no actual view use of coverImageColor	Optional restrained artwork treatment needs no additional provider request. Contrast/monochrome approval still required.
Covers and banners	Already requested/stored; anime query requests extraLarge but mapping uses large	Choose image sizes intentionally; avoid downloading unused sizes or unbounded decoding. Audit before requesting more art.
Alternate titles, format, season/year, source/country, dates	Present in current catalogue/import paths	Better search, concise detail labels and filters; do not dump metadata into Home.
Relations and source IDs	Existing relation/ladder pipelines	Follow a franchise or sequel deliberately; distinguish adaptation, sequel, remake and compilation. A new season is not the same as next episode.
Airing times	Source capability and existing fields, with sparse/stale live data	Refresh selected/followed/current titles efficiently; show source age and confirmed vs unknown.
Chapters/volumes	Large existing datasets and progress paths	Preserve unknown totals, decimal/extra chapters and independent volume/edition concepts. Metadata coverage does not establish authorized reading availability.
Cast, staff, studios, authors	Existing import/model/detail paths	Fetch summaries first and full entity details on demand. Investigate pagination before promising complete credits.
External links	Current query uses site, URL, language and color	Provider schema also offers type, disabled state and notes; ingest only fields needed to avoid stale/mislabelled watch/read links.
Tag spoilers / role distinctions	Tag rank/spoiler fields and limited people/roles are already imported; more detailed dub/role distinctions exist upstream	Audit end-to-end presentation and top-ten pagination limits; keep spoiler-sensitive discovery and exact credit roles source-backed.
Music	AniList Media schema has no structured OP/ED song/version fields	Add a separate permitted metadata adapter if OP/ED is the intended feature. Never associate a song with every image solely by title.

Provider capability is not proof that all records contain the field. Every added field should have a user-visible purpose, measured coverage, freshness rule, stable source mapping and permission to use it. The existing source's terms must cover expanded ingestion; prior private permission was not inspected. See the primary-source music/metadata appendix for dated terms and access limits.

F. Music: useful first version and efficient data flow

Pending the user's scope answer, the narrow proposal is a detail section showing OP/ED, song, credited performer, version/episode coverage when known, and verified official listening links. AnimeThemes is structurally well suited; Jikan exposes less-structured theme strings; MusicBrainz can help identity matching if later needed. Direct sample API access was unsuccessful in this environment, so no working integration or coverage claim is made. Documentation and code licenses do not establish playback or commercial metadata reuse rights.

Treat the anime production, theme sequence, song, performer credit and theme version as separate entities. OP2 does not mean season two. Preserve nullable raw episode coverage and ambiguous matches. Match existing AniList/MAL IDs before title fallback; split cours and compilations may need review. Manga should not acquire arbitrary anime music automatically; an adaptation's soundtrack is a separately labelled association, if ever wanted.

Use one bounded server ingestion queue for authorized sources, batched ID matching, deduplicated song/artist hydration, conditional requests where supported, source timestamps and negative caching. Refresh airing titles more often than finished ones. Serve a compact cached summary; never request a music service separately for every card or image. Preserve last valid data on failure. No autoplay, audio download or background player is assumed. AnimeThemes documentation, AniList Media schema

G. Architecture: modular ownership without parallel systems

Keep SwiftUI, @Observable and Supabase. Extract responsibilities only when a concrete fix benefits from them; splitting extensions by filename alone is not modularity.

1. Session/scope owner: account epoch, persistent medium, selected surface, private-cache reset and cancellation. 2. Library owner: one status/progress truth, mutation results, Undo, paging, search and optional shelf membership. 3. Home/Discover coordinators: different bounded payloads, shared catalogue summaries and scoped loading. 4. Clubs/social owner: identity, membership, consent, scoped activity, visible state and realtime lifecycle. 5. Catalogue details owner: public metadata, relations, credits, optional music and availability adapters, each with one cache contract. 6. Ingestion/operations owner: jobs, leases, retries, checkpoints, provenance, freshness and run completion.

Define dependencies and cache keys before moving code. Keep compatibility while migrating callers. Avoid one enormous Home RPC that simply hides many scans, and avoid dozens of per-card requests. Choose bounded aggregation from measured request/payload costs.

H. Delivery order and remaining agreement

The companion execution prompt is comprehensive but remains a draft until the screens are agreed. It specifies phased acceptance checks and deployment boundaries; its presence is not an instruction to execute it now.

First agree a small set of phone states: auth form, returning and empty Home, Anime/Manga Discover, visual/compact Collection with shelf creation, Club/member profile, and detail music only after scope is clear. Show realistic data, large text, keyboard and empty/error states; use minimal copy. Resolve exact navigation order, Home hierarchy and whether subtle cover accents are appropriate. Do not create another elaborate animation-led Home concept.

Then establish request baselines and reproducible schema; fix tracking/account-state correctness and expensive idle/background work; implement scoped Home/Discover/Collection; improve clubs/profiles (new Home activity stays gated until server-enforced opt-in is ready); add qualified release and metadata paths; validate usefulness before optional music/Taste/monetization expansion. Some correctness and schema work can run independently after implementation is authorized, but unresolved visuals should not be silently chosen.

Cost measurement must separate database compute/storage, Edge execution, egress, Realtime, source APIs, AI and local workers. Record low/expected/high usage assumptions with actual billing inputs if forecasts are needed. No defensible monthly saving or maximum-user capacity follows from static site counts alone.

Outstanding proof: actual installed build/commit and phone visuals; cold/warm network traces; query plans on representative fixtures; installed local worker activation; full migration replay; working music API/rights/coverage; precise final design choices. These gaps are recorded, not presented as completed checks.

Kuro — comprehensive execution prompt

Status: draft for design agreement, 26 September 2026. This document is not authorization to execute changes or deploy. Use it after Max agrees the proposed screen direction and starts the implementation initiative. Unresolved details must remain explicit; do not turn an earlier rejected mockup into the default.

Mission and source of truth

Improve Kuro into a quiet, distinctive, effortless anime and manga tracking app. Align UI, data requests, persistence, ingestion, privacy and background work with the user's actual tasks. Preserve reliable existing foundations; repair correctness and unnecessary work before expanding promises.

Read AGENTS.md, CURRENT_APP_STATE.md, CURRENT_APP_STATE_PLAIN.md, this document, CURRENT_DECISIONS_AND_AUDIT.md, the complete KURO_FULL_REVIEW.md, and the call registers in this directory. Resolve conflicts using Max's latest explicit decisions first, then current verified implementation, then clearly labelled proposals. Older documents may contain superseded screen designs. The original research is qualitative evidence, not a market-share survey or proof a competitor still lacks a feature.

Start by recording working tree status, HEAD, existing edits, migration/function/flag state as necessary and the exact phone build used for tests. Preserve unrelated work, particularly the local AuthView composition. The approved welcome reference is GitHub b943e457fd2b611bafa6a52dcbbada417d5b727a, codex/kuro-onboarding-no-tv; use it as a visual reference, not permission to replace the current branch wholesale. GitHub, local, installed, TestFlight and public-release proof are separate.

Keep SwiftUI with @Observable, existing design tokens and Supabase. No Combine, second networking stack, speculative MongoDB migration, broad database rewrite or parallel tracking truth. Source reading is mandatory before modifying each relevant area. Extract modules incrementally when required by the work.

Product constraints

- Animation belongs to welcome/authentication only. Preserve the approved scene/composition and removal of the TV. Home and the rest of the app are quiet and static except restrained functional transitions.
- Preserve top navigation and swiping. Home is the first signed-in destination. Search and broad browsing belong to Discover, separate from Home.
- Persist an explicit Anime/Manga selection. Pass it through request parameters, caches, paging, progress vocabulary and recommendations. No accidental mixed payload followed by local hiding.
- Home shows current titles, relevant changes and upcoming releases/seasons, club context and small useful recommendations/snippets. Determine first-screen hierarchy through the agreed phone design, not a huge editorial hero by default.
- Collection is a visual shelf with an accessible compact option, canonical statuses and optional named shelves. Search covers the entire selected library, including unloaded items. Status, progress and shelf membership are separate.
- Social relationships exist only through clubs. Profiles show identity by default; activity requires opt-in and authorized shared-club access. Do not create a public friend/follow graph.
- Taste is optional. Existing feature flags do not establish value or approve prominence.
- Keep text short and clear. Status colors remain monochrome unless Max explicitly approves a limited change. Cover-derived accent exploration is optional; original artwork retains color. Use haptics for meaningful confirmed interactions, not every scroll or failed action.

- Keep core tracking, discovery, history, import/export, sorting and correction free. Any partner feature must be useful without compromising relevance or adding friction.
- Songs/playback remain a scoped decision: OP/ED metadata and verified link-outs are the narrow proposal. Full OST, background cues and native playback require distinct source/rights and UX decisions. Do not autoplay or fetch music per image.

Phase 0 — settle the design contract

Present a small coherent phone prototype that covers: approved welcome to auth, returning Home, empty Home, Anime and Manga Discover/search, Collection with statuses and personal shelves, club overview/detail, identity profile with activity off/on, and one detail page. Cover smaller phone widths, large text, keyboard, loading, offline, failure and no-data states. Use realistic bounded content and minimal labels. Mark mock content as such. Do not reuse the rejected first-login, Home, agenda, Collection or email layouts.

Resolve exactly: destination order and whether Browse/Concierge/Taste become Discover entries; Home's immediate balance of updates and continuation; where release history lives; private shelf creation/membership/order semantics; identity fields and activity controls; music scope; any artwork-derived accent treatment. A separate agenda is optional, not a required extra navigation destination.

For each screen, document the user job, essential first-paint fields, later fields, primary/secondary action, accessibility behavior, and the approved layout reference. Verify the user's intent before committing uncertain visual decisions. Implement only agreed states; continue independent correctness work if separately authorized.

Exit: an explicit decision record and screen/data contract. Visual agreement is distinct from authorization for hosted writes or releases.

Phase 1 — establish a baseline and reproducible schema

Reconcile the static registers against the new working tree. They cover client calls and call sites, all 16 local Edge entry points, scripts/research executables and historical cron declarations, plus 49 client RPC names and selected dependencies. Preserve distinctions among requests, wrappers, local storage URL helpers, realtime declarations and actual SQL work. Resolve dynamic targets during runtime tracing rather than claiming static exhaustiveness proves runtime completeness.

Instrument trace IDs and resource ownership without logging tokens, private list notes or club messages. For representative cold/warm launch, media switch, rapid swipe, debounced search, pagination, detail, +1/Undo, batch edits, import, club reaction, foreground/reconnect and account switch, record request count, duplicate count, bytes, cancellations, retries, latency distribution and visible outcome. Record feature flags, account/library size and cache state. Avoid extrapolating p95 from a handful of samples.

Use current statement/cron statistics before expensive diagnostics. Map HTTP/RPC -> SQL, RLS, triggers and downstream jobs. Obtain representative plans in isolation; production EXPLAIN ANALYZE/load tests are not automatically authorized. A LIMIT on output is not a bound on work. Separate frontend latency from background worker duration and database cost from Edge, egress, Storage, Realtime, external APIs and local computation. Set explicit proposed budgets after baseline measurement; do not invent current capacity or monthly savings.

Fix the documented fresh-bootstrap path in an isolated database: late baseline dependencies, missing user_lists CREATE, realm-tier table/matview mismatch and expected grants/policies/indexes. Compare current view definitions before adopting legacy ones. Preserve applied migration history and define an upgrade route. Validate empty setup and upgrade separately; no production reset.

Exit: reproducible baseline, schema contract, supported-job registry and measurable budgets for critical interactions. Record all blocked device/environment checks explicitly.

Phase 2 — tracking and session correctness

Read and repair rating-scale mismatch across AddToListSheet, progress mutation, imports and storage constraints. Define one normalized rating contract with explicit conversions and null behavior; do not silently lose ratings or notes when changing progress.

Replace broad post-mutation refresh chains with canonical mutation outcomes and targeted invalidation. Reconcile Home, Collection and details through shared state. Preserve Undo, optimistic feedback, server conflict handling and per-item batch outcomes. Scope full library reads deliberately; use pagination or bounded membership/status summaries rather than assuming a single unlimited response is complete.

Guard responses and rollbacks by account epoch and request generation. Centralize sign-out/account-switch cancellation and clearing of lists, derived lookups, club bundles, social counts, provider preferences, seen timestamps, pending signals, realtime subscriptions and scheduled local airing reminders. Existing reminder IDs are title-only; namespace/reconcile them by account and cancel private reminders at sign-out or permission loss. Keep only genuinely public metadata across accounts. Test slow replies arriving after a switch.

Correct valid-empty versus request-failure handling in Browse/Search/Collection and ancillary sections. Repair cancellation loops and finite retry policy in image-slot waiting, feature flags, Taste signals and analytics replay as identified in the client audit. Ensure a cancelled request cannot resume a polling/yield loop indefinitely. Verify image decoding/cache identity for size variants; optimize based on observed memory/render costs.

Exit checks: repeated +1 and Undo; simultaneous views; offline failure/retry; stale response; rating/notes preservation; unknown total; large library beyond API page cap; same numeric anime/manga ID; sign-out mid-request and account switch. A basic progress mutation must not refresh both complete libraries and the mixed feed.

Phase 3 — remove background amplification and make ingestion durable

Separate change-driven franchise/component/entry-point refresh from the five-minute similarity batch, preserving relation/cut freshness even when stale seeds are empty. Retain advisory overlap control and last-good serving rows. Measure unchanged idle and active-backlog cases, temp work and foreground contention. Reassess daily re-staling/realm dependencies before changing schedules; do not blindly stop jobs to make metrics look better.

Separate the relations worker's 1,000-title default coverage report from every queue poll. Batch/cache diagnostics and make reporting explicit. Verify actual installed launchd path/config before touching a job; installer cadence is not activation proof. Maintain a registry of job owner, trigger, tables, leases, rate/byte/time budget, checkpoint, completion evidence and stop mechanism. Distinguish current cron from 74 historical schedule declarations.

Make imports genuinely support backfill versus current/changed-title refresh. Track names must match query filters/order. Acquire leases before reading cursor state, fail safely on lock uncertainty, use fencing/idempotency, persist progress with successful writes, and retain failed items for bounded retry. Detect GraphQL errors even on HTTP 200. Respect provider rate headers/Retry-After and permissions, use abortable deadlines, finite jittered retry and honest complete/partial/failed run states.

Batch IDs, shared entities and joins per bounded page; skip unchanged source fields by fingerprint. Preserve source ownership of enriched synopsis, mirrored images, episode dates, manga release fields and language/edition metadata. Replace sets atomically or merge source-owned keys; never delete a valid relation/episode set and leave it half empty on interruption. Queue unresolved external relation IDs instead of dropping them.

Repair indefinite recommendation-edge 5xx retries. Make MangaDex duplicate conflicts stay batched rather than falling back to hundreds of calls. Distinguish final total, maximum indexed chapter/episode, current position and consumed amount; preserve fractional/extra chapters and provenance. Do not overwrite independently enriched fields with null/source defaults.

Review mirror queue claims, shrinking-result offset pagination, priority overlap, timeout/size caps and immutable URL versioning. Persist accepted descriptor generation in bounded durable chunks; scope model budgets and deadlines. The current source has Groq disambiguation/structured generation beyond the documented narration-only policy: explicitly resolve that discrepancy rather than silently expanding or deleting behavior.

Exit checks: interrupt after commit; expired lease; duplicate worker; provider 429/403/500; GraphQL errors; malformed response; no-progress run; unchanged metadata; source removal; unresolved relations; image revision; empty queue. Prove bounded calls, durable resume and last-good-data preservation.

Phase 4 — align SQL and request contracts with persistent scope

Create a typed account/device preference and explicit scope propagation using existing media identities/models. Define default selection and whether cross-device sync is desired; do not infer scope from taste. Include medium, user where private, effective filters/status/shelf/sort/cursor and region/language where relevant in keys. Reset stale cursors and cancel/discard obsolete requests.

Retain useful Collection/Browse keyset paging. Align Search expressions/indexed candidates while preserving exact/native/prefix/fuzzy ranking and safety rules. Compare plans for short queries, typos, empty queries, common filters and deep pages. Use fixed relevance fixtures before changing candidate limits. Address social batch caps and legacy identifier casts without a risky blanket ID migration.

Deduplicate identical recommendation rail builds, batch parse/apply lookups and limit shared concurrency. Correct RAG cache keys to include media type; distinguish backend failure from empty results. Audit Undo's partial-failure status and expected-version checks for rating/notes. Scope account-deletion cleanup and verify paginated storage deletion completeness using isolated fixtures, without deleting real accounts.

Review identified RLS initplan/foreign-key issues and realm-tier uniqueness with data/plan evidence. Do not delete unused indexes based only on counters or assume a missing PK can be added without checking duplicates. Verify exact matview refresh behavior and reader contention before switching strategy.

Exit: no accidental second-medium requests, correct scoped caching, bounded server inputs, stable paging and preserved search recall. Match changes to representative plans and measured interaction budgets.

Phase 5 — build Home, Discover and Collection from the agreed design

Give Home a bounded initial payload: current items/progress, relevant qualified updates, permitted club summary and optional recommendation reason. New person/title activity surfaces stay gated until Phase 6's server-enforced opt-in contract is implemented; membership-only identity summaries must not accidentally expose activity. Move the consent dependency earlier if Home needs it at launch. Reuse existing calls if they meet budgets; introduce a narrow aggregation only if it removes measured waste. Do not hide global scans inside one RPC. Main actions stay usable while secondary enrichment fails. Empty accounts get an easy first title/import route; Taste and tutorials remain optional.

Move intentional search/browse into Discover while preserving filters, scroll, query and deep-link compatibility. Do not maintain two inconsistent search implementations. Make visible-title metadata decoration batched and consumer-driven. Disabled capabilities cannot appear as functioning filters or personal recommendations.

Make Collection shelf-first with easy progress and a compact view where useful. Implement status partitions and optional name-only shelf creation. Store shelf membership separately, support deliberate ordering and multiple membership, define duplicate/limit behavior and enforce ownership/RLS. Removing a shelf or membership never deletes progress/history. Server-search unloaded titles; counts/sorts must reflect the declared whole-library/filter semantics. Provide simple, reversible multi-select actions with per-item outcomes.

Use selected medium's vocabulary and proper model: anime episodes, manga chapters/volumes and unknown totals. Avoid assuming total==current max. Preserve original import IDs and structured fields; improve

resume/pagination, reconciliation and export fidelity rather than converting everything into fuzzy title text. Preserve timestamps only when supplied; do not invent historical completion dates.

Exit: agreed phone-state comparison; first useful action, one-handed reach, VoiceOver, Dynamic Type, Reduce Motion and EN/DE labels; large/empty libraries; loading/offline/failure; no clutter or needless scroll; measured call budgets pass.

Phase 6 — clubs, identity profiles and consent

Build identity profiles within the club relationship model. Explicitly define which name/avatar/bio fields are visible and to whom. Activity/progress/history/notes/taste are separately governed; opt-in must be enforced in server projections and policies, not hidden after downloading. Decide how existing status-sharing memberships transition without silently publishing new fields.

Use one observable club detail state and a scoped realtime owner. A cache update must reach visible views. Filter/deduplicate delivered events, handle own mutation echoes, successful empty membership caches, subscribe failure/retry, backgrounding and leave/remove actions. Remove unused loading/prefetch work only after proving its consumer is obsolete.

Make club entry, invite/join, rails, polls and shared-title context obvious with minimal explanatory text. Do not add direct friendships, followers or public activity discovery. A profile visit must not fetch an entire library by default.

Exit: two-member clubs and clubs with three or more members, plus an outsider, in isolation; explicitly decide whether to preserve the current Club ACTIVE threshold that withholds member statuses below three members and reconcile it with title-level sharing-rank visibility; activity off/on; different overlapping-club privacy settings; last shared-club removal; archived club; stale caches; realtime reconnect; malicious endpoint access; no duplicated events or unintended private payloads.

Phase 7 — releases, details and music

Separate follow-release intent from merely saving a title. Define which statuses receive updates and opt-in reminders, notification history/read state if agreed, and cancellation when intent changes. Use explicit franchise/season relations for sequel notices; do not equate a TV season announcement with a scheduled episode. Distinguish broadcast, regional legal availability, original manga chapter, translated chapter and edition release.

Serve bounded relevant events with source, certainty and checked-at timestamps. No future date means unknown coverage unless verified otherwise. Do not promise a calendar from empty/stale timestamps. Establish source coverage/age metrics and a bounded pilot before broader rollout.

Details render identity and tracking first, then qualified relations/credits and expanded entities on demand. Paginate prolific entity works; avoid fetching full biographies for a tiny preview. Audit unused available fields and existing mappings before adding another source. Each field needs a UI purpose, provenance, freshness and deletion rule.

If OP/ED is approved, verify source permission/access and run an ID-matched metadata pilot. Model production -> theme -> song/credits -> version/episode coverage, preserving ambiguity and raw nullable episode strings. Batch and cache authorized metadata, including negative matches; track child updates and deleted records. Link only to verified official destinations. Do not infer soundtrack relationships for manga from the title alone. Audio playback is a distinct follow-up with an authorized player and user control; no hidden YouTube audio, scraped audio or unauthorized caching.

Keep provider availability at its current rollout until source/cost/coverage and freshness meet agreed criteria. Consolidate duplicate status/V2/legacy reads and full locale keys before enabling. No data-source permission or rollout authority is inferred from code existing.

Exit: representative airing/finished/missing/sequel/split-cour titles; manga unknown/decimal/edition-language cases; stale/outage behavior; no wrong music matches; no blocking detail dependency; real source/rights/coverage

evidence. Expose partial support honestly.

Phase 8 — polish, usefulness and monetization experiments

Apply the approved typography, spacing, artwork and compact hierarchy consistently. Use haptics/transitions only where they confirm meaningful changes; honor accessibility/system preferences and prevent duplicate feedback on retries. Cover-derived accents require an explicit design decision and contrast tests. Auth motion must stop outside authentication.

Measure first successful save/import, successful progress update, return-to-current-title use, meaningful discovery/save, release usefulness and club participation. Distinguish analytics correlation from causal retention lift; compare optional Taste/recommendations with an editorial baseline. Collect no unnecessary private notes/history, and define consent/retention for analytics. Do not use streak pressure or notification volume as a substitute for value.

Only after core usefulness, test neutral official-book/edition link-outs or clearly labelled publisher-funded samples/benefits as described in the monetization research. Verify native-app programme eligibility and current terms. Preserve all relevant noncommercial options, ranking independence, visible disclosure, no interstitials and no tracking requirement to use the app. Do not enroll/contact partners or publish sponsored material without authorization. Revenue estimates must label traffic, conversion, eligibility and commission assumptions; no guaranteed income.

Exit: meaningful user benefit and no degradation in task completion, latency, readability or trust. Remove an experiment if it adds friction without benefit.

Release discipline and completion contract

Work in reviewable phases. After each, update all three canonical state/plan documents and the permitted memory update note, including changed files and current counts. Add focused tests where behavior/risk warrants them; avoid tests that merely mirror visual constants. Run required relevant checks once, expanding only for unresolved failures. Preserve exact build/commit identity for every visual/runtime claim.

Before any deployment or hosted write requiring approval, finish the concrete code, migrations, fixtures, rollback plan and validation evidence so approval concerns a reviewable result. Do not ask repeatedly for already-granted authority. Do not change source permissions, credentials, source programmes, production schedules or rollout flags just to finish a plan.

For each phase report: resulting user behavior; files/contracts changed; tests and observed request/SQL improvements; source vs deployed status; unresolved limits; rollback. Attach any created PR. Completion means agreed behavior, data integrity, privacy, scoped/bounded requests, validated ingestion and confirmed release status for the authorized scope—not merely a green build, a large diff or uploaded source.

Detailed screen/data contracts

These are implementation proposals; current owner decisions above govern any conflicting prioritization. Home rail order still needs agreement.

Kuro — product contract and implementation-plan input

Planning synthesis · 26 September 2026 · No app, database, flags, or canonical documents changed.

This document translates the latest user decisions into implementation contracts. It supersedes the earlier four-destination bottom navigation proposal and the subsequent proposal to carry moving mountains through Home and the rest of the app. Existing research remains the evidence base for usability, scope, reliable progress, release uncertainty, privacy and free-core monetization. Neither earlier mockups nor an old GitHub branch are an implementation baseline.

1. Decisions to preserve

Accepted direction	Consequence for implementation
Keep the approved animation on welcome/auth only.	Reuse the exact approved mountain scene and motion as the reference. Do not add living scenery to Home, Collection, Discover, agenda or Clubs. Authentication should feel coherent with welcome without requiring moving art behind every form.
Keep quiet top navigation and swipe navigation.	Make destinations understandable and reachable within this pattern. Do not introduce the previously proposed bottom tab bar. Final page count/order remains an explicit design decision.
Persist Anime/Manga choice.	One shared selection governs media surfaces and their requests. Changing pages must not reset it. A hidden second medium must not be fetched merely because a view model supports both.
Separate Home from Discover.	Home helps resume and notice relevant changes; Discover combines intentional search and browsing. Do not make two copies of editorial discovery.
Home contains current watching/reading, relevant updates and recommendations, and club activity.	Resume/progress is the primary action. Recommendations need an honest reason and must respect actual feature availability. Social content comes only through shared Clubs.
Collection is a visual shelf, with status and personal named shelves.	Watching/Reading, Planned and Completed are primary. Paused/Dropped can be secondary. Named-shelf membership is independent of tracking status. Preserve whole-library search.
Profiles begin with identity; activity is opt-in.	Do not load or expose another person's history by default. Opt-in is constrained by the shared-Club audience, not permission for a public feed.
Social relationships flow through shared Clubs only.	No unsolicited followers, public social graph or broad activity feed. Check viewer membership and subject sharing permission on the server.
Taste is optional and currently unproven.	Do not require a taste questionnaire or mount its network work at startup to justify the feature. Keep entry discoverable but secondary until measured benefit exists.
Audit every database call, costs, modularity and metadata/music.	Account for client, server and background work, not just visible Swift RPC calls. Produce a cost model from observed workloads and current billing inputs; do not invent savings.
Core utility stays free.	Tracking, discovery, history, import/export and correction must not acquire paywalls or monetization friction. Partner commerce and any sponsored content remain optional, useful and disclosed under the existing research constraints.

2. Questions that remain design or implementation choices

These are decisions to resolve in the detailed plan, not reasons to stop the audit.

- **Navigation:** Home is a distinct destination, while search/browse belong to Discover. Decide the exact top-navigation order, initial page, treatment of an existing Browse deep link, and whether Taste is an auxiliary entry. Do not silently treat a new four-page map as approved.
- **Scope persistence:** propose a per-account preference on this device, with an explicit default for a new account. Cross-device syncing is a separate choice; logging out must not expose another account's personal cached state.
- **Library search:** propose search across all statuses, shelves and loaded/unloaded records of the selected medium. Provide a clearly labelled, deliberate way to search both media if desired. Confirm whether the user's phrase “whole library” instead requires both media by default; either way scope must be visible and the query

intentional.

- **Named shelves:** establish private-by-default ownership, ordering, limits and mixed-media behavior. Shelf membership should be possible in several shelves at once unless the user chooses otherwise. Renaming or deleting a shelf must not delete tracking history.
- **Profile identity:** agree which identity fields exist and which club members may see them. Identity-only does not mean public discovery of every profile. Define progress/comment/activity visibility separately, with conservative defaults.
- **Updates:** define whether this includes a new episode/chapter, official announcement, metadata correction, or club event. These require different provenance and unread semantics. “No known release” cannot mean “no release exists.”
- **Music:** determine whether the goal is opening/ending metadata, performers/composers, soundtrack links, playable previews, or in-app playback. Start with a source/coverage audit; audio playback and external services are separate capabilities and dependencies.
- **Styling and feedback:** cover-derived color, new transitions and haptics are optional proposals to validate, not requirements inferred from the user's liking the welcome animation. Preserve existing monochrome interface rules unless the user explicitly approves a change. Actual cover artwork can retain its original colors.

3. Shared data and interaction rules

A screen requests data for its purpose, current account and selected medium. The shared scope must reach the query boundary; filtering a combined response after download does not satisfy the scope requirement. Cache keys include user where applicable, medium, effective filters/status/shelf, sort and cursor. Use separate public metadata and private user-state caches so one person's status cannot leak into another account.

Keep existing data-service patterns and @Observable. Establish typed request/response contracts before splitting files. A new Home endpoint is not automatically necessary: first see which existing calls can compose a bounded snapshot, and whether a narrow aggregate operation measurably reduces duplicated work. One HTTP RPC can still perform costly SQL scans, joins, function calls and trigger work.

Cancel obsolete searches and scope requests; also reject stale completions using request identity/generation. Cancellation alone does not guarantee the server stopped work. Coalesce identical in-flight requests, preserve useful cached content through refresh failure, and invalidate the smallest affected section after a mutation. No automatically mounted offscreen page should trigger a full second load just because swiping may reach it later.

The payload shapes below describe information contracts, not approved SQL or endpoint names. Counts, timestamps and freshness fields must have clear semantics. Small display summaries should not hydrate full catalog entities and relations merely to render a card.

4. Per-screen contracts

Welcome and authentication

User outcome: understand that Kuro covers anime and manga; create an account or sign in clearly; retain the approved introduction's visual identity.

Data: session/auth state, operation status and precise recoverable errors. Terms and Privacy are independently accessible. The actual Apple/email flows remain explicit. No catalog, recommendations, Taste, club activity or user-library fetch is needed before successful authentication. Audit bootstrap after auth separately; it must load only necessary session/profile/flag state and the chosen destination.

Acceptance: both account paths work; existing-account switching is obvious; keyboards and errors do not obscure the primary action; reduced-motion behavior keeps a still approved image; no perpetual motion remains active while

auth is not visible; no mock Terms text ships as legal content.

Home

User outcome: immediately resume something, see a relevant change and notice shared-club activity without browsing a general catalog.

Data: bounded current items {id, medium, title, cover, progress, knownTotal?, userUpdatedAt}; relevant updates {type, titleId?, eventTime?, source, checkedAt, certainty}; a small recommendation section {candidate, reason, basis, generatedAt} when supported; club summaries {clubId, allowedActivity, unreadState}. Status/progress and named-shelf membership are separate fields. Unknown total remains unknown.

Calls: request only the selected medium's current items and title-related updates. Club identity or general club announcements can be intentionally scope-independent; media activities and rails must obey the selected scope unless labelled otherwise. Reuse cached profile/flags, avoid fetching full user lists for badge decoration, and load later sections only when appropriate. A missing personal recommendation capability yields an honest editorial fallback, not a fabricated “because you liked” claim.

Acceptance: cold launch exposes the continuation action before secondary content finishes; an account with no library gets import/find-first-title options; Taste is optional; a progress update immediately reconciles Home and Collection; recommendation failure does not block tracking; no club access means no friend activity; a stale release source produces an uncertainty message, not an empty-day assertion.

Discover: search and browse

User outcome: search directly or browse a coherent selected-medium catalog using useful filters and editorial routes.

Data: bounded card summaries plus sort/filter metadata and stable cursors; explicit scoped search results; editorial section identity and short reason. Metadata and availability filters must appear only when their data and enabled capability support the promise.

Calls: one effective search/browse request per debounced settled query and filter state; server-side selected-medium filtering; cursor paging; batch decoration for visible result IDs only. Editorial loading must not force both anime and manga rails. Switching medium should cancel/discard old results and use the new scope's cache rather than merge the two.

Acceptance: a title beyond the initial page can be found; changing scope or filters resets the cursor; no duplicate/omitted records across page boundaries in a stable dataset; unsupported filters cannot masquerade as reliable; back navigation preserves query, sort and scroll; whole-page search is not confused with filtering one rail.

Collection

User outcome: recognize their own shelf, find any tracked title, update progress quickly, and create an intuitively named shelf without a setup wizard.

Data: compact selected-medium list records with cover/title, status, progress, knownTotal?, rating if relevant, updatedAt, membership IDs; personal shelf definitions; a paged result cursor. Label the current status Watching for anime and Reading for manga while keeping one clear underlying state mapping.

Calls: selected-medium page/search with explicit status/shelf predicates; lightweight shelf definitions once per relevant invalidation; one mutation for a simple progress change; batched membership/status operations for multiselect. Search must cover unloaded records on the server. Do not fetch the entire collection or providers for every first paint. Remember visual shelf/list preference if the compact alternative remains useful.

Acceptance: name-only shelf creation succeeds with clear validation and local optimistic/error handling; adding to a named shelf does not change status; removing from a shelf does not remove from Collection; deleting a shelf preserves records; progress +1 has Undo and respects known bounds without inventing unknown totals; stale or

rejected writes reconcile honestly; multiselect does not lose selection on incidental metadata refresh; shelf and status filters are visible, clearable and distinct.

Updates / agenda (placement to resolve)

User outcome: know what is confirmed next for followed titles and what Kuro cannot currently verify.

Data: selected-medium, user-relevant events with timestamp/time-zone interpretation, event kind, source and freshness; a separate unscheduled/unknown state. A release date, local availability date and translated chapter publication are different facts.

Calls: bounded time-range request over followed title IDs, with server authorization and practical batching; no full global schedule download. Reuse Home's event summaries where semantics match. Refresh on a measured freshness policy, not a countdown-triggered request every second.

Acceptance: zero matching events is distinguished from missing or stale source coverage; unknown manga release timing never becomes a guessed recurring date; region/provider claims remain unavailable when the supporting capability is off; an event crossing midnight is shown consistently in the user's time zone. Current schedule scarcity is a product/data prerequisite, not something animation can fix.

Clubs and profiles

User outcome: connect through a deliberately joined group, with identity visible and activity under control.

Data: authorized club identity/membership/summary; selected-medium rail entries; activity appropriate to both viewer relationship and subject consent; identity-only profile by default. Expose the permitted information, not a broad payload hidden by UI conditions.

Calls: membership bootstrap once as needed; club detail on entry; bounded activity paging and one scoped real-time lifecycle. Do not query activity when disabled or when the viewer has no shared club. A profile visit must not trigger a hidden complete-library fetch. Invalidation must cover leaving a club, member removal and privacy changes, including cached content and active subscriptions.

Acceptance: strangers cannot obtain activity by calling endpoints directly; shared-club membership alone does not override activity opt-out; leaving the last shared club revokes access; identity and activity permissions remain separate; private progress stays out of analytics and notifications; multiple shared clubs do not duplicate events; no public social graph appears as incidental implementation scope.

Title details and optional Taste

Details data: a minimal identity/cover/synopsis/status/progress payload first, followed by visible and requested relations, cast/staff, author/studio, credits and external links. Optional music metadata needs stable title/season/episode association and source attribution, not a guessed title match. Avoid eagerly hydrating every nested entity, social section and provider relation on sheet presentation.

Taste data: only requested when the user enters the feature or explicitly participates. Explain what an input changes. Treat the current flag state as availability evidence, not proof of recommendation quality.

Acceptance: absent credits/music are distinguished from loading errors; alternate titles and duplicate names do not associate the wrong performer or soundtrack; credits and trailers do not autoplay audio; opting out of Taste does not degrade core access; recommendation benefit is evaluated against an editorial/control baseline before promoting it.

5. Every-call cost audit: required deliverable

Create an exhaustive request ledger that follows each user or scheduled trigger to its full workload:

screen / lifecycle / job → service call → HTTP/RPC/function → SQL dependencies / triggers / RLS → optional provider or storage operation.

Each row needs the owner and source location; trigger and repeat conditions; selected-medium support; authorization boundary; parameters and projection; cache key/TTL/invalidation; cancellation/deduplication; retries and failure behavior; rows returned and (where observable) read; payload/egress; measured duration; call frequency; and whether the work is necessary for visible UI. Flag unmeasured values as unknown.

Include authentication/bootstrap, all catalog/list/detail/search/recommendation calls, import and mutation fan-out, comment and reaction mutations, real-time subscription lifecycle, storage/images, background workers, scheduled imports/enrichment, materialized-view refreshes, analytics writes, feature flags and any third-party AI/provider calls. Separate database cost from storage/egress, Edge Function, real-time, AI, provider and local-compute costs. A local worker can still drive paid hosted work.

Trace representative cold and warm launch, scope switch, page swipe, search typing, scrolling/pagination, refresh, foreground/resume, offline/reconnect, progress update/Undo, bulk status change, named-shelf edit, import and club privacy change. Include repeated actions, retries and cancellation races, not only a happy path.

For each scenario report observed request count, duplicate work, payload size and latency distribution where samples permit. Then model low/expected/high usage using explicit assumptions and actual current billing/rate inputs, distinguishing marginal usage from fixed provisioned resources and included allowances. Do not price a database RPC as if every call had a fixed tariff. Select budgets after baseline measurement; do not fabricate existing p95, user volume or monthly savings.

Concrete source leads already inspected

- Kuro/Views/EditorialDiscoverView.swift: local `selectedMediaType` defaults to `.all`; `refreshSections()` calls `fetchUserLists`, `fetchDailyFeature` and a forced `fetchDiscoverBundle(limit: 30, hours: 24, force: true)`. The response hydrates both anime and manga rails. Follow its conditional recommendations and decoration prefetches through service and SQL layers.
- Kuro/Views/EditorialCollectionView.swift: local medium defaults to `.all`; initial/status/refresh/mutation paths can fetch user lists, a mixed feed and metadata. Audit all paths before declaring any duplicates wasteful.
- Kuro/Services/SupabaseService+Collection.swift: separate anime/manga paging RPCs already exist; `_fetchCollectionItemsImpl` currently invokes both. The mixed feed is explicitly interleaved. These are useful foundations for a scoped contract, not a reason to create a parallel collection stack.
- Kuro/Views/BrowseView.swift: separate anime/manga branches already exist; inspect whether requests, cursor and decoration lifecycle can become the common scope pattern.
- Kuro/ContentView.swift: existing quiet pager and lazy mounting are the navigation baseline. Inspect actual mount/lifecycle behavior before attributing background calls to the pager.
- Social and recommendation service extensions are separate source surfaces. Recommendation ID retrieval can be followed by entity hydration or genre fallback; a single apparent feature load is not necessarily one network or SQL operation.

These are source findings and audit leads. They are not a runtime cost benchmark, deployed parity check or proof that all listed paths execute in one session.

6. Modularity and metadata work

Refactor around accountable behaviors: shared selected-medium state, feature-owned loading/state, typed query contracts, reusable card summaries, mutation reconciliation and authorized social access. Keep `SupabaseService` compatibility while extracting only boundaries that demonstrably simplify ownership or prevent repeated work. Avoid introducing a second networking stack, competing caches or a generic framework for hypothetical screens.

The metadata audit should inventory fields and relationships actually stored, fetched and displayed for anime and manga separately. For each record type measure population, validity, source, freshness, identity match confidence, null meaning and locale coverage. Include episodes/chapters/volumes, dates, alternative titles, formats, descriptions, covers, external links, relations, people/studios/authors, and music-specific data if present. A populated field is not automatically reliable.

Music should begin with a precise content contract and source feasibility. Opening/ending names and credited artists, soundtrack links and audio playback have different matching, region and rights dependencies. Do not add ingestion, playback or a new provider until this distinction is resolved. Audit whether existing music-related code merely represents a genre or taste label rather than a music catalog capability.

7. Plan phases and dependencies

Phase	Work and exit evidence	Dependencies
0. Preserve the current baseline	Record current branch/SHA, dirty work, approved auth reference, enabled flags and existing artifacts. Separate source, local runtime and hosted evidence. Keep research decisions; mark rejected visual alternatives superseded.	None. No branch reset, broad checkout replacement or production writes.
1. Inventory and measure	Complete every-call ledger, scenario traces, schema/metadata/music inventory and cost assumptions. Identify duplication, broad projections, RLS risks, scope leakage and unbounded background work.	Phase 0. Read-only inspection first; instrumentation changes are separately reviewable.
2. Lock behavior and contracts	Resolve navigation order, scope semantics, whole-library search, shelf ownership, update semantics, profile permissions and music definition. Draft typed payloads and measured performance budgets.	Phase 1 findings plus accepted decisions above. No need to redesign all screens.
3. Build shared foundations	Shared scope/persistence, cache isolation, request identity/cancellation/deduplication, narrow selected-medium queries and targeted mutation invalidation. Preserve existing service conventions.	Phase 2 contracts. This must precede new Home data composition.
4. Deliver core journeys	Coherent auth; distinct Home; Discover search+browse; visual Collection and named-shelf operations; honest updates. Ship as small independently reviewable slices with appropriate integration tests.	Phase 3; update UI may need a limited honest state until source coverage improves.
5. Enforce social and deepen metadata	Identity-first profiles, shared-Club/consent authorization and revocation; detail loading boundaries; validated metadata/credit improvements; music capability only if supported by the agreed contract.	Phase 2 privacy/content contracts; Phase 3 shared state. Server privacy protections precede new activity exposure.
6. Validate optional additions	Measure Taste benefit; evaluate optional haptics/transitions/color treatments; validate beneficial partner monetization using earlier research and provider permissions.	Stable core journeys and privacy. No subscription or paywall insertion.
7. Release readiness	Repeat baseline scenarios, quantify actual cost/performance changes, check accessibility and failure modes, update required project documentation and obtain explicit deployment authorization if release is requested.	Completed slices and verified dependencies. Local success is not production proof.

8. Cross-cutting release acceptance

1. **Scope:** a captured Anime session performs no Manga catalog/list/detail work unless the user explicitly asks for cross-scope content; the reverse holds. Intentional scope-independent operations are separately listed. 2. **Persistence:** switching pages, reopening the app, opening details and returning preserves the selected medium; a deep link's target can open without silently changing the global preference unless that behavior is explicitly chosen. 3. **Race safety:** rapidly switching medium, query, status and shelves cannot paint results from an obsolete request or write progress to the wrong title. 4. **Reconciliation:** a successful change appears consistently across relevant screens; rejected writes recover clearly; Undo has defined limits and cannot silently overwrite a newer edit. 5. **Privacy:** direct endpoint tests prove private activity cannot be obtained through UI bypass, stale membership, disabled sharing, cache reuse or real-time continuation. 6. **Completeness:** search finds library entries beyond loaded pages; cursors are scoped correctly; counts disclose their scope; named-shelf deletion leaves tracking/history intact. 7. **Truthfulness:** unknown totals, unavailable providers, missing schedule coverage and editorial recommendation fallbacks are labelled accurately. No guessed release facts or personalization claims. 8. **Efficiency:** every required scenario has a before/after request trace; identified duplicate loads and N+1 decoration paths are resolved or explicitly justified; budget assertions use measured evidence. 9. **Accessibility:** usable type scaling and touch targets, meaningful VoiceOver order/labels, accessible contrast, keyboard-safe auth forms and reduced-motion support. Haptics are supplementary and optional. 10. **Scope discipline:** no app-wide animated mountains, bottom navigation, public social feed, mandatory Taste, unapproved playback or paywall slips in as a dependency. No rollback to an old branch replaces the current working app.

Client call and lifecycle audit

Kuro client request, latency and lifecycle audit

26 September 2026. Read-only current working-tree source audit. No app edits, build, device changes, hosted calls or database writes. Existing AuthView edits preserved. This memo supplies the execution plan, not proof of observed latency or production defects.

Scope and coverage

- Inventoried all **93 Swift files** under Kuro/. Audited owned transport boundaries and complete surrounding service methods, plus the relevant view lifecycle, state, pagination, mutation and cancellation paths. Pure rendering/model files were scanned for direct calls and traced where they create image or service consumers; this does not claim a line-by-line aesthetic review of every styling declaration.
- Six service files are delegated to the companion audit: SupabaseService+Clubs.swift, +ClubRealtime.swift, +Social.swift, +Streaming.swift, +Concierge.swift, +Taste.swift. Their **view callers are registered here**; their endpoint internals and database work are the companion lane's responsibility. Core Concierge methods remain in SupabaseService.swift, so those seven core Edge Function sites are covered here.
- evidence/client-call-register.csv: **381 source records**, comprising 76 ordinary request boundaries, 7 Edge Function invocation boundaries, 7 SDK session accessors, 1 SDK auth event listener, 3 realtime control sites, 228 view service-call sites, 35 image surfaces and 24 supplementary telemetry/image/flag/stream wrappers. **These are not 381 HTTP requests.** A view wrapper may call several endpoints or hit cache; one transport site may run repeatedly, loop over chunks, retry or be dormant. Auth session accessors can resolve from SDK cache or trigger refresh. Realtime registration is not a database query per declaration. Image surfaces generally share one URLSession boundary.
- evidence/client-source-coverage.json: file inventory, line counts and SHA256 at extraction, with delegated files explicitly marked. evidence/client-register-summary.json: machine-readable counts. Line references below are relative to /Users/max/Kuro-Anime-App and describe the inspected dirty working tree.
- Direct transport inventory includes PostgREST tables/views/RPCs, auth calls, function invokes, realtime and URLSession images. No app-side Supabase Storage upload/download SDK call was found. Downloaded cover URLs may be Supabase Storage/CDN URLs, but share the image pipeline. openURL hands off to an external destination; it is not silently counted as an app database fetch. Apple authentication is a system authorization UI followed by the registered Supabase ID-token exchange.

Priorities for the execution plan

C01 — One small list change triggers several broad refreshes

Source: SupabaseService+UserLists.swift:104–116 writes one row, then calls fetchUserLists, fetchCollectionItems and fetchCollectionFeed; anime adds a notification metadata read and throttled upcoming read. fetchUserLists performs two sequential table reads at 229/235; collection items calls two page RPCs at SupabaseService+Collection.swift:92–101; feed calls a third. SupabaseService.swift:2550–2564 debounces realtime by 350 ms, then refreshes the same sets after an echoed list change.

Source-derived request model: one upsert + five base read boundaries = six possible app requests before anime extras, realtime echoes, image loads and retries. Anime adds one reminder lookup and possibly one upcoming query. This is a conditional call graph, not measured per-action traffic. In-flight list coalescing can reduce overlap; collection refresh instead cancels/restarts. Under sufficiently separated responses, own-write echoes cause another broad sweep.

Additional amplification: EditorialCollectionView.swift:750–770 status changes call this full path for every selected item, ignore each Bool result, then refresh feed again. QuickVerdictActionCard.swift:648–671 can add a new completed title, set progress, then set verdict—three writes with repeated full refreshes. AddToListSheet.swift:142–144 fetches both whole lists just to edit one title. Profile/import paths also refresh state independently.

Plan: one typed mutation result should update the normalized in-memory record and invalidate only affected query keys. Coalesce server reconciliation across optimistic mutation, realtime echo and batch work. Add atomic/batched status mutations that preserve rating/notes and return per-item outcomes. Preserve the existing batch-remove improvement: skipRefresh at UserLists:318–349 and one final refresh at EditorialCollectionView:784–794 already avoid per-item refresh.

Acceptance: instrument one save, ten rapid increments, twenty-item batch and import; record outgoing requests/rows/bytes and duplicate invalidations. Use a single canonical mutation contract, not per-screen parallel implementations. Fix rating correctness before measuring perceived speed.

C02 — Whole-list reads are not actually a scalable full-library contract

Source: UserLists:229–238 uses .select() with user equality, no order, limit or pagination. In-flight coalescing at 183–194 is useful but has no TTL and does not guarantee completeness above a server response cap. App-derived membership/progress caches depend on these rows. Do not claim a particular hosted cap without root's configuration check.

Collection UI separately loads 90 card rows (Collection:144, :408), then searches/sorts/counts only loaded cards (EditorialCollectionView:96–178, :666–707) and disables its pagination sentinel while searchResults exists (:633). These are distinct gaps: raw list completeness and card-query global scope.

Plan: explicitly paginated lightweight tracking records, global server-filtered collection search/sort/counts, stable media-type-aware identity and complete import/export. Cache a single normalized library state and keep a selected view/page separate. Do not solve global search by eagerly downloading every full catalogue detail.

C03 — Account transitions need cache and task isolation

Source: SupabaseService.swift:861–885 clears visible lists/collections but not collectionAnimeIds, collectionMangaIds, userListByTypeAndId, userIdsByTypeAndStatus, outstanding list/collection/upcoming tasks, upcoming throttle/backoff, or several user-derived caches. Derived lookup state is defined at 293–296 and normally rebuilt by list fetch. fetchUserLists may let a new caller await an older user's in-flight task (UserLists:183–195); the implementation captures a user ID, then later commits results without an account generation check (:198–292).

Discover bundle cache key contains only limit/hours (SupabaseService:1450) while the bundle has user-sensitive New to You semantics. It is not cleared by resetUserState. Similar user-scoped cache ownership concerns in delegated services need reconciliation with that lane.

Risk: stale badges/state from a previous account and late results repopulating reset state; no cross-account runtime incident was reproduced. RLS can protect server data access while stale local state remains incorrect.

Plan: account/session generation token, account-keyed caches, cancellation/invalidation at logout and account change, rebuilding all derived indexes on reset, and post-await generation checks before mutation of shared state. Keep public image/catalogue caches reusable where appropriate.

C04 — Collection cancellation guards do not cover rollback or loading state

Source: refresh calls cancel the old task and immediately start another (Collection:10–43). Page success checks generation/cancellation (:411–415, :445–449, :478–482), which is good. But an old task's catch unconditionally calls restoreCollectionSnapshot (:102–109, :147–152). That snapshot restores both type arrays and feed/cursors (:156–172), so stale cancellation/failure can overwrite newer results or the other concurrent collection path. Both implementations also share isCollectionLoading and unconditionally clear it in defer (:67–69, :115–117).

Plan: generation-check success, failure, rollback and loading cleanup; scope snapshots to their query state. Coalesce identical active refreshes; cancel only genuinely superseded query keys. Verify rapid status changes with deliberately delayed A/B responses and cancellation during bootstrap.

C05 — Search debounce exists; response identity protection does not

Source: EditorialSearchView:179–215 cancels a 300 ms task before starting a reset. The service stores query/filter/cursor/results globally. Page methods guard shared isSearching, await results, then append without query generation validation (SupabaseService:1133–1267). Reset can change the query while an old page is active; the new fetch may return early because isSearching remains true, and old results/cursors can append to reset state. There is no onDisappear cancellation of searchTask in that view. Combined anime+manga requests are sequential (:1217–1248). refreshSearch resets cursors before requests and restores old arrays/hasMore on error but not old cursors (:1270–1380).

Plan: immutable search key including persistent Anime/Manga mode and all filters; generation-checked results/cursors; cancellation through the request path; independent per-query loading; typed error versus empty. Parallelize independent combined queries only if combined mode remains intentional—approved product direction now prefers persistent Anime/Manga selection.

Acceptance: typing A→B, changing type/filter while A is delayed, closing/reopening search, refresh failure after several pages; only current-query rows append, pagination neither duplicates nor stalls.

C06 — Initial page mounting and forced refresh cause avoidable work

Source: bootstrap starts two list reads, two collection-type reads, one feed read and one upcoming read (SupabaseService:927–934) independently of the first screen. ContentView warms Discover (:364–368), Discover's own first task forces the bundle (EditorialDiscoverView:382–387, :439–441), and discovers daily feature plus list refresh. In-flight bundle sharing limits simultaneous duplicates, but a completed parent warmup is bypassed by force=true. Default Discover also mounts immediate neighbors—Taste/Concierge and Browse—because ContentView:578–596 mounts distance<=1. mountedSections retains every visited page; despite its comment, previously visited distant pages remain mounted. Browse starts its initial network request on mount (BrowseView:307–311), and Taste/Concierge have their own startup calls.

AuthView:97, :592 loads twelve trending full-model anime even when the normal welcome uses only the bundled mountain asset; those covers serve the retained debug Genie scene. Removing this unnecessary data fetch need not alter the approved welcome artwork/composition.

Plan: critical-path Home data only; activation-aware side-page loading; reuse fresh bundle on first render and reserve force for intentional refresh; defer background prefetch by measured value and connectivity. Keep top swipe navigation; activation is independent of layout mounting. Retain the exact approved mountain welcome, not a new welcome design.

Acceptance: cold/warm/restored login request waterfall, first useful paint and main-thread work measured separately. SDK session lookups are conditional requests, not automatically additional HTTP. Restore currently waits for profile upsert before releasing the auth bootstrapping gate (SupabaseService:572–582); assess this dependency.

C07 — Reconnect can spend a request without updating the visible Discover model

Source: ContentView:401–411 reconnect calls fetchDiscoverBundle(force:true) and discards its return. Discover renders view-model arrays populated only inside refreshSections (EditorialDiscoverView:451–490). The service caches that result but does not publish it into those arrays. On Collection reconnect, root fetches status:nil rather than the selected view filter.

Plan: reconnect invalidates/refreshes the owning active query controller, with that screen's full query key. Do not duplicate a low-level fetch in the navigation shell. Test visible recovery after offline initial load and after a filtered

Collection reconnect.

C08 — Detail pages are bounded, but repeated subqueries and failure fanout remain

Positive: full detail fetch is ID+limit1; memory/disk fallback exists (SupabaseService:1492–1562). Tags use one projected nested query rather than one request per tag. Similar IDs are fetched with a max 50 RPC, normally 14, and batch-hydrated rather than normally fetched N+1 (Recommendations:40–94).

Risks: detail memory cache has no TTL or per-key in-flight sharing. Recommendation batch writes bypass the nominal 200 cache cap (Recommendations:89–92, :144–147). A missing/failed batch falls back to unrestricted parallel per-ID tasks (:99–111, :154–166): normally 14 or max 50 extra requests precisely when the service may be failing. Genre fallback can add another query.

Anime detail requests tags/similar/ladder concurrently (AnimeDetailView:207–220), plus flag-gated credits, preview episodes and dock links. Episode preview fetches 10 rows while rendering just the next row (:967–979, :1005). Dock re-fetches an episode link and title links on every progress change (:1528, :1621–1626). Manga's chapter preview task sequentially loads chapters, status and legal links (MangaDetailView:693–696), while its dock separately fetches the same title's legal links (:1389). fetchExternalLinks has no cache or in-flight coalescing (UserLists:526–542).

Plan: shared detail query state; TTL/in-flight coalescing by media type/ID/locale; slim card hydration for recommendations; bounded retry/fallback concurrency and distinguish missing rows from failed batch. Load lower sections when useful. Keep detail full-row reads where actually justified. A detail bundle is optional after measurement, not a mandate to create another giant endpoint.

C09 — Errors becoming empty pages produce false exhaustion

Source: Browse service catches and returns [] (Browse:42–47, :83–88); view sets hasMore from count and only calls initial failure an error if the network monitor says offline (BrowseView:393–410, :454–480). Connected server/decode failures can look like valid empty data. Episode/chapter page methods likewise return[] on error (UserLists:565–616); sheets set hasMore=false (AnimeDetailView:1334–1338, MangaDetailView:1028–1032).

Positive: keyset Browse uses capped at 120 (view 60), 120 ms debounce, generation guards and cancel-on-disappear. Collection keyset pages are capped at 120 (feed 90/type 80). Shared sentinel triggers at most once per count (KuroLoadMoreSentinel:39–49), so it avoids repeated onAppear flooding.

Plan: typed page result separating success-empty, cancellation and failure; retry affordance retaining cursor/items. Sentinel should be able to retry a failed page even if item count has not changed. Keyset-by-number is preferable for mutable long chapter/episode sequences. No autonomous endless database pagination loop was established; errors more often falsely end or stall it.

C10 — Genre detail pays six broad queries per refresh

Source: GenreHubView:164–210 concurrently requests five rails plus tag facets whenever mounted or media kind changes (:83–87). Four rails each fetch 30 full rows, New to You fetches 300 anime/350 manga candidates to return 30, and tags fetch 200 parents with nested edges. No cache, generation token or cancellation for rapid mode changes; shared facets may be overwritten by older work. Full-model .select() rail helpers are at SupabaseService:1570–1878; nested tag sample projections at 2005/2051.

Plan: bounded projected genre cards + server exclusion/qualified facets, reuse fresh per-genre/type results, and generation protection. Do not confuse concurrent queries with zero cost; parallelism improves elapsed time while retaining transfer/DB cost. Legacy helpers with no current view consumers should be marked dormant, not attributed to every app launch.

C11 — Image pipeline has good bounds but cancellation and sizing holes

Positive: ImagePipeline:20–28 defines 80 MB memory, 40 in-flight and 12 prefetch fanout; URL-key dedupe; 20 s timeout; URLCache configured with 64 MB memory/256 MB disk by KuroApp:20–22; decoded detail disk cache caps 300 files separately. Most image surfaces share this path. Legacy PosterView:16 uses AsyncImage, but only Preview callers were found, so don't claim it is a live screen bottleneck.

Risks: ImagePipeline:84–88 waits while all slots are used, swallows sleep cancellation and never checks Task.isCancelled. A canceled waiter can keep yielding/spinning until a slot frees and still start a request. Shared load tasks have no consumer-aware cancellation. Cache/in-flight key is URL only, omitting desired decode size (:19–20, :53–76); a low-resolution first consumer can supply a later hero. clampPixelSize(nil) returns nil (:135–144), and Discover/Browse/Collection prefetch omit maxPixelSize, so those prefetches decode the original image instead of applying the 1200 px cap. This means downsampling controls exist but do not cover every path. No response byte budget is enforced before full download. KuroCachedAsyncImage:54–79 writes phase after awaiting without cancellation or URL-generation validation.

Plan: cancellation-aware bounded scheduler, default decode target for all callers, resolution-aware cache strategy (or safely upgrade cached images), explicit viewport prefetch budget, off-main bounded decode, and HTTP/cache freshness policy appropriate for immutable image URLs. Measure decoded pixels/bytes and canceled work, not just request count.

C12 — Taste retries are not bounded by the stated retry count

Source: TasteDeckView:183–220 launches a Task for each signal; failed rows remain pending even after failures>=3, which merely sets a banner. Every later commit/appear retries the whole pending batch; no queue length cap or time-based retry backoff is present. isFlushingSignals prevents overlapping queue flushes, but a concurrent new signal skips waiting for the current flush and then sends its own write. Undo records another asynchronous signal. This risks increasing work during outages and out-of-order state; server idempotency/order contract belongs to the companion lane.

Positive: batch fetch itself has isFetchingBatch/batchExhausted guards, 12-row requests and local dealt-key dedupe (:254–275). There is no evidence of a fetch-every-frame loop. Endless means user-driven replenishment, not automatically exhausting the catalogue.

Plan: per-title ordered, bounded signal delivery; retract/supersession semantics; bounded retry policy and outage backoff. Preserve an optional Taste tool; do not add obligatory training to login. Companion audit should reconcile endpoint cost and retry semantics.

C13 — Telemetry flush can launch 200 individual inserts

Source: ConciergeAnalytics:19–20 caps offline queue at 200; :73–81 drains it by calling one detached-style fire-and-forget Task per row; each inserts at 93–95. Reconnect triggers flush in KuroApp:116–119. No concurrency cap, bulk insert or persistent retry; failed sends are discarded. Queued rows capture user_id before account changes, and setUserId does not clear/isolate the queue (:36–39). Outbound-link logging is one non-blocking RPC per tap (SupabaseService+Monetization:12–31), with no affiliate flag guard in that method; this ledger is not proof of monetized links or partner approval.

Plan: small bounded batches, backpressure, account-scoped queue, explicit retention/drop policy and privacy preference alignment. Approved product direction keeps social club-only and profile activity opt-in; telemetry must not silently create a social activity feed. Do not make core navigation wait for logging.

C14 — Retry/cancellation and feature flag freshness policy needs consolidation

Source: SupabaseService:218–245 retries only URLError, at most 3 total attempts, with 500/1000 ms waits—finite, useful bounds. It does not explicitly exclude canceled requests from eligible URLError categories; however, a canceled parent task will normally terminate at the throwing retry sleep, so this is a policy gap rather than proof that cancellation always retries. Shared unstructured parse/recommend/bundle tasks do not automatically cancel when an individual awaiting view disappears. Mutating apply/undo do not get that automatic retry, a useful safety property.

FeatureFlags:56–100 has 4 attempts separated by 10/30/60 s; sleep uses try? and no cancellation check. Multiple refreshes have no coalescing/TTL despite lastFetchDate being recorded. Cached flags survive network failure, appropriate for resilience but without age visibility. Bootstrap launches flag refresh without awaiting it, then immediately tests the cached provider flag (SupabaseService:939–950), so a fresh flag change may not trigger provider bootstrap until another path runs.

Plan: one explicit policy for cancellation, finite transient retries, request sharing and stale-cache age. Coalesced flags refresh with observable generation; feature data activation reacts to resolved flags. Do not blindly retry writes or add a universal blocking refresh on foreground.

C15 — Auth validation and profile work are bounded but tied to unrelated work

Source: signup email check is 500 ms debounced and cancellation-checked (AuthView:535–553); service returns false for failed check (SupabaseService:709–718), which can appear “available” rather than “check unavailable.” Profile upsert writes adult_opt_in=false each restore/login (:909–918), not merely insert-if-missing; root should assess intended preference semantics. Auth session property reads may be local; isAppleUser uses an actual user fetch (:795) only on explicit delete flow. No unbounded auth polling found.

Plan: retain approved auth mountain design, eliminate unused welcome network dependency, make validation failure distinct, and avoid requiring unrelated catalogue/collection completion to establish a usable authenticated screen. Functional auth state fixes from the approved-reference review remain separate from visual redesign.

Modularity and approved screen direction

The existing extensions, @Observable and SwiftUI are the right foundation. Do not introduce parallel data stores or replace the stack. SupabaseService.swift is 2972 lines combining authentication, caching, legacy browse/search, Concierge transport, realtime and state; extraction into files alone has not separated lifecycle ownership.

Execution should introduce small, testable query/mutation ownership boundaries on that stack:

1. Session/account scope: auth identity, generation and teardown. 2. Catalogue queries: explicit card/detail projections, media-aware keys, per-key cache/in-flight state, cancellation. 3. Collection state: normalized tracking records, named shelf membership separate from status, mutation reconciliation and one invalidation coordinator. 4. Screen query controllers: Home distinct from Discover; Discover owns search/Browse; persistent Anime/Manga mode is in every relevant key; top swipe navigation only activates useful work. 5. Delivery policies: image scheduling, bounded analytics and Taste queues; independent of navigation rendering.

Club-only social and opt-in profile activity require server/privacy policy work in the companion lane. Removing unconsumed social count prefetch from general pages may save requests, but do not remove useful club context without the agreed scope. Auth animation is confined to the approved welcome; none of these performance proposals requires moving its headline or replacing artwork.

Safe or bounded cases to retain

- ID-specific full detail fetches, deliberate explicit mutation requests, user-filtered realtime registration, finite retry counts.

- Search card RPC max 50, Browse/Collection max 120 and proper keyset cursor components; do not replace these with global catalogue downloads.
- Existing per-key Discover/Concierge in-flight coalescing and TTLs, while fixing account/query identity and invalidation.
- Upcoming 20 s cooldown, exponential failure backoff and 500-row bound; countdown Timer updates local strings every 60 s and does **not** query the server.
- Batch recommendation hydration and 200-ID export title chunks on the normal path; not universally N+1. Fix only the failure fallback and media-type identity issues.
- Batch remove already refreshes once; structured profile import preview caching avoids re-downloading the same snapshot on confirmation. Fidelity/caps are a separate data contract problem.
- Apple Foundation Models work is on-device, not an HTTP endpoint. It still affects task latency; do not put it into database-request totals.
- Dormant methods: legacy searchContent, offset Browse helpers, several older rail helpers and Preview-only PosterView should be documented/removed only after reachability checks, not charged against live use by assumption.

Validation gates, not invented performance results

No request trace, byte count, latency percentile, memory profile, SQL execution plan or end-to-end save was captured in this audit. Root's hosted SQL/cron/worker review is complementary; a capped client query may still execute an expensive plan, and an RPC's returned row count does not reveal its internal cost.

Before changes, instrument request name, safe query-key hash, trigger/screen, cold/cache/coalesced status, duration, bytes, returned rows, retry index and cancellation. Avoid credentials, emails, notes and raw import text. Capture cold/warm startup; ten page swipes; repeated detail visits; typing/scope races; a collection larger than server row caps; twenty-item batch; offline/reconnect; rapid account switch; five minutes of fast Taste judgments; image-heavy scrolling.

For each work package set a measured baseline and explicit budget: request count per action, first useful content, p50/p95 latency, transferred bytes, peak decode memory and canceled work. Proposed correctness gates: no late prior-account writes to UI state; no stale-query append/rollback; no silent page exhaustion on failure; bounded retry and queue lengths; ratings/notes preserved; one coherent collection update per confirmed action. No statement here says these gates currently pass.

Inventory verification: all 381 IDs are unique, all file/line references are in range, and all 93 source hashes still matched at final verification. Forty-eight files contain registered boundaries or consumers; remaining files are accounted for in the coverage inventory. No repository files changed in this audit.

Clubs, social and availability audit

Kuro social, clubs, availability, import and taste client-call audit

2026-09-26. Read-only local source inspection. Scope: Kuro/Services/SupabaseService+Clubs.swift, +ClubRealtime.swift, +Social.swift, +Streaming.swift, +Concierge.swift, +Taste.swift, plus callers needed to understand lifecycle. No hosted query, app execution, deployment or code edit. Source risks below are not measured production costs, traffic, latency or privacy incidents. Paths are relative to /Users/max/Kuro-Anime-App.

Complete register

evidence/social-call-register.csv enumerates **55 sites**: **47 request sites** (33 RPC, 13 PostgREST SELECT, one Edge Function POST) and eight realtime configuration/lifecycle sites (channel, five stream declarations, subscribe, unsubscribe). A realtime stream declaration is not an independent database query. Each row records file/line, enclosing function, method, endpoint, read/write classification, projection, bounds, pagination, repeat/cache behavior and assessment. Every literal .from, .rpc, .functions.invoke, channel/change/subscribe/unsubscribe occurrence in the six assigned files is included. There are no direct auth or storage calls in these files. Caller locations are retained in /tmp/kuro-social-callers.txt (includes some irrelevant encode matches; register is manually classified).

RPC results have no client-side column projection. Client limit parameters describe requested bounds, not proof of server-enforced bounds. RLS and deployed RPC definitions require the separate backend audit. Most writes are explicit user actions; two enrichment enqueue paths also write while browsing.

Highest-value findings

1. Club realtime can refresh a whole bundle for unrelated delivered changes

+ClubRealtime.swift:21–47 configures five AnyAction streams. Only club_polls and club_messages have club_id filters. club_rail_items, club_votes, club_rail_item_reactions have no client table filter. All five schedule a forced full refreshClubBundle after 500ms (:107–114). RLS/publication rules can limit events; source alone does not prove whole-table traffic reaches this client.

One relevant reaction can therefore cause a nested club bundle read rather than an isolated reaction update. Multiple events inside500ms coalesce; events spaced farther apart can each cause a refresh. Own vote already explicitly refreshes (ClubDetailView.swift:433–434), and own realtime event can schedule another. Club chat request methods have no current external Swift callers, but its message stream still causes refreshes.

Positive: feature guard, same-club guard, one service-owned channel, listener cancellation, debounce cancellation and unsubscribe exist (+ClubRealtime.swift:12–17, :92–103). Detail subscribes after load and unsubscribes on disappearance (ClubDetailView.swift:145–164). No explicit scene-background suspension appears in that view. A failed subscribe leaves clubRealtimeSubscribedId set because catch only logs (+ClubRealtime.swift:16, :83–89); a same-club retry can be skipped until cleanup.

Planning decision: retain club-only social, scope invalidation to the active club and changed resource, and make a single owned observation/lifecycle controller. Do not remove the top swipe interaction to solve request ownership. Measure delivered-event count, refresh ratio, payload bytes and concurrent channels before sizing savings.

2. Realtime refresh may not update the displayed club state

ClubDetailView.swift:36 owns a local @State bundle, populated by its own loadBundle (:365–401). Realtime refresh updates SupabaseService.clubBundleCache (+Clubs.swift:297–298) and discards the returned bundle (+ClubRealtime.swift:113). No ClubDetail .onChange/subscription consuming that cache change was found. This is a source integration risk: paid-for refresh work can fail to affect the visible view. Runtime verification must establish actual observation behavior. Use one published/readable club state source or explicitly deliver refreshed data to the view.

3. Club opening deliberately makes a lightweight and a full request

Initial loadBundle starts an unstructured loading-snapshot task (ClubDetailView.swift:368–373) and awaits full fetchClubBundle (:376). Even a full-bundle cache hit does not suppress the loading-snapshot RPC. The snapshot has no service cache/dedup. Full bundle has a good300s TTL and per-club inflight map (+Clubs.swift:276–299); force refresh bypasses TTL but still joins an inflight request.

fetchMyClubs has no TTL/inflight and catches any error from fetch_my_clubs_loading by calling fetch_my_clubs_enriched (+Clubs.swift:188–207). Callers: Clubs initial/foreground/pull (ClubsView.swift:46–78), Profile task (ProfileView.swift:74–76), title detail when empty (ClubActivitySection.swift:71–75), add-to-club sheet and create/join/leave. Empty membership is not distinguished from never loaded in the title-detail guard, so zero-club users can requery on each title opening. The fallback may be compatibility support; do not remove it blindly, but narrow fallback triggers to known capability mismatch and cache successful empty results.

4. Availability detail refresh repeats the same status read

Anime refreshLinks (AnimeDetailView.swift:1628–1648) calls status, V2 batch for one title, and enqueueAvailabilityRefreshIfStale. That helper calls status again (+Streaming.swift:236), then queues if absent or >=30days stale, then the view calls status a third time if queued. Manga mirrors this at MangaDetailView.swift:1395–1415.

When flag enabled and not within successful-enqueue cooldown, this branch alone requests **3 reads for a fresh title** (status twice + V2) or **4 reads +1 enqueue write for a queued title** (status three times +V2). The legal-watch/episode-link reads preceding these are outside this partition. These are source-path counts, not observed requests. Anime ties refresh to progress (AnimeDetailView.swift:1528–1529), so +1 progress can repeat the path; manga uses title ID (MangaDetailView.swift:1298).

Good: ten-minute enqueue cooldown after successful write (+Streaming.swift:230–233, :257), stale threshold, and flag checks in callers. Gap: fresh/no-op status is not cached, status is not passed into the enqueue helper, and no inflight sharing. Prefer one availability state query with freshness metadata, and reuse its result when deciding whether to enqueue.

5. “Cache” often stores results without avoiding future requests

- Provider V2 (+Streaming.swift:147–179) always calls backend for supplied items, then fills caches; no TTL, cache-hit filtering or request dedup. Its key is media type/id even though results depend on audio/subtitle/includeUnknown parameters; proposed persistent Anime/Manga scope and language preferences need explicit cache keys.
- prefetchProviders similarly reissues the entire batch (:200–223). Collection starts V2 and legacy provider prefetch for the same feed items under the streaming flag (EditorialCollectionView.swift:739–745), up to80 passed by callers. Both write providerCache, so parallel completion order also deserves validation.
- Friend-count cache stores only returned rows (+Social.swift:104–110) without TTL or cache-hit filtering. Missing rows are not explicitly set to zero, so a formerly nonzero count may remain stale unless cleared elsewhere.
- Club shared-provider fetch always runs; it writes a cache but does not consult it (+Streaming.swift:374–399).

Each prefetch family has one global task slot. New calls cancel prior tasks for the whole family, not a title/scope key (+Social.swift:95; +Streaming.swift:188, :201). This helps suppress obsolete work but different mounted surfaces can cancel each other. Cancellation may not reverse server work already issued. V2 lacks the post-response cancellation guard present in the legacy/friend implementations.

6. Invisible work is relevant with the retained swipe navigation

ContentView.swift:575–600 mounts current page and immediate neighbors, and also any previously mounted page (distance<=1 || mountedSections.contains). Consequently mount/.task does not mean selected/visible. At default Discover, adjacent Taste and Browse may load. Later, mounted Clubs can receive foreground notifications while not the selected page; its handler fetches clubs then checks notifications (ClubsView.swift:75–79). This is a source risk, not measured SwiftUI lifecycle execution.

Taste fetches12 on appearance, refills when queue<=3 (TasteDeckView.swift:68, :133–152, :244–257), with inflight and exhausted guards. These bounds are good. A new Home separate from Discover should explicitly pass active surface + persistent media scope to data loaders, preserving swipe/neighbor views while avoiding unnecessary requests. Do not assume unmounting or tab-bar replacement is required.

7. Some batched social work has no visible consumer

Collection prefetches friend counts (EditorialCollectionView.swift:734–735) but its custom grid/list components do not read friend counts. Shared catalog cards do, under the social flag. Discover prefetch includes up to100 concatenated titles before deduplication (EditorialDiscoverView.swift:561–571) and omits explicit daily-feature/AiringToday/BecauseYou/Shelf/HiddenGem IDs unless they overlap other rails. Batch reads are preferable to per-card calls; the next step is visible-ID dedup and matching consumers, not additional per-card RPCs.

Under club-only friends and activity opt-in direction, keep identity/profile available without fetching activity implicitly. Gate social counts/activity at consent and visibility boundaries as well as global rollout. Existing service methods themselves generally rely on caller gates.

8. Detail credits overfetch relative to initial display; entity works are unpaged

Five forward-join requests (+Clubs.swift:488, :511, :534, :557, :580) filter one media ID and use explicit projections, but fetch all matching entities including descriptions; no limit/order. Detail executes them only under credits flag (AnimeDetailView.swift:234–238, MangaDetailView.swift:232–235). Cast initially previews8 and limits displayed selection10 (CastSection.swift:11–30); credits curate first2 (CreditsSection.swift:43). Full descriptions would be better fetched on entity detail when needed if payload measurement justifies it.

Reverse entity-work queries (+Clubs.swift:617, :645, :678, :711, :718) have narrow card projections and adult filters, but no stable sort, limit or pagination. Prolific staff/studios can grow. A character sheet issues anime+manga reads in parallel even if future app-wide scope is one type. Good:120s TTL, approximate50-key trim, and source-card mapping; forward joins also cache successful empty results and approximately100 keys. No inflight dedup for these entity requests.

9. Enrichment writes happen during discovery

prefetchMediaRelationRefreshRequests dedups, caps12 and sends serial enqueue RPCs in an unowned Task (+Clubs.swift:810–831). Discover supplies hero +4 current-season +4 trending +4 manga candidates (EditorialDiscoverView.swift:575–588), Concierge supplies8 recommendation candidates (ConciergeView.swift:1086–1093). Per-key12h cooldown begins after successful enqueue (+Clubs.swift:775–792) and is session-local. This prefetch path does not test current ladder strength; detail's enqueue...IfNeeded does (:800–807). Good cap and cooldown, but cold sessions can issue up to12 queue writes merely from Discover load. Backend idempotency/dedup and downstream execution cost belong in the live audit; do not call these12 upstream provider refreshes without evidence.

10. Taste retry count is not a hard retry limit

Local model retains failed pending signals after failure count3 and merely shows a warning (TasteDeckView.swift:195–222); future commits/appearances can retry them again. The queue is memory-only and not user/scope-keyed. Good: batch fetch inflight guard,12-item result request, error distinct from exhaustion, explicit retry UI and undo. Recommendation: bounded delivery policy and identity/scope reset in a Taste data controller; do not change optional taste into mandatory setup. Local source improvements differ from approved welcome reference, as earlier comparison documented.

11. Profile and auth boundary responsibilities need clearer ownership

Profile currently warms clubs on appearance and, under availability flag, sequentially reads service registry, user subscriptions and operational queue aggregate (ProfileView.swift:74–79, :337–343). Bootstrap also loads registry/preferences (SupabaseService.swift:944–949). Registry/realm metadata are small-catalog cache candidates, not proven large costs. In proposed identity-first Profile, place operational diagnostics behind diagnostics and fetch social activity only on opt-in/action.

+Clubs.swift:246 defaults new clubs to sharing status; this differs from a future identity-only/activity-opt-in policy and requires an explicit product/migration plan, not a label change. Base resetUserState (SupabaseService.swift:861–880) clears lists/entity caches but does not visibly clear myClubs, club bundle/inflight maps, friend counts or streaming preferences. Assigned extensions do not add identity reset. Sign-out calls stopRealtimeSubscriptions, whose body at SupabaseService.swift:2567–2577 stops only the separate realtimeChannel/realtimeListenTasks, not clubRealtimeChannel; club cleanup is delegated to detail disappearance. Parent/client owner should verify service lifetime and central cleanup before concluding anything cross-account; this audit does not demonstrate a leak. Global notification/per-club seen timestamps are not keyed by user in +Clubs.swift:212, :406.

12. Import is bounded and reuses preview work

One shared Edge POST at +Concierge.swift:123, used by preview and import modes. maxItems is clamped25..400 (:106), default200. Coordinator passes preview text to confirmation (ConciergeAniListImportCoordinator.swift:81–82, :103–108) so backend can reuse parsed input. This endpoint returns import text; actual list application is another flow, so CSV labels this read/orchestration, not a direct list-write claim. Service spawns an unstructured Task and awaits it (+Concierge.swift:119–125); cancellation propagation on dismissal is not explicitly wired. Avoid duplicating import logic for Anime/Manga/Home; pass persistent scope while allowing an explicit “both” import choice if desired.

Request bundles worth measuring

Interaction	Source-derived request shape in this partition	Good boundary / unresolved part
Open club first time	Loading snapshot + full bundle; optional shared-provider RPC + provider batch; realtime subscription	Full bundle cache may eliminate its read; snapshot still launches; server bounds unknown
Vote	One mutation + explicit full refresh + possible realtime-triggered refresh	Per-poll inflight/optimistic UI good; measure duplication
Open anime credits cold	Characters + staff + studios	Flagged; cache avoids repeats; unbounded entity count
Open manga credits cold	Characters + authors	Same
Open entity works cold	One SELECT; character two SELECTs	120s TTL; no paging
Submit/delete title comment	One mutation + full title activity read	Explicit action; delete reload also occurs after failure
React to title comment	One toggle + full title activity read	No per-comment inflight suppression in current caller
Collection metadata warmup	Count batch; if availability on V2 batch + legacy provider batch	Up to80 passed; visibility/consumer overlap unresolved
Show Leanings	Profile RPC + realm table SELECT in parallel	Per-sheet didLoad guard; service has no TTL
Preview + confirm AniList	Same function endpoint once per step	Cached text and cap; downstream apply excluded

Modularity for the future plan

Keep the current observable pattern and top swipe. These are responsibility boundaries for incremental extraction, not a proposal to replace the backend stack or create parallel systems:

1. **App scope/session owner:** persistent Anime/Manga, selected Home/Discover/etc., identity epoch, opt-in privacy; cancels/clears user-scoped work on identity change. Visual neighbor mounting must not imply request eligibility. 2. **Club data owner:** membership list, detail snapshot/full data, mutations, targeted invalidation, one realtime lifecycle. Shared state must drive both detail and previews. Friends remain derived from club permissions. 3. **Title social owner:** permission-aware friend activity/comments/count batches with visible-ID dedup and scoped TTL. Treat “tracking,” “completed,” and progress as different data, respecting opt-in. 4. **Availability owner:** one consistent cache key including media type/region/language semantics, status+providers contract, stale enqueue decision and bounded background maintenance. Do not expose operations aggregates as default profile content. 5. **Catalog people/relations owner:** move entity/ladder methods out of the already large Clubs extension; retain shared cache implementation, add paging only where needed. This is separation by responsibility, not redundant services. 6. **Taste owner:** bounded batches/signals/retries/profile metadata, active-surface and media-scope aware; shared by Home/Discover without independent duplicate warmups. 7. **Import coordinator:** keep preview/reconcile/apply/undo ownership centralized and explicitly user-triggered. Auth animation remains confined to auth UI, not a reason to refetch data in Home/Profile.

Before implementing: assign request/payload budgets per interaction, verify server result bounds/RLS/idempotency, record actual call counts and bytes for cold/warm/reconnect/rapid-swipe scenarios, and test active-surface/flag-off/identity-change cases. No monetary estimate is justified from this source inventory alone.

Backend functions and ingestion audit

Kuro edge-function calls, workload and metadata audit

Source-only audit, 26 September 2026, repository HEAD 697a5bc4548cc6b6492113f8481fb8bc8403030c. No production queries, writes, requests, deployments or runtime benchmarks performed by this agent. This report describes current local source; deployed function versions/payloads/schedules need the parent's live audit. Source call counts are not execution counts. SQL definitions are owned by the separate SQL audit.

Coverage and register

evidence/edge-call-register.csv contains **229 source call sites** across all **16 local edge-function index.ts files**, plus the shared TypeScript file checked for calls. Breakdown: **146 table/query-builder calls, 56 RPC calls, 13 HTTP fetch calls, 10 Auth calls and 4 Storage calls**. One Storage site, getPublicUrl, constructs a URL locally and is **not** a network request. auth-callback has no server-side DB/RPC/Storage/fetch sites in local source; the other 15 functions contribute register rows.

Every lexical .from(/ .rpc(call excluding JavaScript Array.from was reconciled against the register; no unmatched sites remained. Each row contains exact file/line, operation, table/RPC/endpoint, projection, filters/bounds, frequency/concurrency, retry/checkpoint behavior, source excerpt and the source-only evidence boundary. Dynamic anime/manga table names and variable HTTP endpoints are identified. The register is call-site exhaustive, not an exhaustive SQL query-plan or SDK-internal HTTP trace.

Function	Sites	Workload character
bulk-import-anime	26	Paged upstream reads, five titles concurrently, many per-entity writes
bulk-import-manga	28	Same pattern, duplicated author/staff persistence, optional placeholders
concierge-apply	23	Up to 100 concurrent item chains, reconciliation and undo logs
concierge-import-anilist	3	Auth/rate-limit plus serial AniList pages
concierge-parse	10	Nested concurrency: items and their search variants
concierge-recommend	32	Multiple rail builds and batched hydration, optional narration
concierge-resolve	10	One packed model request with budget bookkeeping
concierge-retrieve-assist	5	Cache, bounded candidate RPC, cache write
concierge-retrieve-feedback	3	Auth, rate-limit, one feedback insert
concierge-undo	15	Sequential read/restore chains
delete-account	13	User deletes plus global orphan-club scan and storage cleanup

Function	Sites	Workload character
manga-chapter-enrich	28	Sequential candidates, external matching, chapter batches
manga-source-review-action	5	Review/mapping writes plus optional one-title enrichment
mirror-images	21	Serial images, uploads and state writes
realm-describe	7	Three reads/title and model calls, one final batch write
auth-callback	0	HTML redirect handler

Counterevidence to a blanket “huge queries everywhere” diagnosis: the edge table calls generally use explicit narrow projections. The only literal select('*') sites found are manga chapter/volume **HEAD exact counts**, which do not transfer all row bodies (bulk-import-manga/index.ts:418, :437). The larger risks here are query multiplication, SQL work behind limited RPC results, repeated unchanged writes, incomplete work budgets and unnecessary metadata reprocessing. A LIMIT 10 response does not prove a cheap SQL plan; a HEAD count can still require database work.

All paths below are relative to /Users/max/Kuro-Anime-App/supabase/functions/ unless explicitly labeled migrations.

Highest-priority findings

E1. Recommendation evaluates more rails than it displays and duplicates one

concierge-recommend/index.ts:2406 caps preferred secondary choices at five. It builds the primary (:2408), builds the first fallback secondary (:2435), then builds all preferred secondaries again in Promise.all (:2448). The first secondary is consequently fetched twice when included in that array. Output is only two rails (:2392). This is a verified source duplication; its production frequency and latency are unmeasured.

Each curated build can fetch anime and manga separately via curated_rail_cards (:2166, :2169), then fall back to algorithmic fill when too few titles survive (:2366). The algorithmic path uses recommend_ids_premium with limit 50 (:1535) and explicit media/boost/tag hydration queries (:1572, :1576, :1582). Some premium candidate context is shared elsewhere in this function; do not multiply every rail by every possible hydration call without tracing its path. Similarity separately calls recommend_ids_similar_to_seeds (:2269) and hydrates by type (:2288).

Plan: memoize rail-build promises by mode/type/constraints within a request, remove the guaranteed duplicate, then use a bounded secondary selection strategy that preserves diversity. Record rail candidates attempted, SQL calls, rows hydrated, time and selected coverage. Cache shared config/editorial mappings by version; avoid global user-result caches that omit user exclusions or state. Set a hard work budget and return a sound two-rail result without exhausting every alternative.

E2. Parse and apply can produce request bursts despite bounded response sizes

Parse accepts up to 5,000 characters (concierge-parse/index.ts:1058), splits text without a separate item cap, runs every item concurrently (:1135), and executes all initial search variants concurrently per item (:1174). Low-confidence items add up to five denoised queries (:1018, :1215). Candidate output is capped at 3–15 per item (:1062), but this does not cap aggregate request fanout. There is also one alias lookup and potentially one existing-list lookup per item (:1146, :1289, :1308).

Apply caps input at 100 items (concierge-apply/index.ts:148) but starts all 100 mutation chains concurrently (:180). A normal applied item may need existing-row read, list write, alias read, alias write and session-item insert; completion/caught-up handling adds title and sometimes last-episode lookup. This is approximately 5–7 calls per applied item on those branches, not a measured runtime average. No deduplication guarantees unique media IDs

before the comment claiming independence. The read/check and write are separate HTTP operations, so the “TOCTOU protection” comment is not an atomic compare-and-set guarantee.

Plan: bound item concurrency with a small configurable worker pool; batch alias and existing-list reads by media type; deduplicate identical normalized searches within the request; provide a structured provider-ID import path that avoids title-search fanout. Move apply plus undo/audit capture into a bounded transactional RPC with an idempotency key and expected row version. Preserve per-item conflict results. Test repeated IDs, concurrent user edits, partial failure and retries. Do not merely increase function timeouts.

E3. Named import tracks do not change which upstream catalog rows are selected

Both bulk import queries always select media(sort: POPULARITY_DESC, type: ANIME/MANGA) (bulk-import-anime/index.ts:283; bulk-import-manga/index.ts:302). Presets named airing_or_releasing, recent_updates and catalog_backfill change run sizing/cursors, not upstream status filters or ordering. Therefore a configured “airing” track alone does not establish airing-first freshness. Parent should compare actual cron payloads and freshness distributions before drawing a live conclusion.

Anime’s scheduleSafe defaults to false (bulk-import-anime/index.ts:130), ignoring the preset’s scheduleSafe value; manga correctly inherits it (bulk-import-manga/index.ts:143). Most page/retry/budget request parameters have no global hard clamp outside the schedule-safe branch. Time-budget checks occur after a page and only in the runToEnd condition (anime :224, manga :246). A non-runToEnd request can exceed the nominal budget while processing its requested pages. Manga’s outer loop also remains bounded by pagesPerBatch (:214) even when runToEnd is true.

Plan: give each track an explicit query contract: popular catalog refresh; currently releasing schedules; recently updated metadata; deterministic catalog backfill. Persist the query/version with its cursor so incompatible cursor state cannot be reused. Apply hard page/item/deadline limits to all requests, including manual invocations. Check the remaining deadline before each network call and batch, not just after a full page. Keep source scheduling and metadata enrichment as separate jobs.

E4. Bulk ingestion contains large per-title write amplification and nontransactional replacement

Both importers run five titles in parallel (bulk-import-anime/index.ts:214; bulk-import-manga/index.ts:221), but links, tags, people and relation edges are processed with individual serial DB calls inside each title. They first look up the title then upsert it. Even unchanged titles do a timestamp update (anime also refreshes schedule fields; anime :320, :341; manga :340, :349).

For an updated anime, a useful source call-count model is:

2 title calls + L external links + (1 delete + E episode inserts when present) + 2×(studios + tags + characters + staff) + 1 relation delete + up to 2×R relation edges.

Here R counts valid edges whose target lookup is attempted; unresolved targets omit the write. This excludes run bookkeeping, triggers and retries. It is an explanatory formula, not a usage forecast. Manga has analogous calls, plus optional placeholder counts/inserts; its same staff nodes are separately written into authors/manga_authors and staff/manga_staff (bulk-import-manga/index.ts:453, :501).

Anime deletes all existing episode rows before inserting upstream streaming episodes one by one (bulk-import-anime/index.ts:415, :429). A failed insert or interrupted request can leave partial coverage and erase richer independently enriched fields. Episode numbers are extracted from English “Episode N” or fall back to array position (:418), so source ordering is not a reliable canonical number. Relation replacement similarly deletes then resolves/inserts edges (anime :534, :553, :559; manga :557, :576, :582). Targets missing locally are dropped rather than queued for later resolution.

Plan: batch title-ID resolution per page, batch shared entity upserts and join rows, compare source fingerprints, and write only changed fields. Replace episode/relation sets transactionally with provider ownership boundaries, or merge source-owned keys while retaining richer data. Queue unresolved relation references by provider ID. Use

source-specific episode IDs/labels when available and model uncertain numbering explicitly.

E5. Checkpoints and locks do not provide a durable import guarantee

Cursor writes occur after the processing loop (bulk-import-anime/index.ts:228; manga :249), not after each committed page. Caught per-title errors increment results.errors but still advance the page and mark the run success. A hard timeout can replay a run's earlier writes on retry; a partial item error can be skipped until a later full sweep. Cursor reads occur before acquiring the lock, and locked/skipped paths write cursor state. Concurrent launches can therefore retain stale cursor knowledge; exact race impact depends on schedules and table semantics.

acquireImportLock returns true on RPC errors/exceptions in both importers (anime :618, manga :641), manga enrichment (manga-chapter-enrich/index.ts:1954) and mirror-images (:471). The system can run concurrently precisely when lock health is uncertain. HTTP fetches generally lack AbortController deadlines. Bulk retry helpers use three default retries with exponential delay but no Retry-After handling or jitter (anime :579, manga :602). They treat any HTTP-200 JSON as success, and missing GraphQL data becomes an empty page; GraphQL errors may therefore look like end-of-catalog. The user-list AniList import explicitly checks GraphQL errors (concierge-import-anilist/index.ts:162), providing an existing pattern to reuse.

Plan: fail closed on coordination errors, acquire/claim before reading progress, use a lease token/fencing value, commit checkpoint with page writes, preserve a failed-item retry queue, and classify complete/partial/failed runs honestly. Separate retryable provider errors from invalid payloads and permission errors. Check GraphQL errors before advancing/wrapping. Give each request a deadline shorter than its lease and respect Retry-After. Avoid broad replay as the normal recovery mechanism.

Manga chapter enrichment: useful safeguards and remaining costs

The enrichment worker has better bounds than bulk ingestion: default 20/max 200 candidates, default 45-second soft budget/max 120 seconds (manga-chapter-enrich/index.ts:6, :287); candidates are processed serially (:223); per-candidate state is persisted (:252); title searches are cached within the run (:222, :1703); requests are paced at default 3 RPS with 429 cooldown and retries (:21, :1882, :1920). These are useful foundations to preserve.

However:

- Candidate selection calls get_manga_chapter_enrich_candidates with a compatibility fallback on **any** error (:433, :442), potentially repeating expensive SQL for transient failures. Narrow fallbacks to known schema/signature errors and retire them after verified parity.
- Mapping reads/writes and review persistence also have compatibility retries (:598, :608, :785, :802, :838, :847). Make fallback usage visible rather than silently doubling work indefinitely.
- A title can cause several title searches (all aliases are considered), entity verification and per-language aggregate reads. Collision tie-breaking can request aggregate coverage for every candidate (:1253). The soft budget limits the normal loop but does not abort a hung fetch or stop a Retry-After sleep from exceeding the deadline.
- Chapter insertion reads existing numbers in chunks of 400 (:1800) then inserts missing rows (:1824). A duplicate-key race falls back to one insert per row (:1827), producing up to 400 extra calls per conflicted chunk. Use a conflict-safe batched insert/RPC and report inserted count.
- It performs exact per-title chapter counts (:1837); source review should establish the supporting (manga_id, number) index and whether that count can be maintained atomically with ingestion. Do not replace a small per-title count with a whole-table client scan.
- updateMangaSyncFields takes max provider chapter number and writes it into aggregate manga.chapters (:1855). This conflates maximum indexed/available chapter with final published total and can undo the product distinction required for ongoing manga.

SQL handoff: the SQL reviewer found a global chapter GROUP BY before candidate LIMIT in supabase/migrations/20260221173000...:8. That is a source plan risk, not measured duration. Ask for a bounded current query plan and table statistics; prioritize selecting due title IDs before aggregating their chapter counts, or use maintained per-title summaries if measurements justify them.

Mirroring and descriptor generation

Images

mirror-images first drains 50 priority assets (:114) with per-type batched row lookup (:133), then default 200 candidates per requested media group (:47). Groups/rows/assets are serial. A title can cause three fetches, three storage uploads, three state upserts and a final title-row update. Even missing/already mirrored assets write state (:413, :427). It uses explicit narrow projections and remote-only candidate filters (:210, :246), so this is not a full catalog image download on each call.

Remaining risks: no per-fetch timeout or response byte cap (:359, :364); mutable remote-only candidates combined with offset pagination (:216, :252) can skip work as earlier rows leave the result set; the lock is keyed by media-type combination and offset (:61) while priority draining is global, so different batches may touch the same priority assets. overwrite is recorded but storage upload always uses upsert:true (:366). The filename has no source revision/hash while the cache-control requests long immutable caching; changing bytes at the same URL needs careful cache semantics.

Plan: claim due assets atomically, track retry_after/attempt count and source fingerprint, use bounded concurrency and byte/deadline budgets, preserve source URL separately from mirrored URL, and avoid writing unchanged skipped state every run. Prefer stable queue/keyset progression over offsets into a shrinking candidate set. A source change should yield a new versioned URL when long immutable caching is used.

Realm descriptors

realm-describe supports both anime and manga, one title or a batch capped at 25 (:465). It makes three per-title reads: explicit base fields (:240), all title tags then locally keeps eight (:250, :260), and realm membership (:263). Move top-tag order/limit into SQL if the metadata cardinality/plan warrants it. Small realm_meta is read once (:101).

Model calls permit two empty-response attempts, a 429 fallback model, and two validation attempts (:327, :348, :378). Successful outputs are buffered for the whole batch then written together (:471, :499), so a timeout late in a batch can lose already paid generation results. No edge-owned queue claim/lease or total time budget was found. External worker ownership must be verified before claiming concurrent duplicate generation happens in production.

Plan: bounded claims, source/model/prompt fingerprints, write progress in small durable batches before the deadline, retry only failed validation/provider outcomes and expose cost per accepted descriptor. Reuse unchanged descriptors. Keep deterministic routing independent of narration; note that local concierge-resolve does invoke Groq for disambiguation and realm generation emits structured classification-like fields, so current source differs from a blanket “Groq narration only” description.

Other endpoints and correctness that can create waste

- **AniList user import:** default 200/max 400 total entries, pages 10–50, types sequential (concierge-import-anilist/index.ts:250, :283). No upstream retry/deadline. It fetches ratings/IDs/update time then turns records into title text (:89); only eight preview samples keep metadata (:188). Confirm reuses that text. Structured ID import avoids repeated fuzzy catalog searches and preserves fields; use resumable pagination for full lists.
- **RAG retrieval:** bounded 10-candidate RPC and a one-hour cache are good (concierge-retrieve-assist/index.ts:45, :262). The cache key receives normalized query, locale and constraints but omits mediaType (:77, :232), while the RPC uses mediaType (:264). Anime/manga results can be cross-served for otherwise identical keys; test and version corrected keys. Cache errors currently fall through; error responses may become cached empty results.

Distinguish legitimate no-match from backend failure.

- **Resolve/narration:** reserve/finalize budgeting exists and narration limits output to 260 tokens/eight titles (concierge-recommend/index.ts:2648). Resolve packs 20 items/max ten choices (concierge-resolve/index.ts:204). Model fetches lack deadlines; resolve's outer error catch returns without finalizing reserved budget (:296). Some rate-limit/budget calls inspect .data.allowed without checking .error; fail-closed claims require explicit error handling, since SDK errors usually resolve as objects rather than throw.
- **Undo:** reads all items in one session (concierge-undo/index.ts:100) and restores sequentially. Apply caps each new session at 100, but undo query itself has no explicit bound/pagination for legacy sessions. It marks the session cancelled even if individual undos fail (:251). Progress/status checks do not compare rating/notes, which can allow overwriting subsequent edits in those fields. Use an atomic expected-version restore and retain retryable per-item failures.
- **Account deletion:** delete-account/index.ts:119 scans all creatorless clubs then counts memberships per club, unrelated to the size of the deleting user's footprint. Scope cleanup to affected clubs or run a separate bounded orphan-maintenance job. Storage .list(userId) has no explicit pagination (:143), so completeness must be verified; a single default page is not proof of complete removal. Keep deletion semantics/authorization unchanged while making work bounded.
- **Review action:** mapping upsert, review approval and downstream enrichment are separate (manga-source-review-action/index.ts:105, :124, :211). The response already distinguishes mapping-saved/status-failed. Make approval idempotent and do not require repeated expensive enrichment when only the review marker needs retry.

Metadata fetched, retained, discarded and still missing

Source area	Verified useful data already retained	Waste/gap or opportunity
Anime core	Titles/aliases, provider IDs, partial start/end dates, format/status, genres, source, adult flag, scores/popularity, next episode/time	hashtag, title.userPreferred, cover extraLarge, timeUntilAiring are requested but not persisted. Drop redundant fields unless there is an explicit product need; timeUntilAiring is derivable. Do not store every source field merely because it exists.
Anime schedules	AniList next episode/time are written on fresh and unchanged paths	Track query is not releasing-filtered. episodes falls back to next episode minus one, conflating aired-so-far with final total (bulk-import-anime/index.ts:314). Separate these semantics for progress and upcoming UI.
Streaming episodes	Title, thumbnail, duration, stream URL/site	No future schedule date from this query; delete/reinsert destroys independently enriched dates/metadata. The fallback index is not authoritative episode identity.
Manga core	Titles/aliases, provider IDs, chapter/volume aggregate counts, partial dates, external links, creators/tags	next_chapter_number and next_chapter_at are explicitly written null in every stale full import (bulk-import-manga/index.ts:369). A future enrichment writer would be erased unless ownership is separated. ISBN, edition language, publisher release schedule and volume dates are not supplied by this AniList query.
Manga source aggregate	Existing mapping identity and integer chapter coverage	Aggregate language probes return the first language with chapters, then optionally all-language fallback (manga-chapter-enrich/index.ts:1730). Persisted chapter rows keep only number/generic title (:1818), dropping language/provenance and decimal/extra labels. This cannot support “latest official chapter in my language” or forthcoming edition alerts.
Relations	Supported source/adaptation/prequel/sequel/side-story/spin-off local-ID edges	Query fetches target titles, format and cover but local write only needs ID/type/relation; remove unused payload or deliberately use it to stage unresolved referenced targets. Missing local target currently means lost relation until another successful sweep.
People/tags	Compact top-ten characters/staff, roles, descriptions, tag rank/spoiler flags	These fields can support credits and explanatory discovery. Manga staff is duplicated into authors and staff; preserve role semantics and reduce repeated writes. Top-ten is an intentional partial catalog, not complete credits.
External links	Site, URL, language, color	Already useful for official destinations, but not evidence of country/audio/subtitle-specific availability, exact edition or affiliate eligibility. Version/source identity and stale-link removal rules are needed before richer claims.

Source area	Verified useful data already retained	Waste/gap or opportunity
Enriched copy/images	Separate state tables and cached/mirrored content exist	Both bulk upserts reset synopsis state to pending (anime :385, manga :387) and send upstream image URLs again. Unless a database trigger/merge layer protects them, unrelated source updates can cause unnecessary re-enrichment/remirroring. Verify with SQL owner before claiming measured churn. Use per-field source hashes/ownership.

The right principle is “fetch and persist what a named user feature or operational quality check needs.” New metadata should have source, timestamp, confidence/date precision, ownership and downstream consumer. A larger catalog count does not itself improve the user's tracking or discovery experience.

Executable work sequence for the parent plan

1. Establish a cheap baseline. Use existing logs/pg_stat_statements/edge run summaries first. Capture counts, p50/p95 duration, retries, rows touched/returned and bytes for representative requests. Distinguish warm cache, cold cache and error branches. Avoid repeatedly scanning enormous tables or fetching row bodies to compute metrics. Current source identifies targets; it does not justify an arbitrary database upgrade.

2. Remove known duplication without changing ranking. Memoize recommendation builds and shared metadata; bound parser/apply concurrency; batch ID/alias/current-state reads. Use fixed fixtures for anime, manga, mixed scopes, constraints and large imports. Require identical recommendation eligibility and explicit conflict outcomes while reducing call count.

3. Make ingestion bounded and resumable. Hard-clamp request sizes, enforce network deadlines and provider budgets, fix actual track query semantics, fail closed on locks, persist checkpoints with writes and retain failed items. Test interruption after the first committed page and show resume without losing or needlessly redoing it. Expose complete/partial/failed states and a finite next action.

4. Batch and isolate writes. Provider adapters produce typed source records; a bounded page writer resolves IDs/upserts changed rows; separate episode/relation/metadata jobs respect ownership. Use small RPC modules by purpose, not a giant function containing all catalog operations. Keep auth/rate-limit/deadline/telemetry helpers shared and provider-specific mapping logic separate. Preserve the existing Swift/API contracts until independently migrated.

5. Add product-relevant metadata. Fix original-total versus available-progress semantics; preserve language/provenance; stage edition-level release/ISBN identity; expose current next-airing data only where freshness is verified. Do not launch “where to watch/read” or manga release promises from empty provider/date columns. Run tiny deterministic source samples before widening coverage.

6. Roll out with budgets and rollback. Baseline one representative user request and one small job per family, then broaden only if the bounded criteria pass. Define hard per-invocation title/query/byte/time limits; alert on repeated no-progress runs, stale leases and retry exhaustion. Pause the job/feature through explicit controls rather than letting an endless retry loop compete with interactive traffic. Deployment and hosted writes remain the parent's separately authorized phase.

Suggested acceptance evidence: recommendation no duplicate mode builds; parse/apply maximum observed concurrency matches configured pool; no cursor advance for unrecorded failure; retry after timeout resumes committed work; unchanged metadata does not reset enrichment; chapter/episode updates preserve independent richer fields; no unsupported dates or language availability invented; budget/error paths return actionable status; both anime and manga fixtures pass. Performance targets should be set after the baseline, not fabricated in this source report.

SQL coordination and live boundaries

The SQL reviewer reported search_titles uses indexed trigram candidates and curated_rail_cards uses narrow rail lookup—retain these strengths. recommend_ids_premium and chapter-enrichment candidate selection have potentially expensive aggregations that warrant bounded plan verification. Similarity has a precomputed serving path; do not attribute every request to its legacy fallback scorer. Exact migration evidence belongs in the SQL register.

Parent supplied earlier live catalog counts/future-date sparsity and rating constraints. Those are valuable separate live evidence but were not re-measured here, and are not substituted for deployed edge parity or query-performance measurements. This audit made no repository changes; only the requested /tmp report/register and a local /tmp generator were written.

SQL, schema and modularity audit

Kuro: SQL, schema and modularity audit

Read-only review, 26 September 2026. Repository: /Users/max/Kuro-Anime-App. No repository edits, migrations, hosted RPC executions, EXPLAIN ANALYZE, configuration changes or deployments were performed by this agent. Root collected the hosted metadata and historical statistics cited below.

Coverage and confidence

Scanned all 235 migration SQL files in filename order, indexed 140 latest function names, and mapped 49 literal RPC names called by the Swift app. evidence/rpc-review-register.csv records the callers and candidate latest definitions, plus supplementary SQL dependencies. This is a function-name index, not a complete SQL parser: overloaded signatures, guarded baseline captures, dynamic invocation and dependency ordering still require explicit review. Major hot paths were manually read. Earlier superseded implementations are not presented as current.

Compared the bodies of **16 current hosted functions** from evidence/live-serving-functions.json and live-functions.json against the latest local definitions after removing SQL comments and normalizing whitespace. **All 16 bodies match**: search_anime_page, search_manga_page, browse_anime_page, browse_manga_page, discover_bundle, fetch_club_bundle, fetch_club_bundle_loading, fetch_friend_activity_for_title, fetch_my_clubs_enriched, fetch_my_clubs_loading, collection_anime_page, collection_manga_page, collection_feed_page, count_friends_tracking, rebuild_media_realm_tier, rebuild_media_similar_titles. This is body parity for the sample, not proof of matching grants, owners, RLS, relation kinds, indexes, every function, or clean migration replay.

The largest measured cost is scheduled recommendation maintenance. The most credible interactive SQL concern from inspection is Search candidate generation/index alignment. Existing collection keyset paging, precomputed similarity, rail batching and typed anime/manga models are worth preserving.

1. Prioritize scheduled maintenance before adding more recommendation work

High confidence; measured historical operational cost, current definition confirmed. Root's live-cron.json, observed 2026-09-25 22:16:55 UTC, records the active five-minute similarity driver (job87): 287 successful runs in the preceding 24 hours, mean25.89s, max442.59s. Realm-tier refresh (job51) ran once in133.57s. These are cron execution durations, not user-request latency or current p95. The wider statistics window supplied by root (Aug4–Sep25 UTC) reports 15,060 similarity-rebuild calls averaging25,543ms and 871,431,881 temporary blocks written; separate counters should not be converted into a realtime throughput claim.

Current rebuild_media_similar_titles has an advisory transaction lock at extracted live SQL lines14–15, so it already guards overlapping work. It seeds missing state, then builds global recursive franchise components (lines73–105), synchronizes component/entry-point tables and evaluates entry-point windows (lines123–159), **before** selecting a bounded stale-seed batch and returning when it is empty (lines168–177). This matches supabase/migrations/20260807010000_leftover_repairs_v1.sql. The source explicitly says this ordering keeps franchise identity fresh even after recommendations converge. A simple early return would lose that behavior.

Decision: separate franchise/component freshness from similarity batch work, or gate that refresh on a reliable relation/cut-data revision. Preserve the precomputed recommendation store, advisory guard, editorial cut edges and newcomer entry points. Inspect the global recursive graph's actual cardinality/temp spill before tuning memory or adding hardware. Do not run full rebuilds from request handlers. Keep the daily realm tier rebuild under review; it is expensive but much less frequent.

Acceptance: empty stale queue plus unchanged relation revision takes a demonstrably cheap path; changed relations/editorial cuts update components within an explicit freshness SLA even with no stale seeds; mixed anime/manga identities and removed/cut edges stay correct; concurrent drivers cannot duplicate work; representative full backlog completes within budget without starving serving queries. Compare equal-duration before/after database CPU, temporary writes, run duration, stale backlog and foreground latency. Set a target after obtaining a controlled baseline rather than claiming the historical mean predicts it.

2. Restore a reproducible schema contract before expanding it

Confirmed source-chain problem, separate from the matching function bodies. supabase/migrations/README_baseline_capture.md:28–77 documents that the late baseline does not make the full chain replayable. Early migrations create indexes or alter tables before those objects are captured by 20260807000000_baseline_capture_v1.sql; lines58–72 list examples. The four hollowed February15 migrations contain only ; (README38–41). Standalone baseline tests recorded at81–96 do not prove whole-chain replay.

The same README42–45 says media_realm_tier is a materialized view in the chain but a plain table in production. Root's current relation catalog confirms relkind r, roughly56,959 estimated rows,6.7MB and no primary key advisory. Do not assume this advisory is a nonexistent-matview false positive. Determine its intended natural uniqueness and duplicate state before proposing a key.

A further dependency is **user_lists**: its CREATE VIEW exists in legacy_sql/08_create_user_lists_view.sql:6, while migrations only ALTER it to security_invoker (20260206150000_security_hardening_rls_and_views.sql:40; 20260206164200_security_hardening_rls_and_views.sql:14). No CREATE for this view was found in the235-file chain. Many RPCs use it. The legacy view's null/constant projection is not sufficient evidence of the live view definition; root should capture pg_get_viewdef.

Decision: establish one explicit supported fresh-bootstrap path plus an upgrade path, reconcile table-versus-matview kind and missing view dependencies, and compare actual grants/RLS/indexes as well as function text. Preserve applied migration history; do not casually rewrite already-applied files or move the baseline and assume that fixes all intermediate compatibility.

Acceptance: an isolated Supabase-shaped database reaches the expected schema from the documented path with ON_ERROR_STOP; repeated application is safe where designed; representative upgrade from the existing schema succeeds; no missing relations/functions, policy/grant differences or relation-kind ambiguity; unified list projection preserves both media's progress and privacy semantics. No production reset is required.

3. Make Search use a deliberate indexed candidate path

Current hosted body confirmed; plan/performance hypothesis remains unmeasured. Latest search_anime_page and search_manga_page are in 20260326234000_exclude_ancillary_anime_from_default_surfaces.sql:14,227. They normalize each title and a lower(trim(concat_ws(...nullif...))) title blob, rank exact/prefix/word/substring/full-text/fuzzy matches, and apply numerous OR alternatives. Fuzzy filtering computes greatest(similarity(...)) >=0.35. LIMIT50 bounds returned rows, not candidate work.

Existing trgm expression indexes (20260203235500_search_rpc_keyset.sql:8,10) use lower(coalesce(title_english,'') || ' ' || coalesce(title_romaji,'') || ' ' || coalesce(title_native,'')), a structurally different expression. Existing full-text GIN indexes (20260203223000_scale_perf_bundle_and_indexes.sql:25,33) include English/romaji/description_normalized, unlike the new title blob/native-title expression. These cannot be assumed to service the new expressions directly. A CTE itself does not prove materialization or a full scan; inspect actual plans.

Decision: use one canonical normalized title/search projection and a bounded index-supported candidate stage, then preserve current ranking and filtering over those candidates. Prefer reusing the established title_search machinery if it can meet user-search semantics, or align an explicit stored/indexable projection. Do not simply add

another GIN index to every column. The Concierge search_titles path is distinct: 20260209000000_search_titles_add_cover_image.sql:5 uses title_search.title_norm % normalize_title(query) with its matching GIN trgm index (20260203171100_concierge_core.sql:64), so it is not evidence of the same mismatch.

Acceptance: exact, native-script, prefix, one/two/three-character, >=5-character typo and empty queries retain ordering/safety/filter behavior; first and deep cursor pages have stable tie breaks; real plans show the desired candidate bound/index access; query work and p95/p99 remain inside an agreed budget under concurrent typing and cancellation. Verify user-visible recall against a fixed corpus before replacing ranking.

4. Preserve good pagination; benchmark conditional Browse plans

Collection anime/manga RPCs (20260204010500_collection_paging_rpc.sql:10,84) filter user_id by auth.uid()::text and use (updated_at,id) descending keysets. Matching (user_id,updated_at DESC,id DESC) indexes are in the same file:5,7. Mixed collection feed (20260204133000_collection_feed_paging_rpc.sql:6) UNION ALLs the two owner-scoped branches and orders by updated_at descending, source_rank ascending, row ID descending. Limits are capped120. These are sound foundations; switching to OFFSET would discard useful work.

Browse RPCs (20260326234000...:443,569) also use keysets and capped limits. Conditional CASE sorts, COALESCE metrics and combined genre/status/year/format filters merit real-plan checks against the existing plain btree and genre GIN indexes. Do not infer those indexes are useless solely from source shape.

Acceptance: large accounts and deep pages; equal timestamps/metric scores; anime and manga with the same numeric ID; null metric values; status changes between pages; stable cursors without gaps/duplicates within the defined consistency model. For Browse, compare each supported sort and common filter combination before adding indexes.

5. Bound social aggregation and preserve privacy semantics

Current count_friends_tracking (20260731040000_clubs_trust_pack_v1.sql:229; hosted body matches) calculates effective sharing from shared nonarchived clubs. Its list join casts stored TEXT ul.user_id::uuid (local line282), although user-list indexes start with TEXT user_id. Cast the bounded UUID friend side to TEXT when equivalent instead of forcing every candidate list column through a UUID cast; verify plans and legacy data first. A malformed legacy ID can otherwise fail the batch cast. This is not a reason for an immediate broad user-ID migration.

The RPC parses a JSON item list without an explicit server item cap. SupabaseService+Social.swift:94–119 sends all received items in one request; cancellation and stable media-type/id cache keys already exist, but there is no chunk in that method.

Club list loading (20260318104000_club_loading_rpcs.sql:7) performs correlated member/rail/poll/activity summaries per club, with repeated latest-activity calculations. Max20 members per club is not a cap on a user's club count or retained history. Bundle-loading starts at132. Membership helpers and member indexes exist; do not claim absent authorization.

Decision/acceptance: bound and deduplicate friend-count input, align join types/index access, retain batched counts, and profile club summary queries across realistic club counts/history. Tests must cover private/status/progress visibility, mixed memberships granting different sharing, archived clubs, no shared club and malicious oversized input. Introduce maintained summary data only if measured aggregation cost warrants its consistency burden.

6. Keep recommendation limits meaningful and measure fallback amplification

- curated_rail_cards (20260205231000_curated_rails.sql:64) fetches a narrow rail with catalog joins and seen exclusion; this is preferable to regenerating every rail.

- recommend_ids_premium (20260203201000_editorial_story_boosts.sql:32) scores broad catalog candidates with tag boosts/penalties/focus aggregates before LIMIT50. The edge agent observed repeated fallback builds; reducing duplicate calls is likely more useful than shrinking the returned limit.
- recommend_ids_similar_to_seeds (20260806020000_serving_hardening_v1.sql:134) reads precomputed similarity, combines seeds and deduplicates/excludes franchises. Cold/no-precomputed seeds or special options can enter _recommend_ids_similar_to_seeds_live (20260804130000_media_similar_titles_precompute.sql:830), with broader candidate/vector/tag/realm work. Measure fallback frequency, coverage and freshness explicitly.
- Latest fetch_taste_deck_batch (20260731150000_realm_graph_v1.sql:1402) caps output2–40 and candidate pools per medium to max58, then performs vector/window/diversity/temporary-table work. Its random()<0.10 negative probe is **not** ORDER BY random(). The latest-definition scan did not find an ORDER BY random() pattern in the indexed function bodies; old superseded functions must not be used to allege a current random full-catalog sort.
- fetch_tonight_shelf (20260806010000_serving_fixes_v1.sql:85) and fetch_realm_hidden_gem (20260806020000_serving_hardening_v1.sql:369) include franchise/window ranking before output limits. Returning one hidden gem does not imply evaluating one row. _franchise_component_labels is a SECURITY DEFINER accessor; check whether the planner can restrict its returned data in each use.
- media_realm_membership_effective (20260805112000...:32) is a security-invoker view merging base/delta/overrides through FULL OUTER JOIN and a materialized override CTE. Existing set-based materialization in expensive workers is intentional; do not replace it with per-row subqueries.
- get_manga_chapter_enrich_candidates (20260221173000_manga_chapter_crunch_mode.sql:8) aggregates chapter counts/mapping/review state before priority and LIMIT<=200. Inspect normal batch and forced-single-title plans separately; the single-title filter may or may not be pushed into each aggregate.

Acceptance: report requests and SQL calls per visible rail, duplicate-fallback rate, cold/warm seed behavior and recommendation quality. Bound parse fanout and rail concurrency at the caller. Measure temp DDL/spill under concurrent taste-deck requests. Preserve exclusion, safety and franchise correctness when reducing candidate work.

7. Refresh and RLS: targeted changes only

The baseline adds unique id indexes to seven catalog materialized views (20260807000000_baseline_capture_v1.sql:2986–3003), enabling concurrent refresh, but its recorded cron command at3019–3025 still refreshes seven serially without CONCURRENTLY. Root's current cron metadata confirms daily01:30 scheduling and one successful10.76s execution in the last24h, but does not by itself prove the exact current command or blocking duration. Confirm that command before proposing concurrent refresh; test readers during it and preserve freshness/failure visibility.

Base user-list RLS in the baseline already uses initplan-style (SELECT auth.uid())::text ownership predicates (anime2201, manga2369); do not blanket-rewrite it based on general advice. Existing view security_invoker changes matter. Definer RPCs need deliberate auth predicates, fixed search paths, intended grants and cross-user tests, not indiscriminate conversion. Root advisors'9 unindexed FKs,8 auth-initplan findings and105 unused-index findings are candidates for object-by-object review; low usage since a stats reset does not justify deleting105 indexes.

8. Modularity decisions for scope, shelves, profiles and clubs

SupabaseService is one @MainActor @Observable state owner, even though domain methods are split into extensions. Core file is2972lines, models1324, RPC parameters637. Core owns auth/bootstrap, catalogs/details/caches/inflight work, collection state and cross-domain DTOs; domain extensions add behavior but do not create independent lifecycle boundaries. Large line count alone is not a refactor mandate.

Preserve the current stack and established typed model split. SupabaseModels.swift:3 already has MediaKind; MediaDisplayable at9 and stableKey at31 provide shared presentation/identity; Anime and Manga retain their different fields. UserList at547 normalizes progress while persisted anime/manga lists remain distinct. SupabaseService+UserLists.swift:35 builds separate mutation payloads. Unknown manga totals and chapter/volume progress must not be forced into an anime episode model.

Scope: introduce a typed, persisted user preference with explicit propagation through request parameters, cache keys, cancellation/generation checks and loading/error state. Discover currently computes both media in one bundle (20260326234000...:707); taste deck also intentionally mixes both. If the new product contract is persistent anime-only/manga-only scope, filtering the visible cards after fetching both wastes work and can starve a rail. Decide which surfaces honor scope, where cross-media discovery is intentional and how fallback behaves. Keep preferences separate from inferred taste.

Shelves: represent user-defined ordered membership with typed media identity, independent of watching/reading status. Do not overload PLANNING/status values to represent shelves. Establish order, duplicates, pagination, ownership, visibility and deletion semantics; index owner/shelf/order and prove RLS before UI exposure.

Profiles: expose an explicit public projection and opt-in fields; do not loosen existing private profiles/list RLS globally. Notes, progress, history and inferred taste need deliberate visibility. Club display-name access via privileged RPC is not permission for public profile reads. Keep public pagination and cache keys separate from personal collection state.

Clubs: reuse existing membership and effective-sharing rules rather than creating a second friend graph for these features. Keep suggestion generation outside heavily repeated membership/summary reads.

Bounded extraction: when implementing each feature, move its state/cache policy behind a cohesive existing-pattern boundary while keeping the auth/client shared. Do not simultaneously replace all services or merge physical anime/manga catalog tables. Extend shared identity/projection/progress traits, retain domain-specific payload validation, and version RPC contracts when semantics change.

Acceptance: scope survives relaunch; two users switching accounts cannot see previous personal cache results; old in-flight responses cannot overwrite a new user/scope; mixed numeric media IDs never collide; shelves retain status independently; unknown chapter/volume totals remain unknown; public requests cannot infer private notes/list membership; membership changes invalidate affected club data. Root's client audit owns detailed lifecycle bug confirmation.

Suggested implementation order

1. Capture reproducible schema contracts and define migration bootstrap/upgrade acceptance; concurrently investigate the measured five-minute maintenance cost. 2. Establish Search query/candidate contract and measure actual plans; fix duplicate recommendation fanout and instrument fallback coverage. 3. Tighten social batch limits/cast alignment and verify refresh/read behavior. 4. Implement persistent scope as a cross-layer contract, then shelves and explicit profile visibility with bounded state extraction. 5. Reassess indexes, club summaries and richer recommendation models against measured load. Do not use new product features to justify a wholesale rewrite.

Evidence limitations: no hosted workload was generated by this review; no p95/p99 distribution or load capacity was proven. The matching16 functions narrow deployment uncertainty but do not remove documented schema replay gaps. Historical counters and SQL-shape risks are labeled separately throughout.

Root follow-up: current unified list projection

At 2026-09-25 22:28:17 UTC the live view definition was read directly; see live-user-lists-view.json. It unions both list tables, projects rating times ten as score, null volume progress/start/completion dates and constant false private. Those placeholders do not establish user history or activity consent; authorization is separate. This closes the live-definition gap, while fresh migration replay remains untested.

Operational jobs audit

Kuro operational database and ingestion audit — 2026-09-26

Scope and evidence boundary

Read-only source review of scripts/ and research/ executable JS/TS/Swift/Python/shell files, plus migration cron registration history. No worker, importer, healthcheck, SQL writer, installation, hosted query or deployment was executed. No credential files were opened. Source checkout remains unchanged.

Artifacts:

- evidence/operational-call-register.csv: 427 static records: 138 SDK table sites, 21 RPC sites, 10 typed REST client sites, 8 SDK auth/function sites, 64 HTTP/CLI transports, 112 selected helper invocations, and 74 historical cron schedule/unschedule declarations (38/36). This is **not 427 unique requests per session**: helpers and their transport sites are deliberately separate.
- evidence/operational-coverage.json: 87 executable files screened; 51 contain registered call sites. The remaining 36 include launch wrappers/installers, local-only dashboards, code generators, quality gates and a legacy UI reference. A wrapper may trigger a registered worker even if it has no direct DB transport call.
- Source targets derived statically; unresolved dynamic table/URL arguments remain explicitly marked. SQL bodies, deployed functions, iOS foreground request graphs and runtime traces belong to the parallel audits. No statement that every dynamic runtime path has been profiled is justified.

Highest-priority findings

OP01 — Empty media-relation queue still triggers 1,000 title RPCs and a full relation scan

media_relations_worker.js:592–604 unconditionally runs buildCoverageReport(REPORT_SAMPLE_SIZE) after queue work, including when the queue is empty. Default sample size 500 is applied independently to anime and manga (:349–365), then each title invokes get_media_ladder with concurrency 8 (:499–516). It also obtains an exact count and reads the entire relation table in 1,000-row offset pages to count distinct sources in JS (:446–468). The installer default is every 900 seconds (install_media_relations_launchd.sh:72); root and report paths are hardcoded /Applications/Kuro, so actual installed activation/path must be checked before claiming current cost.

If defaults were installed and 500 eligible titles exist in each mode, an idle run costs approximately **1,003 + relation-page-count requests**, versus the single queue fetch needed to discover no work. At 96 runs/day the ladder report alone would generate 96,000 RPCs/day. This is a conditional source-derived estimate, not an observed live count. Move wide coverage audits to a manual/daily operation; use bounded SQL aggregates or cached counters; report deltas for the titles actually processed. Idle worker should exit cheaply.

OP02 — Recommendation-edge importer retries server errors forever

import_anilist_rec_edges.js:278–313: upsertEdges has for (...;;) and, on HTTP >=500, sleeps 2 seconds and continues without incrementing a bounded failure counter or enforcing a deadline. Hourly RPC rate-limit waits are separately capped at three, but the server-error branch is not. Add capped exponential backoff with jitter, per-request timeout, total attempt/time budget, cancellation and retained checkpoint. This is a manual importer source defect; no evidence it is presently running.

OP03 — Relation refresh is chatty and nontransactional

media_relations_worker.js:203–244 deletes all AniList edges for a source before resolving target IDs one by one and upserting replacement rows. Request failure after delete can erase a valid ladder until retry. Local-ID memoization prevents repeated lookups within one process, but not the first N target calls or across runs. Queue hydration also performs one anime/manga query per queue item (:257–287), followed by per-item status writes (:296–339). Queue candidate reads are not a transactional lease: local shell PID lock only protects one local report directory, not a second host/worker; errored rows are eligible immediately with no retry ceiling or next-attempt condition. Batch ID mapping, lease candidate rows in SQL, transactional replace only after complete validated source response, retain last-known-good data, and cap failures with backoff.

OP04 — Fast importer batches writes but not ID-map hydration

import_anilist_fast.js:128–134 resolves .in(keyField, ids) without chunking/pagination; a media page can contribute up to 50 characters and 50 staff per title, so unique auxiliary IDs can exceed a PostgREST result cap. Upserts are chunked at 1,000 (:151–156) but subsequent mapping can silently omit later IDs, resulting in missing joins. Verify actual endpoint max-rows; chunk ID lookups and assert every required imported entity maps or is reported. Main accepts unvalidated concurrency (:403): zero/negative concurrency leaves the while loop waiting forever, and very high input launches uncontrolled page concurrency. Default is 3, finite page range; cap and validate. Calls lack explicit network abort deadlines. Page failures log and allow eventual successful process exit; ensure resumable failed-page manifest and failure summary.

OP05 — Old local/legacy importers amplify entity writes and replace child rows

import_anilist_local.js:172–409 upserts one title, then each studio/tag/character/staff and join individually; also deletes and recreates episode/chapter/volume placeholders. Chapter/volume arrays are generated from total counts. scripts/legacy/*edge_function*.js preserves similar patterns but is **legacy reference**, not deployed-edge proof. Distinguish these from the active edge implementations reviewed elsewhere. Consolidate supported ingestion entry points, retire unsafe paths, compare source versions/hashes, batch upserts and preserve enriched child metadata. Placeholder totals must never be represented as verified release dates or songs.

OP06 — Reporting utilities can scan tables and some healthchecks write

collect_db_metrics.js executes separate exact-count queries for 23 tables, filtered coverage counts, 10 list-status counts and many missingness fields. HEAD reduces payload, not exact-count work. Manual use is appropriate at a controlled cadence; don't turn this into Home polling. check_cron_health.js:49 uses exact count just to establish a table exists; :277,:290,:310 invokes actual import, manga-enrichment and mirror functions, so it is **not read-only**. genre_audit.js:50 reads up to 25,000 rows per media type and scans tags in pages; counts/coverage can be incomplete relative to current larger catalog. Prefer bounded aggregate diagnostics with explicit sample size, and separate read-only monitoring from run-work controls.

OP07 — Provider worker has sensible bounds but per-title round trips and soft deadline

provider_availability_worker.swift:582–630: default limit 100, 45s time budget, sequential candidates. Each candidate obtains media seed, source map, optional refresh state, provider source data, offers upsert, refresh state upsert and a stale-offer ID read (up to 1,000 IDs merely for a count at :1143). Candidate deadline is checked between titles, not as a hard deadline on individual network operations. Source matching and offers are intentionally separate; keep that integrity while returning mapping/media fields in candidate batches and persisting result/state atomically. Mapping errors retry at up to 24-hour intervals; Watchmode has a 3 requests/s limiter and bounded retries. Source/price decision remains gated, so code existence does not prove live provider coverage.

OP08 — Synopsis and safety workers are bounded but incur avoidable bookkeeping scans

Synopsis: before counts per media type, candidate RPC per media type, one result/failure RPC per item, after counts. Direct worker default 25 per type capped 200; installer default is 100 per type every 240 seconds. Catalog safety: lexicon read, before counts, candidates, one result/failure RPC per item, after counts and open-gaps RPC; default 200 per type capped 1,000, installer every 600s. PID/TTL locks prevent local overlap. Queue/backoff SQL exists but must be evaluated by SQL audit; transport clients use URLSession without explicit request deadlines/retry policy. Separate inexpensive work polling from full backlog/open-gap scans and batch result commits where it preserves failure isolation. Do not remove needed retry or safety review evidence solely to lower call counts.

Current hosted cron evidence supplied by root audit

Root freshly captured evidence/live-cron.json and evidence/rebuild_media_similar_titles.sql during this initiative:

- realm-similar-drive-5m: 287 runs in 24h; mean 25.89s; maximum 442.59s. This is background job runtime, not a Home-request latency measurement.
- realm-tier-refresh: one run ~133.57s in that window.
- Live similar builder performs global seed sync and recursive franchise-component maintenance **before** checking for an empty stale batch; overlap advisory lock exists. Thus an apparent empty-batch path is not necessarily a cheap no-op.
- Cumulative pg_stat_statements supplied by root: 15,060 similar rebuild calls averaging 25.54s since the stats window, and 54 tier rebuild calls averaging 133.77s. Temp IO and table rows-scanned are cumulative and must not be presented as current per-request cost.

Prioritize change-driven invalidation and moving invariant/global maintenance out of every 5-minute driver invocation; measure quiet and active queue deltas separately. Retain previous serving rows until successful rebuild, advisory overlap protection and timeout guards. Validate whether nightly full re-stale is needed for unchanged inputs rather than simply slowing all fresh-data delivery.

HTTP pg_cron success proves dispatch only; it does not prove an edge function finished or persisted updates. Need edge/import run completion and source freshness evidence.

Operational call graph and schedule intent

Pipeline	Source flow	Source bounds / controls	Actual activation
Synopsis	launchd -> run_synopsis_enrichment.sh -> Swift client -> backlog/candidate RPC -> Apple FM or heuristic -> result/failure RPC -> local reports	installer 240s, batch100/type; direct default25/type; worker cap200; PID/TTL lock1800s	not queried here
Catalog safety	launchd -> run_catalog_safety.sh -> terms + backlog/candidates -> local rules/FM -> result/failure RPC -> open-gap report	600s, batch200/type, cap1000, PID/TTL1800s	not queried here
Media relations	launchd -> shell -> queue -> catalog hydration -> AniList -> local ID resolution -> delete/upsert edges -> status -> full coverage report	900s, queue40, top500/type, AniList20s timeout + max4 retries; lock21600s	not queried here; hardcoded old path
Provider	launchd -> shell -> candidate RPC -> seed/map -> Watchmode search/source -> offers/state -> reports	600s,100 candidates,45s between-item deadline; local lock7200s; rate3/s	not queried here; rollout/source gated
Recommendation edges	manual CLI -> whole visible pool paged -> resume checkpoint -> AniList rec pages -> missing IDs in chunks100 -> upsert_rec_edges chunks500 -> checkpoint	AniList bounded retries + Retry-After; 3 hourly-rate waits; **5xx RPC unbounded**	manual runtime unverified
Realm descriptors	manual driver -> media_realm_llm_pending paged -> local success checkpoint -> realm-describe single or batch -> cooldown	batch default10 cap25; concurrency default3; overall limit optional; batch429/no-progress cooldown and exit guard	cron was repeatedly added/paused historically; inspect live
Similar titles	nightly re-stale -> 5-minute SQL builder -> media_similar_titles -> app indexed serving RPC	source batch100, timeout600s, overlap advisory lock; root observed current runtime	live root evidence above
Realm refresh chain	membership04:30 -> affinity04:40 -> tier04:50 -> similar-nightly05:10	SET-first600s for membership/affinity; expensive materialized maintenance	live root metadata authoritative
Mirror images	five daily HTTP cron dispatches -> mirror-images edge -> bounded storage/DB batches -> run logs	anime/manga offsets0/200/400 at02:00/15/30; character02:45; staff03:00, limit200 source registrations; auth & skip-if-mirrored	edge work reviewed separately; dispatch != completion
Manga enrichment	15-minute queue / nightly sweep cron -> manga-chapter-enrich edge -> canonical mapping and enrichment	several later migrations replace schedule/body	live metadata, not additive migration count

Pipeline	Source flow	Source bounds / controls	Actual activation
Catalog import	hourly edge trigger or manual run_full_import / local/fast importer	servercheckpoint/manualmaxLoops10 ; per-operation limits vary	distinguish separate supported entrypoints
Local dashboards	browser timer -> local /api/status -> JSON/log files	synopsis/safety/provider frontend5s polls	**not recurring DB queries**; avoid falsely counting them

Migration history includes superseded cron registrations. E.g. similar batch changed300->200->100, realm-describe added then paused, old club message pruning removed. Register retains the historical sites as evidence, not as simultaneous live jobs. kuro-refresh-matviews captured in 20260807000000_baseline_capture_v1.sql:3015 uses seven ordinary REFRESH MATERIALIZED VIEW statements rather than CONCURRENTLY; determine current definition and reader contention from runtime evidence. Existing unique indexes alone do not make a nonconcurrent refresh concurrent.

Contract for the comprehensive implementation plan

1. Maintain a single explicit registry connecting each supported executable to owner, trigger, deployment location, operation/write class, input source, tables/RPCs, rate budget, timeout/retry policy, checkpoint, last-success evidence and stop mechanism. Mark legacy/manual scripts clearly. 2. Prevent accidental double scheduling across local launchd, pg_cron and manual backfills using DB leases/idempotency where overlapping jobs mutate the same rows; local PID locks remain useful but are not distributed protection. 3. Ingest new songs/release/relations data through bounded queues prioritized by followed/popular titles, changes and freshness; never fetch external providers separately per visible image/card. Preserve source IDs, source timestamps, confidence/mapping status and licensing constraints. 4. Treat anime and manga as separate freshness contracts. Anime episode dates can expire; manga chapter/volume counts are not reliable dates. Serve unknown and source age honestly. 5. Home should read prepared, bounded, mode-specific results; ingestion and global scoring are background work. Don't place count diagnostics, relation coverage scans or recomputation on a user screen load. 6. Verification: dry-run fixture tests for queue leasing, retries, malformed concurrency, server500 exhaustion and source-partial failures; bounded integration tracing for idle/active worker call counts; measured DB stats deltas and EXPLAIN on representative sizes, including large catalogs/collections. Prove no last-known-good data loss on failed refresh. Keep live-load tests a separate approved step.

Remaining uncertainty

All source files in the declared executable extensions were mechanically screened, and principal worker flows were manually inspected; this is not an AST-complete guarantee for generated URLs, dynamic SQL, dynamic wrapper targets or unexecuted branches. Cron historical declaration coverage is complete for direct cron.schedule/unschedule sites in migration files, while hosted job state is root evidence. Actual launchd installation, environment overrides, alternate /Applications/Kuro copy, API max-rows and current worker process activity were not inspected here. The CSV intentionally marks unresolved bounds/targets rather than inventing certainty.

Music and metadata source research

Anime music and related metadata — source feasibility

Checked 26 September 2026, Europe/Berlin. Primary provider documentation/source only. Read-only research: no catalogue import, music download, playback, account change or project edit. Root agent separately audits Kuro's already-ingested fields; this memo does **not** label any field unused in Kuro without that comparison.

Recommendation

If “songs for seasons/episodes” means opening and ending credits, **AnimeThemes is the best structural match among the sources checked**. Start with a compact Music section containing OP/ED label, song, credited performers, episode coverage where known, and verified outbound music links. Full soundtracks, episode background cues and in-app/background playback are separate scopes. They cannot be inferred from an opening/ending catalogue.

The current documentation supports this proposal, but a live AnimeThemes metadata request returned HTTP 403 from this environment; Jikan returned 504. No working integration or complete coverage is claimed. AnimeThemes' indexed homepage also currently reports a host-datacenter outage; that message may be stale and does not establish the cause of the 403. Provider homepage

Verified source capabilities

1. AniList: catalogue and relationships, not a structured theme-song catalogue

Current official Media reference exposes IDs including idMal, multilingual titles, season/year, dates, format, episode/duration totals, relations, staff, studios, tags, airing schedules, external links, trailers and legal streaming-episode links. It does **not document song, opening-theme or ending-theme fields on Media**. Theme-music credits may be represented by staff roles, but that is not a reliable normalized song/version/episode model. isLicensed describes the media's licensing/self-published classification; it is not permission for Kuro to play music. Media schema, Media edge roles

External links have provider identity, type, language, notes and disabled state. Preserve those distinctions rather than flattening all URLs into a generic “watch” link. Staff/voice roles include language, role notes and dub-group distinctions, useful for correctly labeled credits. External-link schema, Staff-role schema

For root's comparison, candidate field groups are relation type and source IDs; full paginated staff/character relationships; external-link disabled/language/type metadata; season/duration/source/format; airing freshness; tag spoiler flags; staff-role/dub distinctions. Availability in a schema does not prove population coverage or absence from Kuro.

Terms: AniList prohibits mass collection and competing non-complementary tracker services; its under-$150/month commercial allowance is not an override of those restrictions. Existing authorization, if any, must cover the actual product and data use before enlarging an ingestion strategy. Current rate documentation states 90/min nominally but includes a degraded-state warning of 30/min and says rate-limit increases are not currently accepted. Implement adaptive headers/backoff, not a hard assumption of 90. Terms, rate limits

Access: current indexed official schema; official GitHub docs fetched directly. GraphQL introspection returned403, so no live schema-response claim. Docs repository HEAD 03281c0a4bbf0c7f2097e0c935cddaed1096aa65, dated 18 Aug 2026 (repository commit, not each page's publication date).

2. AnimeThemes: structured OP/ED, versions and coverage

The official documentation distinguishes:

Entity	Useful available data	Important interpretation
Anime	Provider ID, name/slug, premiere season/year, format, timestamps, external resources	A production, not an arbitrary franchise umbrella
AnimeTheme	OP/ED type, sequence number, song association	OP2 is a sequence label, not “season 2”
AnimeThemeEntry	Version, nullable episodes string, spoiler/NSFW, notes, timestamps	Coverage is text, not guaranteed normalized integers; version can represent visual changes
Song / performance	Song ID/title; artist associations, credited alias/character/member information	Do not merge recording, composition, performer and character identities solely by title
External resource	Site, link, nullable external integer ID	AniList/MAL joins and music-provider links; preserve URL because many music IDs are not integers
Video / Audio	Separate resource IDs, storage URLs and technical metadata	Technical access is not playback/redistribution licensing

Sources: Anime, Theme, Entry, Song, performance credits, external resources.

External-resource enum includes Spotify, YouTube Music, YouTube, Apple Music, Amazon Music and Official Website. This means the model can represent these links, **not** that every song has them or every linked upload is rights-holder verified. The song endpoint supports its resources; the anime endpoint's allowed nested includes do not explicitly list animethemes.song.resources, so use a supported song hydration request or verified GraphQL selection rather than assuming unlimited nested REST includes.

The GraphQL findAnimeByExternalSite(site: ANILIST, id: [Int!]) accepts multiple external IDs without authentication; documentation explicitly allows **multiple anime per external resource**. Do not enforce a one-to-one crosswalk prematurely. REST has sparse fields, includes, pagination and updated/deleted filters. Batch join example, REST index

Both API styles document90 requests/min with remaining/reset/retry headers. GraphQL also documents depth13 and complexity10000, with complexity explicitly experimental. REST quota, GraphQL limits

Video fields include creditless/subtitled/lyrics/uncensored distinctions, source format and whether visuals overlap episode content. Audio is described as the audio track of the theme video, not necessarily a full commercial recording. Neither the supplied URLs nor the repository's MIT software/documentation license establishes audiovisual rights, a commercial data-reuse grant or complete attribution requirements. Public API support contact is documented; exact intended cache/republication and playback terms remain unverified. Attribute factual metadata sources and preserve per-row provenance, but do not pretend attribution resolves missing rights. Video, Audio, license

Access: official repository docs fetched directly after documentation-host timeout. HEAD f4ac78c1e0046a63bb4b3336e504e8bcafd56224, dated10 Sep2026. One public metadata GET with includes returned403; no audio/video bytes requested. Confirm access, data terms and sample coverage before implementation.

3. MAL official API versus Jikan

The MAL official v2 reference could not be opened in this environment. **Official theme-song availability was not independently verified here.** Do not conflate Jikan's fields with MAL's official API, and do not present an official-API absence as a fresh verified fact from this lane.

Jikan's own OpenAPI specifies GET /anime/{id}/themes: data.openings and data.endings are arrays of strings. This is useful fallback display text, not normalized artists, playable URLs, version IDs or validated episode ranges. Preserve source text; any parsing needs confidence and a raw fallback. Official Jikan OpenAPI, theme response implementation

Jikan states that it scrapes MAL, is unofficial/unaffiliated, and provides read-only GET data without authenticated list updates. Its documentation gives60/min and3/sec, upstream MAL may throttle too; requests are documented as cached24h, with ETag/If-None-Match/304 and cache headers. Respect these freshness limits rather than polling rapidly. Software is open source; the README explicitly directs users to respect MAL's terms. This is not a new grant to republish underlying MAL data commercially. Jikan README, API documentation

Access: documentation HTML pointed to the directly fetched provider OpenAPI JSON; source HEAD 1e33a79bcc9161f1831e4428004e110c67a48676, dated14 Jun2026. One sample anime/1/themes request returned504; no success or freshness claim. Dedicated Jikan terms page was inaccessible in the web tool; exact reuse terms remain to be checked before adoption.

4. MusicBrainz: identity and discography enrichment, optional

MusicBrainz can resolve artist, recording, work, release and release-group identities, ISRC/ISWC and URL relationships. This helps disambiguate a TV edit from a full recording or an artist from a similarly named performer. It is not a reliable general source for which song plays in which anime episode, and does not supply music streams. API, scope statement

Core metadata is CC0; supplementary data and live feed have different noncommercial/share-alike licensing. **Public API commercial access is a separate question:** documentation says free for noncommercial use and points commercial users to plans/contact. API requires a meaningful User-Agent and at most one call/second. Do not promise commercial MusicBrainz API use is free because core data is CC0. Data license, rate limits

Access: direct docs initially429; official indexed full pages read. No music/API enrichment requests made. Do not introduce this dependency for the first OP/ED release unless unresolved identity needs justify it.

Proposed minimal data model

This is a design proposal, not a migration request.

- anime_music_source_map: Kuro anime ID, provider, source anime ID/URL, external AniList/MAL ID, match method/confidence and review state. Allow multiple source rows per production and explicit ambiguity; use existing source IDs first, titles only as a fallback candidate search.
- music_song: Kuro song ID, source song IDs, original/display title, optional recording/work identifiers. Do not treat an AnimeThemes song ID as proof of one specific commercial recording.
- music_credit: song, artist source ID/display name, credited alias/character, role when explicitly supplied and credit order. Leave unknown composer/lyricist roles unknown; theme performance is not authorship.
- anime_theme: Kuro anime ID, song ID, OP/ED/other-known-role, sequence, source theme ID. Insert/OST roles need a different verified source; do not invent them from OP/ED data.
- theme_version: theme, source entry ID, version, episodes_raw, nullable normalized ranges/exceptions, parse confidence, spoiler/NSFW/notes. Preserve odd cases such as episode0, specials, decimals, comma sets and unknown coverage without forcing integers.
- music_link: song/version, provider, canonical destination URL, link kind (official song, artist, source reference, video, audio), verification method/date, availability/disabled state, rights status. Keep source-reference links separate from approved playback URLs.
- Shared provenance: source URL, fetched_at, source_updated_at when available, payload hash, parser version, last_success/error and deletion marker. Optional raw metadata snapshot only to the extent allowed by the

source's terms; never copy media as part of this ingest.

Efficient ingestion and serving

1. Build an authorized bounded pilot around active/current-season and frequently opened detail titles, not an indiscriminate whole-catalogue crawl. Prefer existing source IDs and batch matching. Review collisions between split cours, sequel seasons and film compilations. 2. Fetch only metadata required by the feature. Keep song/artist/entry hydration batched and deduplicated; cache negative matches with a shorter expiry for currently airing titles. A missing result means unknown/unmapped, not “this anime has no music.” 3. After initial permitted hydration, schedule current-season refreshes more often and finished titles less often. Use provider timestamps and supported updated/deleted filters with a small overlap and stable cursor. Do **not assume child-song or entry edits update the parent anime timestamp**; track child changes or periodically reconcile bounded hydrated titles. Retry transient failures with jitter, honor429/Retry-After, and pause on403 rather than switching endpoints to evade it. 4. For Jikan, use conditional ETags and documented cache freshness; one shared server scheduler enforces both second and minute quotas. For AnimeThemes, follow response quotas and GraphQL complexity limits. Neither provider's per-minute allowance is an endorsement of bulk collection. 5. Serve a compact music summary from Kuro's own permitted cache alongside the detail payload. Never issue live third-party requests per card or block title rendering on enrichment. Detail expansion can enqueue refresh and continue showing the last valid snapshot with an honest missing-state. 6. Deduplicate writes by provider ID and payload hash; keep old valid data through transient outages, track removals, and expose administrative mapping/provenance checks. Measure match coverage, ambiguous matches, usable music-link coverage and age, not just successful HTTP calls.

Link-out versus playback decision

The narrowest useful version requires no in-app audio: users see credits and choose “Open in Spotify/Apple Music/official video” for an individually verified destination. This avoids treating a song catalogue as a playback service. It does not guarantee users can listen free or in their region; destination service/account rules apply. Do not invent links from title-only searches or label community-hosted media “official.”

If playback is requested, evaluate the chosen provider's authorized player/SDK separately. Spotify documents embeds for controlled websites under its developer terms; it is not blanket permission for a native custom player, background soundtrack, caching or full-track access. YouTube explicitly prohibits separating audio and hidden/background players. Do not use a YouTube video URL as an audio-only background music source. Spotify embeds, Spotify widget terms, YouTube developer policies

Until the pending scope clarification is answered, recommend **OP/ED factual metadata plus official link-outs**, with episode coverage shown only when source-backed. Present OST/insert-song cataloguing and background playback as separate follow-ups with independent sourcing and rights work. Do not hide useful title details behind music loading or autoplay music on detail entry.

Retained market and current-product evidence

The following earlier review supplies the complete 2025–2026 research, source register, monetization analysis and original design rationale. Earlier recommendations such as a separate agenda or continuation-first Home are proposals, not overriding decisions. The withdrawn app-wide animation construction is deliberately excluded. All evidence retains its original observation date.

Kuro — Design, retention and current-product review

25 September 2026 research, direction updated 26 September · Prepared for Max · No app implementation or deployment.

Approved welcome reference: Max explicitly confirmed the uploaded welcome/onboarding in GitHub commit b943e457fd2b611bafa6a52dcbbada417d5b727a, branch codex/kuro-onboarding-no-tv. Preserve that composition and the removal of the TV. The replacement welcome and condensed introduction remain withdrawn. The local AuthView contains separate composition edits and is not the approved visual baseline. The email form and signed-in experience remain the improvement scope. A source reference is now identified; installed visuals and release equivalence remain unverified.

1. The decision

Latest owner direction — 26 September 2026: Animation belongs to welcome/authentication only. Preserve top navigation and swiping; persist Anime/Manga selection across screens and requests; make Home distinct from Discover/search. Collection is a visual shelf with statuses and optional named shelves. Social relationships are club-only; profiles show identity by default and activity only when enabled. Earlier first-login, Home, agenda, Collection and email concepts, including the moving-landscape Home, are withdrawn. Exact screen layouts and navigation order remain to agree. No implementation or rollout is authorized by these planning documents.

Keep the approved animation within welcome/authentication. The rest of the app is a quiet space with restrained functional transitions. After login, make Kuro immediately useful: **continue a title, remember progress, see relevant releases, and find something through people you chose.** Preserve editorial discovery, Taste and private Clubs; change their prominence according to the user's situation.

The strongest opportunity is making the existing catalogue useful in daily tasks. Kuro already has many of the required ingredients. They are distributed across screens, several useful features are switched off, and some foundational tracking paths have correctness or completeness gaps. Solve those before amplifying them on the home screen.

First priority: reliable entry and tracking. **Second:** a personal, compact home and easy collection actions. **Third:** a dependable personal release agenda and privacy-respecting social context. Personal named lists and edition-aware manga tracking are genuine additional product work; they are not already delivered by status filters or club rails.

The current owner brief is ../2026-09-26-kuro-plan/MAX_BRIEF.pdf. This document now includes the original market/revenue study in full: 18 product decisions, 10 monetization decisions and their 110-record evidence register, alongside the 16 current-product design decisions and 11 additional community records. These are overlapping decision layers, not 44 independent feature commitments. PHONE_DESIGN_PROPOSAL.html now opens a withdrawal notice and the current brief; both earlier screen studies are superseded. Its example title, concept cover and progress are demonstration content, not production data. The four-destination navigation shown there is an exploratory alternative, not a prerequisite for improving the current five-page app.

2. What was actually reviewed

Six independent work lanes covered authentication/onboarding source, collection/import/manga source, home/social source, fresh community evidence, installed/release identity, and the visual proposal. The primary review reconciled those findings with read-only live backend queries and current files. This builds on the earlier 110-record market/revenue report; the additional 11 community records are listed separately, not presented as a new representative survey.

Evidence layer	Result	Limit
Current source	HEAD 697a5bc4548cc6b6492113f8481fb8bc8403030c; local tracking ref reports 12 commits ahead; existing AuthView edits preserved; freshly checked GitHub welcome reference listed below	Describes the inspected working tree, not a released binary
Approved welcome source	GitHub b943e457, uploaded 25 September; no TV; scenery height 52% plus safe area; grouped wordmark/headline	Owner-selected design reference only; not an instruction to replace current product source with the whole branch
GitHub release path	Reference branch 33 commits ahead of main; manual release workflow; no recorded Actions runs	Confirms uploaded source, not an Apple upload or release; local/manual uploads could occur outside Actions
Local build settings	Version 1.0, build 17	No fresh build or test suite run for this document-only initiative
Connected iPhone 15 Pro	Installed com.Kuro.app version 1.0, build 19; device reports developer-built	Producing commit and equivalence to this checkout unknown
Live backend	Kuro project bkdifromsqxkndnllmdj; active/healthy project; 235 migration records, 16 active Edge Functions	Inventory and schema are not proof of end-to-end feature behavior or source/function byte parity
Live flags and catalogue	Queried 25 September, approximately 21:10–21:24 UTC; detailed snapshot below	Point-in-time data, not a freshness SLA or usage measurement
Visual evidence	Existing saved Discover/Collection smoke images inspected; current AuthView layout and scenery asset inspected	Saved smoke images are historical, not fresh current-phone screenshots
Physical screen	Read-only QuickTime preview attempted, including after user confirmed unlock; feed remained black	No current phone visual/accessibility reproduction claimed; no recording or installation
TestFlight / public App Store	Unverified: configured Fastlane key file missing, so no authenticated request ran	Installed build 19 cannot be called the latest beta or public release

No user histories, messages or private club content were extracted for this report. The live checks used configuration, schema and catalogue aggregates. No production flags, catalogue records, credentials or accounts were changed. User-visible failures predicted by source analysis are clearly distinguished from reproduced runtime failures.

3. Built, enabled, staged and absent

The word “live” below refers to an observed backend flag/object, not to a claim that every installed client successfully exposes it. Feature evaluation also depends on enabled state, region, cached values, user allocation and debug overrides.

Capability	Current local implementation	Live backend / exposure	Design consequence
Painted welcome, Apple/email auth	Approved remote welcome pinned; default welcome → email; local composition edits kept separate	Installed phone build differs; auth settings/session flow not tested	Keep atmosphere; improve clarity and form continuity; verify failure paths
Intro tour	Five optional cards after authentication; Skip exists; final CTA may enter Taste	Client behavior, no standalone server flag	Keep its visual identity; offer a useful action without requiring five explanations
Default Discover	Daily editorial hero, catalog Airing Today, discovery rails	Daily feature exists separately from personalized gates	Returning users need current-title actions above a full-height cover
Taste Deck	Endless deck with signal/undo/leanings	taste_deck_v1: enabled, 100%	Retain as voluntary refinement; do not describe as a finite setup exercise
Because You / personalized anime NTY	Fetch, qualification and fallback code exists	personalized_new_to_you_v1: enabled but 0%	Built and staged; turning on Taste does not expose this rail
Realm Shelf / Hidden Gem	Fetch and display code exists	discover_realm_rails_v1: enabled but 0%	Evaluate relevance and empty states before rollout
Collection grid/list/status/sort/batch	Present, with quick progress on eligible surfaces	User-list tables and 1–10 rating constraints present	Improve consistency, full-library scope and failure feedback
AniList import	Profile entry, preview, confirm, reconciliation/undo paths	Import Edge Function v5 active; apply v25 active	Reuse entry flow; improve capped/lossy transfer before presenting full migration
Personal anime next seven days	user_airing_next, fetch, grid countdown, local reminder code	View exists; live catalogue has zero next-seven-day timestamps	Missing agenda UI plus a data-freshness dependency
Friend title activity	Shared-club statuses/comments and catalog-card counts	social_activity_v1: enabled, 100%	Reuse permitted context; tracking count does not mean friends finished
Club reactions, pace, realtime, badges	Present with additional privacy/member conditions	Respective flags enabled, 100%	Existing retention features; do not rebuild a generic social network
Enriched club cards	Implemented directly	clubs_list_enriched_v1 100%; accessor has no consumer in inspected client	Flag is not an effective UI rollback switch in this source
Where-to-watch/read provider filters	Registry/preferences/filter/render paths built	streaming_availability_v1: enabled, 0%; provider availability table empty	Do not enable an empty integration; legal external links are a separate existing path
AI assistance	Deterministic fallbacks plus optional assistance	RAG/FM/clarify enabled at 5%, DE/AT/CH only	Core tasks must remain usable outside this allocation and device support
Other Concierge flags	Accessors/code present in varying forms	interaction/performance/editorial flags disabled, 0%	Do not infer whole feature absence from a flag whose consumption differs

Capability	Current local implementation	Live backend / exposure	Design consequence
Club chat	Historical infrastructure, current detail has Rails/Active/Polls	clubs_chat_v1 disabled even though percentage is 100	Enabled boolean takes precedence; do not claim chat is exposed
Affiliate links	Historical backend scaffold	affiliate_links_v1 disabled, 0%	Monetization proposal is not a launched revenue feature
Named private personal lists	Not found in inspected model/UI paths	Not established by this audit	New membership model; status categories and club rails do not substitute
Volume progress / rewatch history / portable export	Partial model properties or internal text helper only	No complete round-trip established	Real product/data work, not simply new labels
Personal release inbox / cross-title friends feed	No complete surface found	Existing personal/title/club pieces can support parts	New aggregation and preference contracts needed

4. The data already ingested—and what it can support

Live catalogue observation at **2026-09-25 21:14:35 UTC**:

Dataset	Observed size	Useful existing foundation	Important limit
Anime	21,668 titles	Search, details, statuses, relationships, saved-title membership	Only 170 non-null next-airing timestamps; 5 still future; 0 within seven days
Manga	41,721 titles	Search, details, chapter indexes and collection	Zero next-chapter timestamps; zero titles updated in the preceding seven days by updated_at
Episode rows	51,553	Indexed episode information	Zero future air_at rows at query time
Chapter rows	622,696	Chapter progress and indexed availability	Zero future release-date rows; indexes do not establish official translation schedules
Volume rows	49,793	Volume metadata scaffolding	Zero future release-date rows; generic placeholders do not establish ISBN/edition coverage
Media relations	16,639 edges	Franchise/adaptation navigation	Edge count is not complete franchise coverage
Similar-title store	225,596 rows	Existing recommendation infrastructure	Row count is not judged relevance or user satisfaction
Provider availability	0 rows	Schema and client scaffolding only	No populated provider dataset to expose

165 of the 170 anime next-airing values are already past. The catalogue also contains 295 anime and 7,570 manga marked RELEASING, but these labels alone cannot establish that a future date should exist for every one. There were 1,159 anime rows updated in the last seven days; this does not prove that their airing fields were refreshed correctly. These observations justify a targeted ingestion/freshness investigation, not a claim that the entire import system is stopped.

Practical implication: build the personal agenda around qualified data. “No confirmed date” and “Schedule needs an update” are honest, useful states. Do not display “Nothing coming out this week” merely because the current data has no future timestamps. A broadcast date also does not establish German streaming, dubbed availability or official translated-manga release.

The live user_airing_next view joins anime to saved anime records and selects future next-airing timestamps. It does not filter list status. The client then applies a seven-day window and current user. A product preference is required for whether Planned, Paused, Dropped or Completed titles should generate reminders. Saving a title and following its releases are different intentions.

5. What the additional community evidence changes

These are qualitative signals, not market-share estimates, representative demand rankings or measured retention effects. The full 2025–2026 dates, original links, access depth and limitations are in the source register.

- **Personal relevance:** One Neko commenter explicitly praises Next up and history-based picks; several replies discuss navigation speed or reachability. A useful home must be responsive as well as personalized. RT06
- **Release continuity:** users ask for a calendar of their shows and future seasons; one calendar user replaces a Discord weekday note after a duplication fix. Personal schedules are a concrete workflow, not a reason to build an overwhelming calendar. RT02, RT05
- **Organization should be discoverable:** custom-list creation, ordering and filtering are different needs. Some apparent omissions were existing controls in Settings or Experimental. Anime-Planet's requested exclusion filter already existed. RT03, RT08, RT09
- **Remembering should not depend on push:** SIMKL feedback requests a persistent release inbox. Treat notifications as an optional delivery method for information available inside the product. RT07
- **Social should answer a question:** relevant status, overlap and comparison are more concrete than an undifferentiated activity feed. AniList already offers title-level friend activity and one-person comparisons; Kuro must compete on clarity, privacy and usefulness. RT04
- **Avoid novelty for its own sake:** several participants report that their MAL/AniList/LiveChart workflow is sufficient; one participant prefers collaborative tags to curated public lists. RT11 “Everyone hates their tracker” is unsupported. RT01

Apple's current onboarding guidance also favors learning through use and brief optional guidance. It is a platform design reference, not 2025–2026 community demand evidence. Apple onboarding guidance

6. Authentication: preserve the welcome, complete the experience

The approved reference and the boundary of change

The exact uploaded source is the reference for the welcome. Its normal path is the painted scene, not the debug Genie/orb variant. The scene occupies 52% of geometry height plus the safe area and fades into white; the wordmark and serif headline remain grouped, with the two auth controls below. The TV is removed. The local working copy instead uses 72% scenery and separates the high wordmark from the lower headline. Those local edits are preserved but must not be mistaken for the owner-approved composition.

The actual default welcome reads KuroZenMountains (kuro-zen-loop.jpg), not the separately stored KuroSceneryLoop image. The mountain asset in the approved commit and local checkout has the same Git blob hash, 0d663fcc14227d5ea0a639bb71a5c869b3742f99. The email concept now uses this verified mountain asset. That verifies the source asset, not an on-device render. The email form and authentication logic are otherwise identical between these two AuthView versions, so the source findings below survive the reference correction. The separate five-card post-authentication tour is not the same screen as this pre-authentication welcome; preserve it and assess its routing independently.

Concrete email-form design specification

- **Visual bridge — revised after rejection:** compose the form inside the continuous mountain scene, using its pale sky and mist as space for typography and fields. Preserve scene phase across navigation; pause while editing. This replaces the rejected shallow-banner treatment. Keep a still Reduce Motion equivalent and adapt to keyboard/large text without changing the normal approved welcome.
- **Information order:** Back and identity → explicit Sign in / Create account choice → email → password and visibility control → one primary submit action → recovery when relevant → independently accessible policy links. Preserve entered email when changing mode; clear irrelevant validation errors.
- **Hierarchy:** retain the serif voice for the form title; use shared scaled body/caption styles for fields and help. Labels remain visible when filled. Keep errors next to their field/action and confirmations neutral; do not use pale decorative text for necessary instructions.
- **Phone/keyboard behavior:** use safe-area-aware scrolling; no fixed full-height art above the keyboard. The active input and next action must remain reachable. Email Return advances to password; password Return submits when valid. Visual focus state must match real keyboard focus.
- **Recovery:** Apple cancellation stays useful, provider failure cannot lead to transparent form controls, failed submission keeps input, reset success is clearly confirmed, and back navigation is available when no request is in flight. Never delay operability until a decorative animation completes.
- **Review states:** empty create account, returning sign-in, filled form, email already used, invalid credentials, loading, offline, reset confirmation, Apple cancellation, large type, Reduce Motion and German copy. The browser concept illustrates structure; native keyboard, VoiceOver and iPhone runtime checks are still required.

Q01 — One visual language from scenery to form

Decision: keep the painting, restrained wordmark, serif voice and calm light palette. The email screen should feel like a new composition inside the same moving landscape. The earlier compact-crop/banner proposal was rejected; typography, space and transitions must belong to the animation. On short phones the proposed email artwork gives up height before labels or buttons do; the normal approved welcome composition remains unchanged. Retain a static scene for Reduce Motion.

Why: the user likes the introduction, and current source already contains a strong scene. Replacing it would spend effort where there is no demonstrated need. Functional continuity matters more than a more complicated animation. Current fixed type, raw animation recipes and looping default scene depart from the shared design system.

Alternative rejected: a second full-height hero on the email screen. It would preserve decoration at the expense of keyboard space. **Acceptance:** title, current action and recovery remain clear with keyboard open, large text and German copy; no required control is clipped on the smallest supported phone.

Q02 — Make authentication intention explicit

Decision: retain the approved welcome labels, order and composition. Resolve Create account versus Sign in clearly inside the email destination, with matching heading and submit text. The proposed first-use destination defaults to Create account and makes Sign in immediately visible; returning users can select it without losing entered email. Keep both existing providers. Make the existing policy text independently tappable without changing its normal visual footprint.

Current evidence: welcome defaults to sign-in; registration is a second mode action. Legal agreement copy is plain text. These are observed source facts. **Acceptance:** a first-time participant can select registration without explanation; a returning participant can sign in; both can recover and inspect policy links. Do not claim these changes improve conversion until measured.

Q03 — Repair auth states before increasing animation polish

Decision: a single transition path must reveal the email form after every route into it. Apple cancellation should preserve a useful screen. Errors must appear near the relevant action; password-reset success must be a neutral confirmation, not red error copy. Use actual input-focus and submit chaining; give the password-eye action an accessible label and usable target.

Evidence: source catch routes to email without setting the form's reveal state; several guard errors have no welcome outlet. This predicts a hidden-form risk but has not been reproduced on the installed build. **Acceptance:** fresh welcome→Apple cancel/failure/retry, email offline/invalid credentials, reset success, VoiceOver and Reduce Motion all remain operable.

Q04 — Keep the intro optional and connect it to a first useful action

Decision: preserve both the approved pre-auth welcome and the existing optional post-auth tour. Offer “Bring my list”, “Find my first title” and “Explore” within the first-use home as a separately scoped experiment, without an extra mandatory setup gate or required Taste completion. Returning/deep-linked users should continue their original intent. Taste is optional refinement, with truthful “Not for me / I know this / Calls to me” semantics and a visible exit.

Current mismatch: the last tour card describes Clubs while its primary action enters Taste; the deck is endless, not a promised twelve-choice setup. Onboarding completion is device-local rather than per-account. **Alternative:** retain all five cards, but fix destinations and add useful actions. This is acceptable if testing demonstrates the tour helps; removing it is not a prerequisite. **Acceptance:** reach one saved/current title or a successful import without finishing a deck; a club invitation survives login and education.

7. Home: make the first screen personal without losing Kuro

Q05 — Adapt the home to the user's actual state

Decision: empty account → clear first action plus one editorial invitation. Active account → Continue first, relevant upcoming second, a small social clue where relevant, then the editorial pick. All-caught-up account → next confirmed release or one choice from Planned. Do not show empty social/calendar boxes merely to preserve a fixed dashboard grid.

Why: current default Discover puts a roughly 441-point hero on an illustrative 393-point-wide phone before personal work. Existing screenshots corroborate editorial dominance historically; the size calculation comes from current source. The current hero opens detail, not a progress action. **Tradeoff:** retain discovery's distinctive image treatment below utility; avoid an austere database front page. **Acceptance:** the returning user sees one relevant progress action in the first viewport at default text size, with no forced horizontal hunt.

Q06 — Use explicit, reachable navigation

Updated decision: retain and improve the labelled top selector and current swipe pager; the bottom-navigation proposal is withdrawn. Preserve search and profile as separate accessibility elements. Do not rely on position dots to communicate Taste, Discover, Browse, Collection and Clubs.

Low-risk route: keep the five destinations and add reachable labels/selection; integrate personal modules into Discover. **Exploratory route shown in the phone proposal:** Home / Discover / Collection / Clubs, with Browse inside Discover and Taste entered deliberately. The latter requires navigation and deep-link design; do not slip it into a cosmetic patch. **Acceptance:** users can find their collection, search and club without learning an undocumented gesture, including with VoiceOver.

Q07 — Recommendations must explain usefulness, not just expose infrastructure

Decision: keep one concise, defensible reason and a short set of choices, with More available. Use “From your saved titles” or a verified seed reason when supported; don't invent personalized explanations for generic rails. Allow “Already seen” and “Not interested” without corrupting watch history.

Built versus available: Taste is at 100%; personalized NTY and realm rails are at 0%. More ingestion is not the immediate need. Evaluate those paths on real examples, cold-start behavior and source freshness before increasing exposure. **Acceptance:** recommendations differ meaningfully by valid signals, avoid accidental sequel entry points, respect exclusions and show an honest fallback. Existing heuristic evaluation is not a substitute for owner/user relevance judgment.

8. Lists: make collecting effortless and maintenance dependable

Q08 — Separate fast save from detailed tracking

Decision: a clear Save action adds to Planned with visible confirmation and Undo. A secondary sheet offers Watching/Reading, episode/chapter position, rating, notes and optional named-list membership. Expose “Bring my list” directly from empty Collection and first-use Home using the existing import coordinator.

Why: list creation often begins with one title, not naming a folder and completing metadata. Progressive detail preserves power without making each save expensive. **Tradeoff:** a single action must have a clear default; silently inferring Watching from a bookmark would distort intent. **Acceptance:** save a title in one intentional action; inspect/change its status easily; only confirmed writes receive success feedback.

Q09 — Keep one tracking truth; allow several personal lists

Decision: status is an attribute of the user's title. Personal lists are independent memberships: “Rainy evening”, “With friends”, “Short manga”. A title can belong to several without duplicating progress. Create a named list in the Add to list sheet or Collection, requiring only a name; ordering, description and sharing are optional later.

What is new: Kuro's status filters and club rails do not implement private named personal lists. This needs a separate scoped data model and privacy defaults. **Alternative:** saved smart views may solve most demand with less management. Validate “Continue”, “This season”, “Not started”, “Under one evening” and named-list creation side by side. **Acceptance:** moving between views never rewrites Watching to Dropped merely to hide a title; duplicate membership never duplicates calendar entries.

Q10 — Prefer compact rows for daily updates; preserve the poster view

Decision: current/Continue defaults to compact useful rows; library exploration can remain a poster grid. Remember the chosen view. Each current row exposes position, +1 and numeric edit. A meaningful thumbnail carries identity without consuming the entire phone. Keep rating/advanced sorting available rather than stripping information experienced users value.

Critical dependency: existing search, sorting and counts operate on loaded cards; first page is 90. Search disables the pagination sentinel. The surface must query the whole saved collection or explicitly disclose partial scope. Fix partial batch-success reporting. **Acceptance:** a known title beyond record 90 is findable; global sort/count is correct; a partly failed batch reports successes and failures accurately.

Q11 — Progress must work when totals are unknown, and ratings must survive updates

Decision: always allow position entry. Show “Chapter 37 · total unknown” instead of hiding the editor. Separate total series length, latest indexed availability and personal position. Use consistent +1/direct edit/undo affordances across layouts.

Correctness prerequisite: current UI sends score×10 from the editor and cached 100-scale score from increment; live tables constrain rating to 1–10. Upsert forwards values unchanged. This is a confirmed source-to-live-contract mismatch, not a reproduced user-write incident. Define an explicit scale at the mutation boundary; don't blindly divide every caller, because setUserProgress already converts to ten-point values. **Acceptance:** unrated, 1-rated and 8-rated anime/manga all preserve ratings across editor, +1, chapter marking and batch actions; rapid taps do not lose progress; failure is visible and retryable.

Q12 — Treat migration and export as trust features

Decision: use structured provider IDs and fields throughout preview/apply, paginate fully, preserve unmatched records and disclose unsupported fields. Keep review, reconciliation and undo. Add a portable user-facing export with a round-trip fixture.

Current limits: import defaults to 200, server caps at 400 across media types, and converts structured records into natural-language text that drops fields. A helpful import exists; lossless migration does not. An internal capped text helper is not a backup feature. **Acceptance:** import a mixed collection above 400 with ambiguous names, unknown dates and distinct ratings; no silent loss, duplicates or invented dates. Export remains free, consistent with the earlier revenue direction.

9. Releases and friends: practical reasons to return

Q13 — A personal agenda before a universal calendar

Decision: start with Next for you: date-grouped anime releases belonging to titles the user explicitly follows. Reuse user_airing_next after investigating its freshness. Add an in-app release inbox for meaningful changes; notifications are optional and independently controlled. Remember dismissed/read items without treating a notification dismissal as watching progress.

Data requirements: source timestamp, fetched timestamp, timezone and date precision; distinguish scheduled, released, delayed, unknown and stale. The existing one-next-episode field is not a whole-season schedule. Local reminders are currently scheduled during anime upsert, with no per-title preference check in the inspected caller, and are non-repeating. **Acceptance:** a delayed episode updates/removes stale reminders; OS permission is requested after an explicit alert choice; background delivery is not promised without an implemented scheduler. Never turn empty current data into a false “all caught up” claim.

Q14 — Manga needs its own model

Decision: chapter position, volume read, owned copy and edition are separate. Preserve decimal/special chapters where supported by sources; don't equate the count of indexed rows with a complete total. A title-level reading link must not pretend to open the exact chapter.

What requires new work: independent volume progress is not round-tripped by current normalized records. Official forthcoming editions need work/edition/language/publisher/region/ISBN and qualified dates. Existing 49,793 volume rows do not establish that information. **Acceptance:** distinguish a Japanese chapter from a German volume release;

keep chapter progress unchanged when buying a volume; label a schedule unknown until a verified source exists.

Q15 — Social context beside the title, with truthful wording

Decision: add one small relevant social clue to Continue/Collection and a compact “From your clubs” entry. Use “2 friends tracking” for existing counts; use “finished” only when completed status and permission support it. Show a mutual shortlist (“Both planned”) as a later experiment before a broad activity feed.

Reuse: title comments/statuses, shared-card counts, club preview/unread, rails, polls and pace already exist. Collection prefetches counts but its custom cards do not consume them. **New work:** a cross-title feed needs a bounded server-filtered aggregation that preserves current sharing rules; never assemble it by downloading friends' full lists. Club Active is latest state on rail titles, not complete history. **Acceptance:** private/status/progress visibility remains correct across overlapping clubs, leaving, privacy downgrades and two-person clubs; no spoilers or progress leaks.

Q16 — Create useful return moments without pressure

Decision: test “Resume after a break” using saved position and an optional personal note; “One evening” from Planned using verified duration; “Both planned” for consensual shared selection; “New since you were here” for qualified releases. These are design experiments, not built features or measured wins.

Why: each starts with an existing intention. Avoid streak penalties, guilt about backlog size and repeated taste-training prompts. The actual retention mechanism should be a saved effort or better decision. **Acceptance:** a participant can describe why the module helps; modules disappear when data is insufficient; turning them off does not harm core tracking.

10. Retention measurement and execution order

No current cohort analysis was performed, so no claim is made about which feature presently retains the most Kuro users. The priority ranking combines severity, frequency of the job, existing implementation and evidence confidence—not a fabricated numerical opportunity score.

Order	Work package	Why now	Exit evidence
1	Rating/save consistency; auth transition/error clarity; accessible navigation	Can obstruct the core job	Targeted path tests and phone checks; rated progress round-trip; error/retry/accessibility success
2	Whole-library search and honest import/batch behavior; visible tracking controls	Trust comes before more surface area	>90-title search fixture; >400 mixed import fixture; explicit partial failures
3	Auth/form continuity and state-aware home with compact Continue	Makes existing value visible	Fresh phone layouts, task tests, empty/returning/offline states
4	Schedule freshness investigation, personal agenda, alert preferences	Current live data cannot support a trustworthy weekly agenda	Coverage/freshness monitoring, delay/timezone cases, opt-in delivery checks
5	Personalization canary and contextual social reuse	Already built in part	Judged recommendation quality; no privacy leakage; state-labelled social copy
6	Named lists, full volume/edition support, richer mutual shortlist	Additional models and source dependencies	Clear user demand and successful usability tests; explicit rollout decision

Proposed task-based study: recruit a small formative sample spanning seasonal watchers, returning fans, manga readers, large-library users and private-club users. Eight to twelve participants is a practical first round, not a statistically representative survey. Give tasks without coaching: create a list from three titles, find an older saved item, mark progress with an unknown total, find one's next confirmed episode, determine what a friend actually finished, and recover after an auth failure. Record task success, hesitation, wrong status changes and perceived confidence. Compare the current design to the proposed layout with task order varied; do not ask only which one looks prettier.

Activation definition to instrument: first confirmed meaningful action after authentication: successful import, first saved/current title, or explicit participation in a club. Keep these cohorts distinct; a tour completion or a Taste swipe is not automatically equivalent value. Measure auth-start→success→useful action and time to that action. Avoid logging credentials or private free-text notes.

Return-use measures: D1/D7/D28 return rates for activated cohorts, plus purposeful return events (progress update, useful release open, chosen recommendation or club action). Separate passive app openings from task completion. Establish current baselines before choosing a lift target or sample size. For an experiment, predefine exposure, primary outcome, duration, power assumptions and stopping rules. Compare cohorts with the same catalogue freshness and notification treatment; otherwise a layout change can get credit for an unrelated ingestion fix.

Guardrails: save-error/duplicate-write rate, search misses, import loss, notification opt-outs, stale schedule exposure, slow interactions and privacy violations. Do not ship based on longer session time alone: a successful tracker may let someone finish faster. Navigation alternatives should be tested after the minimal hierarchy change, so the effect of each is understandable.

11. Monetization fits after utility

The earlier free-core decision remains: no fees for tracking, list creation, sort, history or export. Never place a promotion between a user and +1, import confirmation or an auth action. A useful optional next-volume retailer comparison belongs on the relevant edition/detail decision, after edition identity and merchant permission are established. Keep neutral ordering and clear affiliate disclosure. A publisher-funded sample or club discussion kit may be useful if voluntary and separately labelled; it must not buy organic recommendation rank or access to private clubs.

The present provider-availability table is empty and edition-level release evidence is absent. Those facts strengthen the case for validating source/merchant eligibility and user usefulness before a commerce surface. The previous illustrative affiliate economics remain illustrations, not a forecast. The full market/revenue study is included below, with programme evidence, ten monetization decisions and sensitivity examples. No partner approval or new revenue model is implied. The short-list priority is an optional exact-volume referral utility, then a clearly labelled publisher-funded sample if a real partner and useful asset exist. Keep the tracker free; exclude commerce from authentication, progress controls, import and export.

12. Limits and next decision

Approve a focused implementation initiative only after reviewing the concrete designs and ordered work packages. This initiative itself changed research documentation only. Existing application edits were preserved. No build, upload, production mutation, flag rollout, user-list edit or account change was performed.

The important unresolved proof is fresh runtime behavior on an identified build: auth cancellation/recovery, small-screen and VoiceOver layout, rated progress saves, large collection behavior and deep links through onboarding. The connected phone carries build 19; source settings say 17, and there is no verified build-to-commit mapping. Backend object/flag checks are current; the app's installed and public release behavior remains a separate question.

The approved-source reconciliation, detailed source findings and original market/community evidence follow. They are retained so each recommendation can be challenged without repeating the investigation.

13. Source appendix — authentication and first use

Reconciliation note: The following three independent memos preserve their source-only audit scope. Any original uncertainty about live flags, rating constraints, catalogue counts and the user_airing_next view is superseded by the verified observations in sections 2–4 and 17. Their remaining runtime checks are still outstanding.

Audit date: 2026-09-25. Repository: /Users/max/Kuro-Anime-App. Scope: current source, canonical docs and local design specs. No builds, screenshots, hosted checks, account changes or repository edits. Existing AuthView changes were read and preserved (17 additions / 16 deletions): expanded scenery height to 72%, moved brand higher and placed headline above buttons. Those changes are user work, not this audit's implementation.

Reference correction: This memo used the dirty local AuthView, including its 72% scenery layout. The approved uploaded welcome uses 52% and a grouped wordmark/headline. Section 6 and the reference reconciliation supersede that composition assumption; the auth logic and email-form findings remain valid.

All file references below are relative to that repository. Line numbers reflect the working tree inspected, including the dirty AuthView. “Source finding” means directly observable code; “runtime risk” means a predicted user effect requiring reproduction.

Current first-use flow

1. **Root gate:** Kuro/KuroApp.swift:66–112 renders an offline strip, then configuration-error screen, bootstrap spinner, authenticated ContentView, or AuthView. No unauthenticated browsing path exists in this gate. The light appearance is explicitly forced at line32. 2. **Welcome:** Kuro/Views/AuthView.swift:16–27 defaults to .welcome and .signIn. Debug alternatives exist, but normal users see the moving landscape (140–225). The promise mentions anime only (184); the main pill says “Get started” (197) and opens email (195). Apple is second (207). Terms/privacy are text (215). 3. **Email:** AuthView.swift:281–434 presents “Welcome back”, a Sign in / Create account mode pair, email/password, Continue, and Forgot password. A new email user must select Create account after choosing Get started. Registration checks an address after a 500ms debounce (534–553). Both modes demand eight password characters (525–531, 556–559), independent of what existing backend accounts might accept. Backend acceptance is outside this audit. 4. **Success gate:** authenticated state switches to ContentView; KuroRootView briefly overlays its launch mark (Kuro/ContentView.swift:35–65). Its 160ms delay +250ms fade is an explicit extra visual transition, not a load-readiness signal. 5. **First-install tour:** ContentView.swift:140, 471–479 presents OnboardingView as a full-screen cover when a local UserDefaults boolean is false. Five cards: Taste, Discover, Browse, Collection, Clubs (OnboardingView.swift:154–200). Skip is available; completion writes a device-local global boolean (136–139, 247–250). This is once per installation/user-defaults state, not inherently once per account or cross-device. 6. **Destination:** Skip preserves the underlying selection; on an ordinary launch without a pending destination this is Discover (ContentView.swift:131). The final card's primary CTA goes to Taste when its flag is enabled (OnboardingView.swift:84–105, ContentView.swift:474–476); otherwise Get started completes to Discover. The final visible story is Clubs, so the CTA does not match its card. 7. **Taste:** current code explicitly describes an endless deck, not a 12-item onboarding exercise (Kuro/Views/TasteDeckView.swift:7–16, 66–68). Twelve is batch size. Active actions are NOT FOR ME / I KNOW THIS / CALLS TO ME (500, 520, 538). The primary exit callbacks inspected appear in exhausted/empty states (878–889, 1001–1008); normal navigation is the app pager. A newcomer sent here needs a clearly discoverable way to move on. 8. **Incoming links:** app-level pending links are saved before login (KuroApp.swift:33–47), then consumed on ContentView appearance (ContentView.swift:422–428) while the first-use cover is also eligible. Ordering of onboarding versus club/title sheets needs runtime testing; source alone does not prove a lost link.

Findings ordered by user consequence

A01 — Apple failure can enter a form whose controls remain invisible

Evidence: AuthView.swift:47 initializes emailRevealed=false; 98–101 reveals only when AuthView initially appears in email mode. Normal email navigation reveals it through 113–120. But handleAppleSignIn's catch switches to email at 636–640 without calling that reveal path. emailStep uses emailRevealed || reduceMotion (282); AuthStaggerEnter makes unrevealed controls transparent (691–696). **Source conclusion:** inconsistent transition state. **Runtime risk:** from the initial welcome, an Apple cancellation/error can show the error text while hiding header, fields and CTA when Reduce Motion is off. Controls on this screen have individual entrance modifiers. This is not a claim that Apple authentication itself is broken. **Proposal:** use one route-to-email transition helper that always establishes visible/focused state; cancellation should keep the welcome state without presenting a generic failure. Reproduce cancellation, provider failure and retry.

A02 — Header's accessibility wrapper can suppress Search/Profile without an alternative action

Evidence: Search/Profile have labels at ContentView.swift:768–798, but their containing HStack has .accessibilityElement(children: .ignore) at 805; only section-navigation label/value/hint follow (806–808). No adjustable/action modifier exists in this file. Section paging is a manual HStack+offset (552–571), and dots are not buttons (737–755). **Source conclusion:** child semantics are intentionally ignored by parent. **Runtime risk:** VoiceOver users may lose Search/Profile and cannot use the hinted ordinary swipe to change sections as described. VoiceOver gesture behavior requires testing. **Proposal:** preserve Search/Profile as separate accessible controls and make the section title an explicit accessible selector or adjustable control. Don't make onboarding depend on learning an invisible gesture.

A03 — Account creation intention is ambiguous before the form

Evidence: AuthView.swift:27 sign-in default; primary “Get started” only calls goToEmail (192–197); next headline is “Welcome back” (312). Registration requires a second mode action (332–335). **Updated proposal:** preserve the approved welcome wording and resolve account intent inside the email destination with explicit modes and matching heading/submit text. The earlier suggestion to relabel the welcome is superseded by the owner reference lock. No new identity provider is required.

A04 — Terms/privacy appear as agreement language but cannot be opened here

Evidence: welcome AuthView.swift:215–220 is plain Text; alternative debug welcome also uses plain Text (266–271). No Link or URL action exists in AuthView. A privacy Link exists only after authentication at Kuro/Views/ProfileView.swift:824–831. **Proposal:** real independently tappable Terms and Privacy links with meaningful labels before signup; confirm destination contents separately. This is a source usability observation, not a legal-compliance verdict.

A05 — Motion support is inconsistent on the actual default scene

Evidence: top view reads Reduce Motion (AuthView.swift:11), but default AuthSceneryLoop has unconditional 30Hz TimelineView and changing offset (1298–1339), with no reduction branch. Press and form entrance use raw springs (676–697); the curtain still uses move transitions (80–93). Onboarding directly animates cards/entrance (OnboardingView.swift:60, 129–132). **Proposal:** static crop for Reduce Motion; same content and controls immediately visible; use shared motion policy. Reducing duration alone does not remove positional motion. Do not remove the user's chosen scenery or composition.

A06 — Small-screen resilience is incomplete; distinguish it from already-present scrolling

Evidence: welcome is GeometryReader+VStack, not scrollable (140–147, 171–224), with 72% scenery in the local edited copy (52% in the approved GitHub reference), 13%-height top placement, fixed33pt headline, fixed54pt buttons and fixed margins. Email is already a ScrollView (310)—do not claim it lacks scrolling. Tour card pager is capped at420pt (OnboardingView.swift:42), and its card has several fixed gaps,80pt icon and nested horizontal padding (209–241); bottom area reserves48pt (124). **Risk:** narrow/short devices, German text and large type can require more height than the tour reserves; no runtime truncation is established. The welcome's fixed typography can stay small rather than honor requested text enlargement. **Proposal:** adapt art height to remaining content space; let functional content scroll at accessibility sizes; keep both auth choices and legal links reachable. Allow tour content height to grow/scroll rather than shrink text or crop it.

A07 — Auth uses fixed typography and faint labels despite scaled design tokens existing

Evidence: fixed sizes across AuthView.swift:175, 185, 198, 216, 303, 313, 321, 378, 402, 443, 485, 496, 587; labels/prompts use low ink opacity at 322, 444, 486, 491–493. KuroDesignSystem.swift:96–120 provides scaled typography. Onboarding already uses scaled tokens; this is a difference, not an app-wide absence of Dynamic Type. **Proposal:** retain serif hierarchy while mapping text to scalable tokens. Strengthen field labels, secondary mode and recovery information. Runtime acceptance should include larger accessibility sizes and contrast over the landscape; no universal contrast failure is claimed from screenshots we did not take.

A08 — Field focus and button semantics are under-specified

Evidence: focus is ordinary @State (AuthView.swift:35) set on a tap (498), not FocusState; no .focused, return-key submit chain or .onSubmit exists in this file. Password reveal has a15pt eye glyph with no explicit label/hint/44pt frame (364–369). Back has a44pt area (291–295) but no explicit Back label. Mode tabs have visual selection but no explicit selected accessibility trait (436–452). **Proposal:** true input focus; email Next→password, password submit, consistent hide/show semantics, generous eye-button hit area; selected mode announced. Explicit labels should survive symbol/localization changes. These are source gaps; actual system-inferred labels must be checked before describing them as wholly inaccessible.

A09 — Success feedback reuses error presentation; welcome errors have no dedicated outlet

Evidence: forgotPassword assigns successful “email sent” copy to errorText (654–655), rendered with red accent (376–380). Non-throwing Apple guard failures assign errorText then return without changing away from welcome (457–460, 619–630), but welcome has no error view. Auth observer also only sets errorText (105–108). **Proposal:** separate neutral confirmation, actionable error and working states. Give Apple errors a visible welcome-level outlet and prevent cancellation being treated like a fault. Provide a recovery state with resend/back affordances, pending backend verification by root.

A10 — Five-page tour contains no direct first useful action until its end

Evidence: every nonfinal card uses Continue and Skip (OnboardingView.swift:57–83); cards are feature descriptions (154–200). Final Clubs copy promises keeping conversation together (193–198), but primary CTA enters Taste (86–87). There is no import/start-tracking action in this tour. **Proposal:** one optional setup choice after auth: Find something / Bring my list / Start tracking, with “Explore now” always available. Existing routes should be reused where supported. If the five-card tour remains, fix final card/action pairing and provide page position semantics. This is a product recommendation, not an implemented route or a claim that current import infrastructure is missing.

A11 — Taste tutorial language can teach the wrong signal

Evidence: onboarding says “pass, know, or love” at OnboardingView.swift:161, while active Deck uses “NOT FOR ME” (TasteDeckView.swift:500); current deck is endless (11–16). **Proposal:** exact action language, explain that dislike is not a neutral skip, avoid promising “a few choices then done” unless an actual finite introduction/exit is added. Offer neutral “Not sure” if supported by the action model; root should verify downstream semantics before implementation.

A12 — Local onboarding state and incoming intents need an explicit policy

Evidence: single UserDefaults key OnboardingView.swift:138, 249; pending link consumption ContentView.swift:422–428 and first-use cover 471–479 independently trigger. **Proposal:** treat onboarding as optional feature education, not a fragile account setup gate. Club/title invite intent should survive auth and any tour. Decide deliberately whether a second account/new device sees education; do not assume the current flag is per-user. Validate first-install invitation, auth cancellation, restart halfway and account switching.

Concrete proposed hierarchy (preserve the current visual direction)

Welcome: painting and Kuro wordmark remain. One short value sentence includes anime **and manga**. Two plainly named authentication choices, an obvious returning-user Sign in route, and real Terms/Privacy links. Art yields space to functional content on short screens. A transient auth error appears beside the relevant choice; choosing/cancelling Apple does not unexpectedly dump users into another hidden form.

Email: back + small wordmark → one purpose title (Create account / Sign in) → labelled fields and inline guidance → primary matching action → recovery/alternate-mode action. One error region and a distinct neutral success state. Preserve entered email when switching modes; make keyboard navigation functional.

First success: route to the user's pending title/club intent if one exists. Otherwise show one optional action choice or let them immediately explore Discover. Import an existing list should be discoverable without completing taste training. Taste remains voluntary, explains its three actions truthfully and offers a visible path back to Discover. This proposal does not require changing the five-page main pager.

Learning later: contextual hints at the first actual tracking action and first pager use. A first-time map may help, but a five-page marketing tour should not be the only explanation of navigation. “Done” means the user reached a useful state, not that they tapped through feature copy.

Verification matrix for root (not performed in this audit)

- Default welcome + email + create modes on the smallest supported phone class and iPhone17Pro; portrait/landscape only if supported by app configuration.
- Default/large/accessibility Dynamic Type; German and English; keyboard open; long address and long error text.
- VoiceOver order: welcome CTA→Apple→legal links; email labels→password reveal→submit→recovery; post-login Search/Profile/section choice.
- Reduce Motion on from cold launch; setting changed while view is open.
- Apple cancel, nonce/credential/provider failure, retry; fresh path without first visiting email is essential for A01.
- Email submission while availability lookup is pending; offline→online; reset request success vs failure. Root owns backend/RPC semantics and hosted proof.
- First-install skip, complete tour, sign-out/second-account, restored session and club/title deep link through authentication.
- Confirm exact accessibility tree and rendered layout before classifying source risks as reproduced UI defects.

Documentation/context boundaries

Read: AGENTS.md, current-state opening/UI/auth sections, plain current-state auth notes, docs/superpowers/specs/2026-07-30-auth-light-redesign.md, docs/superpowers/specs/2026-07-31-adr-design-direction.md, and July31 overnight epic freeze context. July30 spec says Apple + explicit email welcome choices and existing auth backends; source now has Get started + landscape default and a retained debug Genie scene. July31 epic specifically froze Auth visuals during that prior initiative (130–136); this audit makes no edits and does not assume permission to overwrite current user work. Source is current implementation authority for this audit, while specs explain design intent. Root should reconcile current-state text claiming the final card is a taste card with the actual final Clubs card.

No source recommendation here proves production behavior or validates current backend email-confirmation settings. No repo files were changed; root owns initiative documentation updates.

14. Source appendix — collection, import and manga

Source-only audit, 25 September 2026. No code changes, live database reads, build, simulator or production validation. Paths below are relative to /Users/max/Kuro-Anime-App; line references are to files inspected during this audit. Existing capability means present in current local source, not confirmed deployed. Parent agent handles database truth and final documentation.

Main conclusion

Kuro already has an interleaved anime/manga collection, status filters, two layouts, direct progress actions, AniList import preview/reconciliation, manga chapter previews and personal anime airing data. It does not need these described as greenfield features. The strongest opportunities are to make existing actions reliable and visible, complete the metadata round-trip, make collection search cover the entire collection, and expose existing anime schedule data as a personal agenda. Named personal lists, true reading history, independent volume progress, and verified upcoming translated manga editions require additional modeling.

A source-level rating contract conflict is more urgent than visual polish: the database migration constrains list ratings to 1–10, while two UI mutation paths send values on a 10–100 scale. This predicts failed saves and failed progress increments for many rated entries; it needs a targeted runtime check against current database constraints.

1. Actual collection versus personal named lists

Exists. Kuro/Views/EditorialCollectionView.swift:29 defines All, Watching, Completed, Planned, Paused and Dropped. Media scope is All/Anime/Manga (:51), completed verdict filters are Unsorted/Masterpiece/Okay/Bad (:57), and sort choices are Last Updated, Title A–Z, Rating, My Rating and Progress (:84). These are views of one per-title status record, not independent user-created lists.

The normalized mutation signature in Kuro/Services/SupabaseService+UserLists.swift:36 accepts media, status, progress, rating, notes and verdict. Payloads at :52 and :62 do not contain custom list IDs, names, tags or memberships. Upsert conflicts are user_id,anime_id (:74) and user_id,manga_id (:86). The corresponding local schema is supabase/migrations/20260807000000_baseline_capture_v1.sql:177 and :436.

Gap. A personal list such as “Watch with Emma” or “Short manga for travel,” with titles belonging to more than one list, is not represented in these records. A search across Kuro source found no personal custom-list creation/model under relevant names. Private club rails are a different group-sharing feature; do not count them as evidence of private named list support.

Decision. Keep status as a title attribute. If named lists are validated, model separate memberships and let a title appear in several lists without changing its Watching/Reading status. Rename generic “Edit List” to “Edit tracking” or similar if user testing shows that it implies list creation. That entry currently appears in a card context menu (Kuro/Views/EditorialCollectionComponents.swift:623).

Further fidelity gaps. .repeating is converted to READING/WATCHING (SupabaseService+UserLists.swift:30); the reverse decoder has no repeating case (:10). Rereading/rewatching cannot round-trip as a distinct status through this path. Model properties exist for volume progress and start/completion dates (Kuro/Models/SupabaseModels.swift:547), but normalized fetch fills progressVolumes, startedAt and completedAt with nil (SupabaseService+UserLists.swift:249, :253, :269, :273). Do not infer supported historical tracking from optional model properties alone.

2. Collection presentation, discoverability and large-library behavior

Exists. Grid is the default (EditorialCollectionView.swift:12). Grid/list switching is present (:345), followed by two render paths (:616). Sorting, media, provider and language controls exist in the horizontal control strip (:328). Provider/language filtering depends on streaming_availability_v1 (:108, :114); source alone does not establish its live rollout. Cached data is explicitly identified offline (:607).

Critical scope limitation. Collection cards come from collectionFeedItems (:92). The first interleaved page fetch is 90 records (Kuro/Services/SupabaseService+Collection.swift:141). Search filters the current items array, either using an on-device parsed intent or title matching (EditorialCollectionView.swift:666, :674, :705). Once searchResults is non-nil, the pagination sentinel is suppressed (:634). Therefore an off-page saved title may be invisible to search even though it exists in the user's library. Sorting is also applied to loaded items (:135), not the full server collection. Status counts iterate loaded cards (:160). These are source-derived behaviors requiring a >90-title fixture to demonstrate their user impact.

Backend paging already exists: collection_feed_page (SupabaseService+Collection.swift:408), collection_anime_page (:442) and collection_manga_page (:475). Extend these contracts deliberately for full-library search/sort/count semantics, rather than merely changing the UI labels.

Design decisions. Make Library the explicit home for tracking. Give quick progress and edit actions visible labels in the appropriate layout. Preserve both grid and compact list because they serve browsing and frequent maintenance respectively. Treat search as whole-library search; if a temporary loaded-only mode is retained, label its scope rather than presenting an empty result as authoritative. Manga-only views should say Reading: current collection filters and batch action wording use Watching (EditorialCollectionView.swift:29, :194).

Batch caution. EditorialCollectionView.swift:750 loops through status changes but ignores the per-write Bool result and emits a final success message. Validate partial failures and report changed versus failed records. No claim here that nil optional fields necessarily clear values—the encoding/server behavior must be tested.

3. Unknown totals and progress: inconsistent, not entirely absent

Surface	Known total	Unknown total	Source
Tracking editor	Shows progress for Current/Completed/Repeating	Hides progress editor because total becomes 0	Kuro/Views/AddToListSheet.swift:23, :28, :94
Grid inline progress	Shows bar/count and +1	Hidden because display requires a positive total	EditorialCollectionComponents.swift:394, :537
Grid context menu	+1 for eligible Current entries	+1 is allowed when total equals 0	EditorialCollectionComponents.swift:384, :604
Compact list row	Visible +1 for eligible Current entries	Visible +1 allowed; count can display “X ch/ep”	EditorialCollectionComponents.swift:742, :752, :819
Chapter detail	Mark chapter read	Uses chapter number without total cap when chapterCount is nil	Kuro/Views/DetailPages/MangaDetailView.swift:748

Do not state “unknown totals cannot be tracked.” The actual issue is that the default grid and editor hide the most discoverable controls, while other paths work. Selecting Completed auto-fills a known total only if current progress is zero (AddToListSheet.swift:173); it does not invent an unknown total.

Decision. Always expose an editable “Read through chapter / Watched through episode” value for active tracking, independent of total. Render `37 read · total unknown` instead of an empty editor or fabricated denominator. Keep latest known available, final total and personal progress separate. Offer +1, direct numeric edit and undo consistently across grid/list/detail. Preserve correct progress when the provider revises a total downward; a total mismatch should be explained rather than silently discarding progress.

Offline and concurrency boundary. The editor explicitly refuses an offline save with a visible message (AddToListSheet.swift:165). SupabaseService.swift:2778 computes +1 from the cached entry and performs a full upsert; it does not show a durable offline queue or increment serialization in this function. Upsert refreshes all user lists, separate collection pages and the interleaved feed (SupabaseService+UserLists.swift:109). Validate repeated rapid taps, offline quick actions and error feedback before promising instant reliable progress. A cached library is not evidence of offline writes.

4. Rating contract conflict: targeted correctness check first

Stored ratings are locally constrained to 1–10 (baseline_capture_v1.sql:1332, :1431). The normalized fetch transforms 8 into `UserList.score = 80` (SupabaseService+UserLists.swift:251, :271). `setUserProgress` converts it back to 8 before saving (:637). The importer also clamps list rating to 0–10 (supabase/functions/concierge-apply/index.ts:284, :389).

However, the tracking editor loads `entry.score / 10` (AddToListSheet.swift:148) and then sends `score * 10` as the database rating (:180). The quick increment passes `entry.score` unchanged (SupabaseService.swift:2786). With those constraints active, score 8 yields rating 80 and should fail; score 1 yields 10 and could save an incorrect rating. This is a concrete contract inconsistency, not a visual preference. Confirm current hosted constraints and reproduce save/increment for an unrated, 1-rated and 8-rated title. Correct the scale at the shared mutation boundary and verify unrelated progress edits preserve ratings. No fix made in this audit.

5. Personal upcoming anime: much of the data path already exists

SupabaseService.swift:312 defines a user-scoped upcoming model and :933 loads upcoming data during bootstrap. SupabaseService+UserLists.swift:394 queries `user_airing_next`, filters current user and the next seven days, sorts by airing time and limits to 500 (:400–:406). The collection grid finds the matching title and displays a countdown for a future RELEASING anime (EditorialCollectionComponents.swift:360, :587). This is existing functionality.

Anime detail also distinguishes a scheduled next episode from a schedule awaiting an update (Kuro/Views/DetailPages/AnimeDetailView.swift:263, :323), displays Next Up (:380), and supplies fallback episode preview rows when episode records are absent (:909, :954). Do not recommend replacing these useful uncertainty states with invented dates.

What ingestion already supports. AniList import requests `startDate` and `nextAiringEpisode` (supabase/functions/bulk-import-anime/index.ts:292, :296); regular writes populate `next_episode_number`, `next_airing_at` and start date components (:367). The schedule-only update path writes next episode/time separately (:333). Combined with saved-title membership this can support a personal next-airing agenda without a new schedule vendor for that first scope.

Actual gap. No separate user-facing personal upcoming agenda was found. A date-grouped “Next for you” view would expose the existing data in a more usable place than per-card countdowns. A broader season calendar requires more than one next episode per title. Partial premiere dates must retain month/year precision; a broadcast timestamp is not proof of a dubbed release or local streaming availability.

Notifications exist, but control/reliability needs review. Anime upsert calls `scheduleAiringNotifications` (SupabaseService+UserLists.swift:113). It requests OS authorization if needed (:459) and schedules two non-repeating local notifications: at airing and one hour earlier (:480, :506, :510). This function does not check a per-title notification

preference or list status. A user may be prompted simply from editing an anime record. No all-title future rescheduling loop was observed in the paths read. Validate rollover, cancelled/delayed airings, expired schedules, denied permission and notification limits before claiming ongoing release monitoring. Design a per-title opt-in and digest choice around user intent.

Live dependency for parent. user_airing_next is referenced in app code, but a CREATE VIEW definition was not located by source search; local migrations include view alteration rather than its definition. Confirm its actual joins, eligible statuses, security semantics and populated future rows. The app's seven-day filter is established; live view behavior is not.

6. Manga chapters, volumes and forthcoming editions

Exists. Manga detail renders Chapters independently of a known total (MangaDetailView.swift:93). It fetches ten rows from personal progress +1, chapter enrichment status and a legal reading link (:694). It shows Continue with chapter N (:637), can mark a chapter read (:748), and has syncing/unresolved/no-source copy (:764). The backend dependencies are chapters (SupabaseService+UserLists.swift:583, :602), get_manga_chapter_status (:620) and the legal external-link helper (MangaDetailView.swift:724).

Important presentation limit. Opening a chapter currently opens the same title-level legalReadLink, not a chapter-specific destination (MangaDetailView.swift:716). Do not label this as exact chapter navigation. The displayed known total may fall back to chapter-row count (:611) and is rendered as TOTAL (:630); partial row coverage should be labeled “chapters indexed” rather than conflated with final series length.

Volumes are presentational, not a personal edition collection. MangaDetailView.swift:1098 shows at most the first six generic volume cards from volumeCount; :1132 notes that no volume detail destination exists. The normalized user record persists one integer progress. Although UserList.progressVolumes exists, current fetch fills it nil and the payload omits it. There is no demonstrated independent volume-read count, owned-volume inventory, edition identity or edition-aware upcoming alert in this path.

Schema versus actual ingestion. baseline_capture_v1.sql:210 provides chapter date fields, and :507 provides volume dates, ISBN and prices. These columns alone do not prove usable release data. The bulk manga import requests aggregate chapters/volumes and start date (supabase/functions/bulk-import-manga/index.ts:311, :367). Its opt-in placeholders contain only manga ID, number and generic chapter title (:414); volume placeholders similarly lack date/ISBN evidence (:433).

The MangaDex enrichment fetches aggregate chapter indexes with an optional language filter (supabase/functions/manga-chapter-enrich/index.ts:1757). It accepts positive integer chapter keys and skips fractional chapter numbering (:1766). Upsert rows contain manga ID, integer number and generic title (:1818); this path does not persist future publication dates, edition identity, or a provenance/language record for every chapter. Aggregate availability therefore cannot establish official translated release dates. Do not turn a source's chapter count into a universal “latest chapter.”

Decision. Support chapter and volume progress independently. Before exact-volume buying assistance or alerts, establish work/edition/volume identity, language, publisher, region, ISBN and date precision with source attribution. Separate original publication, official translation and reader-available release. A reliable manga upcoming calendar requires a verified release feed or publisher/retailer integration, not just rendering existing date columns. Preserve special chapters instead of silently dropping 0/decimal labels if that segment is in scope.

7. Import and export: guided onboarding exists; fidelity is incomplete

A Profile action explicitly says Import from AniList (Kuro/Views/ProfileView.swift:375). It opens a dedicated preview/import sheet (:129); this is not solely hidden behind conversational prompting. The sheet lets the user choose anime/manga and statuses (Kuro/Views/ConciergeView+AniListImport.swift:195). The coordinator has preview, confirm, retry and actionable errors (Kuro/Views/ConciergeAniListImportCoordinator.swift:47, :89, :153). Profile import checks ambiguity and existing entries before auto-apply (ProfileView.swift:566) and can route to Concierge review.

Limits. The app defaults to 200 items (ConciergeAniListImportCoordinator.swift:53); the endpoint permits at most 400 (supabase/functions/concierge-import-anilist/index.ts:250). It reads types sequentially, so one large first type can consume the combined cap (:283, :309). The profile contains a truncated-import warning (ProfileView.swift:760). This is a capped snapshot, not proven full-library migration or continuous sync.

The AniList query fetches status, progress, score, update time and media identity (concierge-import-anilist/index.ts:126). The importer then converts records into natural-language lines (:89). Those lines preserve current progress but discard score and stable ID; completed, paused and dropped lines omit progress. Only up to eight preview samples retain their original metadata (:188). Confirmation reuses cached text (ConciergeAniListImportCoordinator.swift:108), which is then parsed/reconciled. This is a helpful assisted import but not a lossless structured import. The query also does not request volume progress, notes, custom lists or historical dates.

Decision. Retain structured external IDs and source fields through preview and apply, use direct provider-ID mapping when available, show unsupported fields and unmatched items explicitly, and paginate until complete with resumable checkpoints. Keep review/undo safeguards. Do not imply current capabilities are absent; improve their fidelity and clarity.

Export boundary. SupabaseService.swift:2583 contains an internal library-to-text helper for Concierge, capped at 400, with status/progress text and truncation. No portable user-facing file export was found in the Kuro views searched; ShareLink usages are title/club sharing, not a full collection backup. A user-owned export needs stable IDs, statuses, ratings, notes, independent progress dimensions and dates, plus a tested re-import round-trip.

8. Suggested implementation order and validation gates

1. **Correct mutation contracts.** Rating scale, successful/failed progress feedback, rapid taps and partial batch failures. Test with rated and unrated anime/manga, unknown totals, offline state and a revised total. 2. **Complete tracking controls.** Consistent +1/direct edit for unknown totals, visible compact-row access, precise count labels and Reading wording. Test discoverability without teaching users the long-press menu. 3. **Make the library trustworthy at scale.** Server-scoped search/sort/count and complete import/export semantics. Validate a fixture above the 90-card page and 200/400 import caps, with duplicates, ambiguous titles and both media types. 4. **Expose existing personal anime schedule.** Group the existing next-airing data by day, preserve TBA/stale states, expose explicit notification controls. Verify the live view and delayed/rolled-over episodes. 5. **Add real volume/edition support where validated.** Treat this as data-model and source work, not just a new volume selector. Validate one ongoing translation, one omnibus edition, decimal/special chapters and uncertain release dates. 6. **Validate named custom lists separately.** Prototype list creation and multi-membership with real users; do not conflate it with the existing status filters or club rails.

Feature flag names relevant to parent verification are streaming_availability_v1, taste_deck_v1 (changes first navigation page), and on-device/Concierge capability flags. Kuro/ContentView.swift:150 selects Taste versus Concierge while keeping Discover/Browse/Collection/Clubs; :131 defaults to Discover. Kuro/Services/FeatureFlags.swift:64 fetches rollout data. No live flag percentage can be inferred from the source snapshot.

15. Source appendix — home and social

Audit date: 2026-09-25. Local checkout HEAD 697a5bc. Read-only source inspection; no app run, screenshots, API mutations, deployment claims or production flag assertions. References below are relative to /Users/max/Kuro-Anime-App; line numbers describe the working files inspected. Root agent is checking live backend state separately.

Main finding

Kuro already implements most of the ingredients for a useful returning-user home, but the current home leads with editorial discovery. Personal continuation, personal next releases and friends' progress are distributed across Collection, details and Clubs. This is primarily a hierarchy/integration opportunity, with some privacy-aware backend work needed for a truthful cross-title friends feed. It is not evidence that these existing components are live, populated or reliably used.

What opens after login

- Returning user: KuroMainView initializes selection and mounted sections to Discover (Kuro/ContentView.swift:131, :134). A handled deep link or launch argument can override this; do not call Discover unconditional for all launches.
- First launch: onboarding is controlled by local completion state (ContentView.swift:140). Its five-card introduction offers Skip and, when Taste is enabled, “Teach Kuro Your Taste” (Kuro/Views/OnboardingView.swift:77, :84, :136). The completion handler moves to Taste only for that choice (ContentView.swift:471); otherwise Discover remains selected.
- Navigation: Taste, Discover, Browse, Collection, Clubs when taste_deck_v1 evaluates true; Concierge occupies the first position otherwise (ContentView.swift:150). The visible page is a horizontal swipe pager; header uses the same order (:611). Clubs unread activity is a small dot on its positional indicator, hidden from accessibility (:736–755).

Actual Discover hierarchy

Primary render sequence in Kuro/Views/EditorialDiscoverView.swift:

1. TYPE menu, default All (:19, :113–134). 2. THE ONE THING daily feature, or skeleton/legacy featured-anime fallback (:138–154). This feature is not gated by the personalization flag; its RPC result is resolved independently (:490–495; Kuro/Services/SupabaseService+Discover.swift:188–202). 3. AIRING TODAY: catalog airing-next-24-hours rail, up to ten cards (:156–164). This is not the separate user_airing_next personal-list query. 4. BECAUSE YOU LOVED if eligible data exists and TYPE is All, otherwise anime NEW TO YOU (:167–186). 5. THE SHELF if realm flag is enabled and results exist (:188–200). 6. ESSENTIAL ANIME (:202–210). 7. HIDDEN GEM if realm flag is enabled and its type matches (:213–226). 8. NEW TO YOU (MANGA), TRENDING, ESSENTIAL MANGA (:228–258). 9. SHOW MORE, persisted with kuro_discover_show_more, reveals classics/current-season/top-rated/newly-added and additional manga rails (:18, :263–286, :291 onward). It is progressive disclosure of more discovery, not a personal dashboard.

No Continue, personal upcoming, friends-watched or friends-upcoming rail appears in this render hierarchy. The page has offline cached-data disclosure and retry (:65–110) and refresh, so failure handling is not wholly missing.

First-screen implication

Hero width is min(currentWidth - 40, 520) (EditorialDiscoverView.swift:56–64), and hero height is floor(width * 1.25) (Kuro/Views/DiscoverOneThingCard.swift:22–24). On an illustrative 393-point viewport this is about 441 points of hero height, before header/menu/spacing. This is arithmetic from source, not measured screen coverage. The card's action opens the detail sheet; no inline add/progress action is present (DiscoverOneThingCard.swift:123–136). A returning user therefore receives a large discovery invitation before personal next actions.

Existing capabilities that can be reused

Capability	Source evidence	What it does and what it does not prove
Current progress and quick increment	Kuro/Views/EditorialCollectionComponents.swift:384–387, :536–584, :819–835	Existing +1 EP/+1 CH actions for current items, including unknown-total handling. Reuse this interaction for a compact Continue component; do not invent a playback-resume promise.
Personal next seven days	Kuro/Services/SupabaseService+UserLists.swift:371–418	Throttled/deduplicated query of user_airing_next scoped to current user, future window, sorted by airing, capped at 500; source already supplies upcoming data. Population/freshness requires runtime/backend verification.
Per-title countdown	EditorialCollectionComponents.swift:360–370, :587–592	Grid anime cards show next episode countdown for releasing titles with a future time. The custom list-row implementation has progress controls but no matching countdown call.
Airing reminders	SupabaseService+UserLists.swift:113–115, :458–510	Every successful anime list upsert calls scheduling; future airing triggers OS permission if not determined, then “airs now” and one-hour-before local notifications. No per-title opt-in check appears in this caller. Removal cancels its two notification IDs (:355, :520–523). This is not evidence of a continuous push scheduler or automatic rescheduling of all future episodes.
Title-level friend statuses/comments	Kuro/Services/SupabaseService+Social.swift:24–36, :57–62; Kuro/Views/DetailPages/FriendsActivitySection.swift:27–66, :95–146	Displays permitted named friends tracking the title, statuses/progress and comments. Both detail pages gate the section (AnimeDetailView.swift:121; MangaDetailView.swift:115).
Catalog-card social proof	Kuro/Views/KuroRefinedCard.swift:194–204, :371, :539; Kuro/Views/Cards.swift:268, :476	Shared card variants can display “N friends” when cache count >0 and social flag true. This is “tracking”, not “finished” or “liked”.
Club list retention	Kuro/Views/ClubsView.swift:138–164, :355–440; SupabaseService+Clubs.swift:210–227	Clubs show preview/time/cover/member information and per-club unread status based on locally remembered timestamp. Discovery of social value currently requires navigation to Clubs.
Club activity	Kuro/Views/ClubDetailView.swift:25–35; Kuro/Views/ClubDetailTabComponents.swift:651–676, :693–742, :804–821	Detail defaults to RAILS; second tab is ACTIVE, not docs' “This Week”. Feed is built from latest member_statuses of club rail items, sorted by updated time. It is not a durable action/event history or all titles tracked by every friend.
Pace synchronization	ClubDetailTabComponents.swift:745–781	Existing behind-median banner is additionally restricted to progress-sharing club and >=3 members, plus flag. Do not generalize to every friend or duo.
Taste onboarding and control	Kuro/Views/TasteDeckView.swift:482–520, :711–730, :754–761	Visible Not For Me / I Know This / Love controls, judged count + Leanings, undo affordance. :1164–1188 discloses profile freshness and signals not folded in yet. Taste learning is already an experience, not a blank slate.

Precise flag consumption

Flag accessors are in Kuro/Services/FeatureFlags.swift:27–44. Evaluation at :114–135 checks override, presence/enabled, region, percentage and user hash; missing flag evaluates false. Cache survives fetch failures (:54–56). Therefore neither source accessor nor migration default establishes current user exposure.

Flag	Actual source consumption
taste_deck_v1	Pager substitution (ContentView.swift:150) and onboarding CTA/destination. Does not itself enable all personalized Discover rails.
personalized_new_to_you_v1	EditorialDiscoverView.swift:500–529: first request Because You; require >=4 results; else request personalized anime NTY; empty/error preserves bundle rail. Because You renders only in All mode (:169). Manga NTY has no analogous personalized replacement in this branch.

Flag	Actual source consumption
discover_realm_rails_v1	Independently fetches Shelf and Hidden Gem (EditorialDiscoverView.swift:532–556), also checked at rendering (:189, :214). Does not gate the daily One Thing hero.
social_activity_v1	Discover/Collection count prefetch, shared-card count display, anime/manga detail friend section. Does not create a home friend rail.
clubs_notifications_v1	SupabaseService+Clubs.swift:413–436 protects global badge check. Badge is cleared when entering Clubs (ContentView.swift:358–362; SupabaseService+Clubs.swift:444–446), independently of per-club last-seen dots.

Flag	Actual source consumption
clubs_realtime_v1	Subscription guard at Kuro/Services/SupabaseService+ClubRealtime.swift:13. A live flag alone does not show whether subscriptions are active/reliable.
clubs_pace_sync_v1	Activity pace banner and other rail pace UI (ClubDetailTabComponents.swift:234, :745); also requires sharing/member conditions.
clubs_reactions_v1	Rail reaction UI (ClubDetailTabComponents.swift:430).

Flag	Actual source consumption
clubs_list_enriched_v1	Accessor exists (FeatureFlags.swift:34) but repository search found no consumer elsewhere in Kuro. fetchMyClubs always tries fetch_my_clubs_loading then fetch_my_clubs_enriched (SupabaseService+Clubs.swift:188–202); ClubsView always renders JournalClubCard. Treat this flag as ineffective for rolling back this client presentation until deliberately wired.
streaming_availability_v1	Collection service/language filters (EditorialCollectionView.swift:108–117) and related availability metadata prefetch (:722–746); separate from personal airing/calendar data.

Current docs label Taste/social/club retention flags on and personalized/realm rails off in some sections (CURRENT_APP_STATE.md:539–546, :898–910); this memo deliberately does not turn those labels into live assertions. Historic migration percentages differ, so parent should use current database checks.

Gaps and important distinctions

1. **Personal continuation exists away from the default page.** Collection defaults to All + last updated + grid (EditorialCollectionView.swift:10–12), with filters including Watching and Planned (:29–46). A compact Continue module can use current items and existing increment behavior without new conceptual tracking machinery. Preserve bulk editing/filters in Collection. 2. **Personal upcoming is fetched but not surfaced as a home/calendar module.** Source stores upcomingAirings (SupabaseService.swift:320) and bootstraps seven-day data (:933); discovered display is Collection grid countdown. The Discover AIRING TODAY rail is a different catalog-centric surface. Reuse the user-specific query for “Your next episodes”; do not label airing time as regional legal-streaming availability. 3. **Collection social prefetch has no custom-card consumer.** EditorialCollectionView.swift:722–746 prefetches counts under the social flag, but EditorialCollectionComponents.swift renders its own CollectionGridCard (:103–121) and custom row; search finds no friend/count consumer in that file. It is therefore misleading to assume existing shared-card badges automatically appear in Collection. 4. **New Discover surfaces are omitted from explicit friend-count warmup.** EditorialDiscoverView.swift:561–571 gathers essentials/trending/top-rated/NTY/newly-added/current-season; it does not explicitly include daily feature, Airing Today, Because You, Shelf or Hidden Gem, nor classics. Some may overlap. This is an integration gap/opportunity, not proof of an observed missing badge. The concatenation truncates to first 100 before deduplication. 5. **Friends tracking is not friends seen.** Latest source RPC counts all matching anime/manga list rows without completed-status restriction (supabase/migrations/20260731040000_clubs_trust_pack_v1.sql:271–284). A “Friends watched” label requires completed-filtered evidence or explicit display of status. “Friends upcoming” additionally needs allowed friend-list-to-airing joins; the current personal query does not supply it. 6. **A cross-title friend feed needs a deliberate contract.** Existing title RPC has shared nonarchived-club visibility, takes highest allowed effective rank across shared clubs, and honors a subject member's privacy downgrade (20260731040000_clubs_trust_pack_v1.sql:107–135). Rank 1 exposes status but nulls progress/rating; rank 0 invisible (:159–167). Do not infer cross-title access merely from client caches or bypass these checks. 7. **Duo club activity is intentionally limited.** ACTIVE empty state says activity unlocks at three members because member_statuses are withheld below that threshold (ClubDetailTabComponents.swift:707–719). Title-level friendship visibility and club bundle thresholds are different contracts. A proposal must explicitly choose the appropriate contract rather than claim “Clubs already gives a universal friends feed.” 8. **Do not silently amplify reminders.** The local reminder path already requests permission during anime list saving. Adding a prominent home notification CTA should first consolidate user intent/per-title controls and distinguish Japanese airing from relevant availability, rather than schedule more of the same notices. 9. **Stale documentation affects planning.** Source labels ACTIVE while current-state docs still reference This Week (CURRENT_APP_STATE.md:630–637). Source also reveals the unused enriched-list flag. Parent should update authoritative docs if changing product plan, without asserting deployment.

Proposed product direction (recommendation, not implementation)

- Returning user with current titles: compact Continue first, next personal release second, one concise editorial recommendation third. A meaningful next action should be visible before a full-height discovery cover. Empty accounts retain editorial-first discovery and a clear import/taste action.
- Show one understated social clue on relevant Collection/Continue items: “2 friends watching” or “Mira finished”, only when backed by the right permitted status. Tap into the existing detail friends section. This costs less attention than a new social-first home.
- An optional “From your clubs” preview can reuse enriched club preview/unread data and deep link to ACTIVE. Label it as club activity, not broad friend history; preserve duo and sharing behavior.
- A richer Friends seen/upcoming module is a second step requiring a bounded, server-filtered contract, explicit status semantics, deduplication, title-type support and privacy tests. It should not be built by downloading friends' whole lists.
- Keep Taste/Leanings as an optional refining tool; source already supports reversible signals/freshness. Prioritize demonstrating one resulting recommendation over requiring prolonged swiping before utility.

Suggested review criteria: on a small mobile viewport, a returning user can find a current title and increment progress without scrolling past the editorial hero; next-release cards correspond to their own list; every social sentence matches an allowed status; empty/error/offline/flag-off/duo states remain honest. These are proposed criteria, not measured results.

Remaining verification for parent

Live flag values and per-user evaluation; actual home screenshots at target viewport and large text; production RPC definitions vs local migrations; personal-airing freshness/timezone semantics; recommendation empty/populated behavior; private/status/progress visibility across overlapping clubs; whether OS reminders are scheduled as source suggests; visibility of Collection custom-card social clues. No such runtime evidence is claimed by this audit.

16. Additional 2025–2026 community source register

Eleven records, with dates and access limitations. Selected qualitative evidence; no population-frequency or retention-effect inference.

RT01 — How do you keep track of the anime you’re watching and stay updated about new episodes or news?

How do you keep track of the anime you’re watching and stay updated about new episodes or news?

Date: 2025-11-04; replies 2025-11-04–05 · **Platform:** Cross-platform: MAL, AniList, LiveChart

OP wants a more personal watching journey. Replies counter that MAL, AniList and LiveChart already cover tracking and release alerts; one values only the airing day because Japan-time countdowns do not reflect accessible streams.

Limits: Builder-research OP; substantial satisfaction counterevidence. No proof existing products universally lack reminders. Exact dates from indexed thread; direct post/comments read.

Access: Direct web post and replies, plus dated indexed thread

RT02 — Useful Anime Tracker Apps

Useful Anime Tracker Apps

Date: 2025-02-22 · **Platform:** General anime tracker workflow

User wants a calendar limited to their shows and an easier way to follow future seasons without repeatedly searching and adding separate entries. Replies suggest AniList/AniChart and general TV trackers.

Limits: Tested apps unnamed; proposed alternatives not confirmed as solving the exact request. Supports franchise-following need, not claim all trackers lack personalized calendars.

Access: Direct web post and replies; exact date corroborated by indexed thread/footer

RT03 — Is there a way to create custom lists (or an alternative site with this feature)?

Is there a way to create custom lists (or an alternative site with this feature)?

Date: 2026-04-07 · **Platform:** AniList website

User wants ordered favorites like Backloggd. Replies point to Settings→Lists, then distinguish available custom groups from unavailable arbitrary ordering; favorite-heart reordering is offered as a workaround.

Limits: A request about ordering is not evidence custom lists are absent. Implementation intention has no ETA; current completion unverified.

Access: Direct web post and replies; exact indexed date

RT04 — What’s missing from anilist?

What’s missing from anilist?

Date: 2026-03-30; contemporaneous replies · **Platform:** AniList website/mobile web

Requests include own-list tag filtering while retaining scores, status visibility inside custom lists, curation without declaring reading intent, privacy controls, and comparing friends' lists. A reply explains title-level friend activity and one-person comparison already exist.

Limits: Prompted wishlist with mixed demands and praise of existing site; not representative prevalence. Existing friend capabilities must be acknowledged. Some later comments excluded.

Access: Direct web post and multiple reply chains; exact indexed date

RT05 — Made a monthly calendar view for your AniList watching list

Made a monthly calendar view for your AniList watching list

Date: 2026-07-07; replies through 2026-07-10 · **Platform:** Third-party AniList/MAL calendar

Third-party calendar uses a public username. User reports duplicated episode entries; maker fixes duplicate custom-list records and user confirms, saying it can replace their Discord note of airing weekdays.

Limits: Promotional post and one self-reported successful workflow. No independent product benchmark or usage/retention measurement. Cross-list deduplication fixed in thread.

Access: Direct web post and developer/user replies; dated indexed thread

RT06 — Neko update: the AniList forum inside, push notifications for episodes, custom lists and much more

Neko update: the AniList forum inside, push notifications for episodes, custom lists and much more

Date: 2026-09-02 · **Platform:** Neko Android third-party AniList client

User praises Next up, Because you've watched, and list-relevant forum threads. Others request reachable search and customizable list layouts. Multiple replies criticize navigation slowness; maker acknowledges slowness but cannot reproduce one freeze report.

Limits: Developer promotion with genuine-looking user replies; no independent performance test. Do not adopt claimed installs or most-requested ranking as verified. Date from indexed result; direct relative timestamps vary.

Access: Direct web post and replies; exact indexed date

RT07 — On-site Notification Center for New Releases

On-site Notification Center for New Releases

Date: 2025-08-28 · **Platform:** SIMKL official user-feedback forum

SIMKL user requests a persistent, searchable release inbox with read/unread state, because email can be buried and push dismissed. Page shows 47 votes when accessed.

Limits: Suggestion, not shipped feature or evidence of retention lift. No visible resolution/comments; voting system allows multiple votes per person, so 47 votes is not 47 users.

Access: Direct official feedback page; visible publication date

RT08 — Aluminium for AniList — custom-list and friend-tab review exchanges

Aluminium for AniList — custom-list and friend-tab review exchanges

Date: 2025-03-23; 2025-09-26–28 · **Platform:** Aluminium Android third-party AniList client

March reviewer praises custom lists but initially thinks creation requires a website detour, then edits to say it exists in Experimental; developer agrees it is buried. September exchange praises uncluttered UI; developer says a Friends tab has now shipped.

Limits: Third-party client, not AniList website. Historical discoverability issue, not current missing functionality. Store locale variants return different review subsets; NZ indexed/retrieved page used.

Access: Google Play listing plus indexed review/developer-response text

RT09 — Resolved — Removing anime in your watchlist from all anime browsing

Resolved — Removing anime in your watchlist from all anime browsing

Date: 2025-01-13; resolved by requester 2025-01-24 · **Platform:** Anime-Planet forum / website

New user asks to exclude already-categorized titles from discovery. Reply supplies existing unmarked-only filter; requester confirms it solves the problem and moderator supplies location guidance.

Limits: Resolved discoverability/onboarding friction, not absent filtering. Full four-post text available through search index; direct forum fetch failed. Moderator screenshot location not visually inspected.

Access: Indexed full four-post forum exchange; direct fetch failed

RT10 — Upcoming Features & suggestions

Upcoming Features & suggestions

Date: 2026-09-08; replies through 2026-09-11 · **Platform:** Darshaka third-party AniList/MAL client

Darshaka maker says release notices currently check Watching; optional automation moves Plan to watch into Watching on release to receive notices. Another user anticipates manga chapter notices to replace repeated manual checking.

Limits: Promotional roadmap, not verified delivered manga notifications. User accepts offered workaround; treating automatic status change as undesirable is a design judgment, not stated complaint.

Access: Direct web post and replies; exact indexed dates

RT11 — A suggested tool for anilist

A suggested tool for anilist

Date: 2026-02-22; replies 2026-02-22–23 · **Platform:** AniList website

User wants shareable, curated thematic lists such as cyberpunk or existentialism. Response promises eventual work with no ETA and points to existing custom lists; another user prefers collaborative tags and dislikes the proposal.

Limits: Demand is contested, so curated public lists are not universal necessity. Custom groups already exist; shareability/searchability is the distinction. Roadmap completion unverified.

Access: Direct web post and replies; exact date in indexed thread and Reddit footer

17. Release and flag evidence

GitHub upload versus release

Fresh GitHub read: the approved welcome commit b943e457 is on codex/kuro-onboarding-no-tv, committed 25 September at 13:12:57 UTC. main is still 10f3c9a from 30 July; the reference branch is 33 commits ahead and not merged into it. The release workflow uses manual dispatch, with TestFlight disabled by default; no Actions runs or GitHub releases were returned. These facts do not rule out a separate manual Apple upload. The connected phone was rechecked and still reports 1.0 (19), developer-built. No build-to-commit mapping has been obtained.

Installed versus released

Checked: 2026-09-25T23:16:20+02:00 (Europe/Berlin).

Result

Evidence boundary	Observed state	What this proves
Current repository	Marketing version1.0; CURRENT_PROJECT_VERSION17 in Kuro.xcodeproj/project.pbxproj (app settings lines434/446 and471/483)	Current source build-number settings only; not installed binary identity or shipped version
Connected physical iPhone	com.Kuro.app version1.0, bundleVersion19; builtByDeveloper=true	App currently installed on inspected device is build19; developer-built flag is reported by device
Latest TestFlight build	Unverified	Configured Fastlane key file missing; authenticated API query could not be performed
App Store review/public release	Unverified	No App Store Connect state or public-store release proof obtained

The device result contradicts assuming the phone is running current repository build17. It does not establish which commit produced build19. Developer-built classification must not be converted into a claim that build19 is the latest TestFlight build, publicly released, or equivalent to today's source. Source could be dirty; this task did not trace a build-to-commit manifest.

Safe evidence collected

- Read-only Xcode devicectl app inventory filtered exactly to com.Kuro.app; command succeeded.
- Sanitized installed fields: name=Kuro; version1.0; bundleVersion19; builtByDeveloper=true; appClip=false; defaultApp=false.
- Current repository project version settings read directly.
- Existing fastlane/Fastfile read for its configured App Store Connect API-key identity/path. The referenced file under ~/.appstoreconnect/private_keys does not exist.
- A different .p8 filename exists in that standard directory, but this task did not establish that it belongs to Kuro or shares its issuer. It was not opened or used.
- Tool inventory had no purpose-built App Store Connect/TestFlight connector.

API attempt boundary

A local read-only script was prepared to perform only GETs for app identity, builds, App Store versions and beta details. It stopped while trying to read the missing configured key, before token creation or any App Store Connect HTTP request. No credential value or token was printed or saved. No key/config was changed.

Actions not performed

No install, uninstall, launch, sign-in/sign-out, data mutation, upload, TestFlight submission, release, deployment, key creation or account change. Raw device JSON exists at a temporary local device-inventory file; it includes a device identifier/app container path, so this memo is the sanitized reporting artifact.

Remaining proof needed

An existing verified Kuro App Store Connect credential or authenticated UI access is required to resolve latest uploaded build, beta availability and public release state. A build-to-commit artifact is separately required to connect phone build19 to source. User-visible screens can be inspected independently on the authorized phone by the root task.

Live flag inventory

Flag	Enabled	Percentage	Markets
affiliate_links_v1	False	0%	*
clarify_v2	True	5%	DE, AT, CH
clubs_chat_v1	False	100%	*
clubs_interaction_v2	False	0%	No market restriction
clubs_list_enriched_v1	True	100%	*
clubs_notifications_v1	True	100%	*
clubs_pace_sync_v1	True	100%	*
clubs_reactions_v1	True	100%	*
clubs_realtime_v1	True	100%	*
concierge_editorial_v1	False	0%	No market restriction
concierge_perf_v2	False	0%	No market restriction
credits_cast_v1	True	100%	No market restriction
discover_realm_rails_v1	True	0%	No market restriction
fm_assist_v1	True	5%	DE, AT, CH
personalized_new_to_you_v1	True	0%	No market restriction
rag_assist_v1	True	5%	DE, AT, CH
social_activity_v1	True	100%	*
streaming_availability_v1	True	0%	No market restriction
swipe_tap_guard_v1	True	100%	No market restriction
taste_deck_v1	True	100%	No market restriction

Active Edge Function versions

bulk-import-anime v79, bulk-import-manga v78, mirror-images v17, concierge-parse v45, concierge-apply v25, concierge-undo v16, concierge-recommend v49, concierge-resolve v16, concierge-retrieve-assist v7, concierge-retrieve-feedback v5, delete-account v8, auth-callback v4, concierge-import-anilist v5, manga-chapter-enrich v66, manga-source-review-action v5, realm-describe v9.

Migration version comparison: {"local_count": 235, "remote_count": 235, "local_only": [], "remote_only": [], "note": "Version-list parity only, not SQL checksums or Edge Function source-byte parity."}

No Edge Function source-byte comparison or production write-path test was performed.

18. Approved-reference reconciliation

Cross-version source comparison

Read-only comparison performed 2026-09-25. Approved reference: GitHub commit b943e457fd2b611bafa6a52dcbbada417d5b727a on the user-named codex/kuro-onboarding-no-tv reference. GitHub API resolves the exact commit and reports “Auth: remove Zen TV from welcome screen (owner request)”, committed 2026-09-25 13:12:57 UTC. Current local HEAD remains 697a5bc, with a modified AuthView and unrelated documentation changes. No checkout/reset/edit/deploy occurred.

Scope of approval: the user approved this commit as the welcome reference. That does not establish approval of every home, collection, Taste, social or backend implementation in that commit. The earlier source audit described the local working tree. The correct distinction is “approved welcome reference” versus “current-source product audit,” not “approved whole app” versus “stale audit.”

Result

The prior home/list hierarchy findings are not stale relative to this reference. Collection and flags are byte-identical; the Discover differences are width capping/centering rather than section order or recommendation logic; ContentView differences concern gesture arbitration rather than default page or navigation hierarchy. Taste reliability/accessibility claims do need version-specific wording. The welcome itself differs materially from local AuthView and should be compared against the approved reference directly.

Meaningful differences

ContentView — landing and navigation findings still apply

The entire diff consists of:

- Local uses KuroGesturePolicy.edgeMarginPt instead of literal 24 (both versions around line158).
- Local corrects a launch-command comment to capital-K bundle ID (around line257).
- Local computes fast fling before rail-drag/exclusion guards, allowing intentional fast paging even through full-page exclusion zones (reference line315; local line315).

Default Discover selection, onboarding choice, Taste/Concierge substitution, page order and club badge behavior otherwise match in this file. Local line numbers after the gesture change are five lines later than reference; do not transfer every old local line citation to the GitHub source without adjustment.

EditorialDiscoverView — only larger-screen sizing differs

Reference uses currentWidth - 40 for One Thing, skeleton, featured fallback and Hidden Gem. Local introduces heroWidth = min(currentWidth - 40, 520) at line64 and centers all four card uses. Reference hero branch begins around line133; local around139. No section ordering, data request, personalized fallback or social-prefetch changes appear in the diff.

On a normal illustrative 393-point-wide viewport, both calculate hero width353, hence approximately441-point height with the identical OneThing card's4:5 rule. The local maximum-width protection matters above560-point view width; it does not invalidate the mobile-first audit. Do not describe the520-point cap as part of the approved reference.

Still true for both versions: editorial One Thing → catalog Airing Today → Because You/NTY → gated Shelf → Essentials → gated Hidden Gem → more discovery; no personal Continue or friend-first home rail. Both use existing personal upcoming data elsewhere in Collection.

Collection, flags, social and club components — exact equality

Both collection files, FeatureFlags, OneThing card, OnboardingView, Discover/UserLists/Social service extensions, ClubsView, ClubDetailView and ClubDetailTabComponents are byte-identical. Their same line references remain valid for the checked files.

Therefore the audit's Collection +1 progress actions, grid-only countdown, custom Collection cards without friend-count rendering, flag evaluation/accessors, club ACTIVE/latest-member-state behavior and three-member empty-state disclosure are unchanged. The service extensions for personalized Discover, personal upcoming and friend-count requests match as well. This comparison did not redownload all backend migrations, SupabaseService base, shared catalog cards or every detail view; do not extend hash equality to unexamined files or claim deployed RPC equivalence.

TasteDeckView — local improvements are not approved-reference facts

The core taste interaction, Leanings/profile freshness and undo remain in both. Local adds:

- In-memory failed-signal queue/retries plus a failure banner; reference records fire-and-forget without this local queue.
- Explicit initial-fetch failure state and an eight-second loading watchdog with Retry.
- VoiceOver undo window eight seconds instead of a universal four seconds; accessible “Read synopsis” action.
- Dynamic Type label wrapping/scaling and brighter I KNOW THIS label.
- Screen-edge gesture protection for leaving the deck without recording a judgment; coordinated with ContentView gesture changes.
- Downsampled image prefetch.

These are source differences, not measured behavior or proof one version was shipped. Any final document describing resilience should clearly say these are in the local source, not automatically in the approved commit. Earlier audit's basic optional taste/leanings/undo recommendation is unaffected.

AuthView — approved welcome must be anchored to reference

This file is not equal. Approved commit line151 sets scenery height to52% of available height plus top safe area. Local line151 uses72%. Reference welcomeChrome places a Spacer first, then groups Kuro brand and headline in a VStack with14-point spacing. Local instead places Kuro high inside the painting at top safe area +13% viewport, separates the headline below a Spacer, and adds36-point bottom spacing before buttons.

These changes materially affect welcome composition. The approved reference is not the local72% scenery/top-brand arrangement. Both versions retain the same headline wording and comparison shows no other AuthView delta outside these layout changes. Do not generalize from the commit title into rendered proof; source comparison confirms the layout differences, not screen appearance on a particular device.

Evidence and limits

Downloaded15 files through authenticated read-only GitHub contents API at the immutable commit, using raw-content responses into /tmp/kuro-approved-reference/. Computed SHA-256 for each downloaded file and local working file, and reviewed all four unequal-file diffs. Eleven files equal; four differ. No repository fetch/checkout, build, simulator run, hosted query or write was needed. GitHub commit metadata checked separately. Downloads, comparison JSON and four .diff files are retained in the temporary folder.

Reference link:
https://github.com/Humbl-club/Kuro-Anime-App/commit/b943e457fd2b611bafa6a52dcbbada417d5b727a

Exact equality register

File	Equal	Reference SHA-256	Local SHA-256
Kuro/ContentView.swift	No	d06532a58b6bdcb421cff7a05475a1be95822adefd43a2a4de34a14b0f5985d0	223cdf5b13def1f578f2dc94c85a3841e955710fb930283f39fd70024aaf0e48
Kuro/Views/EditorialDiscoverView.swift	No	0631452466a05d5a7203ddf6eb8300c1adb3bba2d0694437a3658f77ef6c83a6	b1c21ad15a7467fdeca03a1fd0236a3cf1e2b5535fe4f8623343e40c151b7a4
Kuro/Views/EditorialCollectionView.swift	Yes	dfe7553e6f0e6edccf4d9f6500d204bff207d005eacda6c94393e65e785c8ec2	dfe7553e6f0e6edccf4d9f6500d204bff207d005eacda6c94393e65e785c8ec2
Kuro/Views/EditorialCollectionComponents.swift	Yes	065a0d340fa42677da1a62cd112080f19cba3448f51a4f6cc7ed540c07e0c77d	065a0d340fa42677da1a62cd112080f19cba3448f51a4f6cc7ed540c07e0c77d
Kuro/Services/FeatureFlags.swift	Yes	e024b0ce5a04cd52248fb6c60c2bb66a7a9ed5d48bac3cc6328bb76a9cf55ee0	e024b0ce5a04cd52248fb6c60c2bb66a7a9ed5d48bac3cc6328bb76a9cf55ee0
Kuro/Views/DiscoverOneThingCard.swift	Yes	edc9069f0ab3724b0f664bc619e41c2dbec37a199e92140952b6ba2f52ddb42e	edc9069f0ab3724b0f664bc619e41c2dbec37a199e92140952b6ba2f52ddb42e
Kuro/Views/OnboardingView.swift	Yes	24d74b1c9a18a6d85f9fa27bd646270698b2f55aa59746aa8653f5e3bf5138cc	24d74b1c9a18a6d85f9fa27bd646270698b2f55aa59746aa8653f5e3bf5138cc
Kuro/Views/TasteDeckView.swift	No	35a54ce6cb73e32d46305e30825f3ef7e278215018af5e063ef4edbf464039d1	ab43bf0f80031ea3f3fbdb12b23fb687e13133aabf6ee1b612c822600651bc81
Kuro/Views/AuthView.swift	No	acc6fcd8d881cc9c727abdf84f8a34494395858c03fb623bc62ef6258d020b03	2ab7191662ebd88b256bafecd5d197769161140496aa9ecb92a2557457cb9fba
Kuro/Services/SupabaseService+Discover.swift	Yes	8399858415b2664d554cdb085944b77817d6918ba5b3f9f91b3fabff206a64f2	8399858415b2664d554cdb085944b77817d6918ba5b3f9f91b3fabff206a64f2
Kuro/Services/SupabaseService+UserLists.swift	Yes	a85aec5e339fdcad0f878742f3ac069405aef9e2aca374ada4b4527e540ed7d4	a85aec5e339fdcad0f878742f3ac069405aef9e2aca374ada4b4527e540ed7d4
Kuro/Services/SupabaseService+Social.swift	Yes	be13ed7c68f3717645b36a3ef38ffcc3800cebd660a3b80114441cc52f03002c	be13ed7c68f3717645b36a3ef38ffcc3800cebd660a3b80114441cc52f03002c
Kuro/Views/ClubsView.swift	Yes	fc617c4b03791cc69a65f42e44fbad9dc3775f05396e678e8b93512ef68d593a	fc617c4b03791cc69a65f42e44fbad9dc3775f05396e678e8b93512ef68d593a
Kuro/Views/ClubDetailView.swift	Yes	7dc9453792d3673272d8a9dc180e133649fc848da1df5abe0a9afeacd633361d	7dc9453792d3673272d8a9dc180e133649fc848da1df5abe0a9afeacd633361d

File	Equal	Reference SHA-256	Local SHA-256
Kuro/Views/ClubDetailTabComponents.swift	Yes	784465ae2e092813e323d1d03147fbc3b50f4c7080f8570e9cb900aef8f06f05	784465ae2e092813e323d1d03147fbc3b50f4c7080f8570e9cb900aef8f06f05

Independent authentication recheck

Chronological limit: This source reconciliation predates the later rejection of all five visual concepts. Its code findings remain applicable; any shallow-crop email styling below is superseded by section 20.

Read-only source comparison, 25 September 2026. Approved reference: Humbl-club/Kuro-Anime-App, branch supplied by owner codex/kuro-onboarding-no-tv, exact commit b943e457fd2b611bafa6a52dcbbada417d5b727a. GitHub commit API confirms message **“Auth: remove Zen TV from welcome screen (owner request)”** and its only changed file is Kuro/Views/AuthView.swift. Content was downloaded directly with authenticated gh api at that immutable commit; no checkout, app install, device changes or deployment. This establishes source identity, not rendered or released behavior.

Approved AuthView copy: /tmp/kuro-approved-AuthView.swift. Also downloaded approved OnboardingView, ContentView and TasteDeckView into /tmp/kuro-approved-*.swift to check Q04. All line references marked **approved** below refer to these remote commit files. Report line numbers are the inspected current report and may shift as root edits it.

1. Material baseline correction

The approved welcome and dirty local welcome are **not the same composition**. AuthView comparison yields exactly two diff hunks, both within welcome rendering:

Element	Approved reference	Dirty local working tree
Scenery height	52% of geometry height + top safe area, approved AuthView:150–151	72%, local AuthView:150–151
Brand/headline relationship	One VStack with 14pt spacing, between upper/lower Spacer; brand then headline, approved:171–189	Brand high at safe area + 13% height; headline separately positioned above CTA, local:171–190
Controls/wording	Get started then Apple, 54pt capsules, existing legal copy, approved:191–222	Same controls and text
Email/auth logic	Existing form and auth state logic	Identical; after the welcome diff, local line numbers are approved line numbers +1

The local user edits remain 17 insertions /16 deletions against local HEAD. They were not overwritten. Owner approval now identifies the **52% grouped composition** as the design reference. Keep the existing local edits preserved as separate work; do not silently treat them as the approved design or reset the workspace to achieve reference parity.

Preserve approved welcome artwork, fade, composition, wordmark/headline placement, wording, Get started/Apple order and capsule treatment. Do not restore Zen TV, switch to the debug Genie scene, add extra welcome choices, relocate its typography, enlarge its scenery to the dirty 72% version, or substitute a different illustration.

2. Q01–Q04: what survives and what must be corrected

Q01 — One visual language from scenery to form

Still valid: the approved default is the painted mountain loop fading into white (approved AuthView:138–168; asset KuroZenMountains at1309–1318). Its type is serif, ink is warm near-black (50), and buttons are rounded dark/outlined capsules (191–212). Email currently has a white ground, small serif wordmark, serif heading and dark capsule submit, but no mountain artwork (280–432). A compact scenery echo on the email form is a proposal, not an existing feature.

Corrections: report Q01:99–103 should explicitly name the exact approved commit and state that only the email screen is being developed to match it. “On short phones the artwork gives up height” should refer to proposed **email** art; do not use it to redesign the approved default welcome. The appendix's 72%/13%-placement description (currently A06:281) correctly describes dirty local code but does not describe approved reference. The initial appendix provenance (241) likewise needs to keep those identities separate.

Accessibility finding still holds: approved default scenery uses unconditional30Hz TimelineView (1297–1338) with a34-second pan (1304–1308), regardless of Reduce Motion. Fixed welcome fonts are at177 and182; email fixed fonts include302,312,320,377,401,442,484,495. A static Reduce Motion presentation and large-text resilience are functional/accessibility variants to validate, not authorization for a new normal-mode welcome design. No clipping or motion failure was reproduced.

Q02 — Make authentication intention explicit

Still valid: approved mode defaults to signIn (27); Get started calls goToEmail without changing mode (192–196;113–120); form then says Welcome back (311), with Create account as secondary mode selection (325–335). Welcome legal language is plain Text (214–219). There is no Terms/Privacy Link in the downloaded AuthView.

Required scope correction: retain the approved welcome CTA labels/order and visual composition. Clarify Create account versus Sign in **inside the email destination**. Do not interpret Q02's “clear Apple and email choices” or appendix A03's suggested alternative CTA as permission to redesign/relabel the approved welcome. Making existing legal text independently tappable can preserve its visual footprint; this is a functional behavior change, not new welcome art.

Q03 — Repair auth states

All listed defects remain source-supported on approved reference:

- emailRevealed=false at47; initial onAppear reveals only if already email (98–101); normal goToEmail explicitly reveals (113–120). Apple catch switches to email without reveal (635–639). The email show condition is281 and opacity/offset modifier is685–697. Hidden form after a fresh Apple cancellation/failure remains a **predicted runtime risk**, not a reproduced failure.
- Apple nonce/credential/token guards set errorText and return (456–458;618–629), while welcome has no visible error outlet (140–224).
- Successful reset assigns confirmation to errorText (643–655), shown in red accent by the email error region (375–379).
- Input focus is ordinary State (35) plus tap bookkeeping (481;497), not actual FocusState/submit chaining. Eye button lacks explicit label and44pt frame (362–368). Mode buttons lack selected accessibility trait (435–451).

These can be repaired without moving or redesigning the approved welcome. Preserve the welcome on Apple cancellation; show meaningful failures in a transient, accessible state that does not rearrange the normal composition. Any deliberate route into email must establish revealed controls. Do not indiscriminately reuse goToEmail if it clears an error that must be displayed (114).

Q04 — Keep introduction optional and useful

Still valid at the approved commit: downloaded OnboardingView is byte-identical to local. Five cards remain; final Clubs card is193–198; primary CTA selects Taste when enabled (84–87); Skip exists (100–104); completion writes global UserDefaults (136–139;247–250). Approved ContentView:466–473 consumes that destination. Approved TasteDeckView describes endless batches at66–68 and labels actions NOT FOR ME408 / I KNOW THIS426 / CALLS TO ME440. This is not a12-choice setup exercise.

Scope correction: distinguish the **pre-auth approved mountain welcome** from the separate **post-auth five-card feature tour**. User approval to preserve the former must not be read as a request to replace either. Q04's new “Bring my list / Find my first title / Explore” gate should remain an optional later experiment; do not insert another mandatory screen in the email-continuity work. Improve first-use routes only under a clearly separate approved scope.

Small factual clarification: Skip preserves underlying selection; it does not unconditionally force Discover. Approved ContentView:466–473 sets selection only for Taste. Pending deep links are consumed at419–425, so a deep-linked launch may have another underlying selection. The current report's default-Discover wording needs “on an ordinary launch without a pending destination.” Runtime modal ordering still needs testing.

3. Email continuation specification — welcome stays intact

This is a proposed design/behavior contract, not implementation.

1. **Reference lock.** Normal welcome remains the approved52% mountain composition, grouped Kuro/headline, existing fade, Get started pill then Apple pill, existing legal line. Preserve animation character; no TV and no alternative hero. Keep the source difference from dirty local visible during any later implementation rather than overwriting it silently. 2. **Same place, smaller art.** Email receives a shallow crop of the exact KuroZenMountains asset, fading into the same white ground. Reuse its existing warm ink and serif identity. The crop is decorative, excluded from accessibility, and never becomes a second full-height hero. Let it collapse when keyboard/large text needs space; the welcome geometry is unaffected. Default email art need not run a second indefinite animation. Reduce Motion uses a still crop and immediately visible controls. 3. **Hierarchy.** Back and small Kuro wordmark → compact optional scenery echo → clear purpose heading → explicit Create account / Sign in control → labelled Email and Password → inline state → matching main submit → recovery/alternate mode → policy links where required. Use the existing scroll view (approved309–430); do not rebuild as a fixed-height screen. Keep existing28pt horizontal form rhythm (428) where available and54pt submit capsule (405). 4. **Intent.** The existing welcome Get started can lead to Create account as the proposed first-use email default, with obvious Sign in immediately beside it. This changes destination state rather than approved welcome appearance. Headings and CTA must match the selected action (Create account / Sign in), and the user's entered email survives mode changes. Keep Apple usable in the approved welcome; no new provider or additional welcome action is required. 5. **Visual cleanup confined to form.** Replace the current blue focused field styling (485;506–508;514) with a stronger monochrome border/label consistent with welcome. Replace red mode underline (445), successful validation checks (569;580) and recovery link (417) with the shared neutral hierarchy. Reserve a distinct error presentation for actual failures; reset success is a neutral confirmation. Scalable body/heading tokens should retain the reference's serif/sans relationship rather than introducing a new font or card aesthetic. 6. **Interaction completeness.** Real keyboard focus, email Next→password, password submit, announced selected mode and “Show password” /“Hide password” semantics. Password eye44pt hit target; retain44pt Back target (293). Loading blocks duplicate submission while remaining understandable. Accessible announcements do not expose password contents. 7. **Transition reliability.** Every path into email reveals it deterministically. Apple cancel keeps approved welcome useful. Guard/provider errors have a visible accessible outlet; retain actionable error when transitioning. Back returns to the same approved welcome, not the debug Genie scene. Do not delay operability behind decorative entrance timing. 8. **No extra setup gate.** A successful login follows existing app flow and preserves pending title/club intent. Import and taste improvements are separate post-auth work. No claim that form continuity solves onboarding-deep-link ordering.

4. Acceptance evidence required before implementation is called done

Compare the normal welcome to an image/build made from the exact approved commit, and separately check the new form on the smallest supported phone, large text, German/English, keyboard shown, VoiceOver and Reduce Motion. Reproduce fresh Apple cancel/failure before ever opening email, retries, invalid/offline email submission, reset success/failure, Back, mode switching and pending deep links. Confirm commit/build identity. Source inspection alone does not show the installed build renders either composition.

No repository files changed by this review. Output and downloaded references are in /tmp only.

Community evidence recheck

Source	Date/access reverified	Claim and issue-state assessment
RT02 — Useful Anime Tracker Apps	22 Feb 2025 corroborated by date-specific Reddit footer. Direct OP and replies read.	OP requests own-show schedules and easier future-season following. Replies suggest alternatives; no accepted solution is shown. Tested apps remain unnamed. Keep this as a workflow request, not an unresolved defect attributed to MAL or AniList.
RT04 — What's missing from AniList?	Indexed timestamps confirm 30 Mar 2026 opening and 1 Apr friend branch. Direct thread plus indexed replies.	Friend-overlap request is partly answered: title Social pages and one-person list comparison exist. Custom-list status/score visibility and profile privacy requests remain user reports, not independently reproduced current limitations. The memo appropriately credits existing friend features.
RT06 — Neko update	Index confirms 2 Sep 2026 post and detailed personal-home reply; reachability comments 3 Sep; performance acknowledgment 6 Sep. Direct replies also read.	One commenter values list-centered home and contextual forums. Navigation slowness is acknowledged; a stronger freeze report was not reproduced by the maker. Another commenter says performance is already acceptable to them. No verified fix or current-device test. Promotional OP, not independent adoption evidence.
RT07 — SIMKL release inbox	Direct official feedback page visibly dates suggestion 28 Aug 2025.	Persistent history/read state is requested because email/push can be missed. No visible completion status or response. Displayed 47 votes cannot be converted to users because voting permits multiple votes. Keep as a requested behavior; absence today and retention benefit remain unproven.
RT08 — Aluminium reviews	Current locale opening sometimes omits reviews. Search-indexed canonical listing exposes 23 Mar and 26–28 Sep 2025 exchanges.	Reviewer corrects initial missing-editor impression: editor exists in Experimental. Developer agrees discoverability needs work. September developer response reports Friends tab released. Existing memo correctly treats both as historical/resolved-or-present functionality, not current omissions.
RT09 — Anime-Planet unmarked filter	Direct fetch fails again; full four-post indexed exchange shows 13 Jan 2025 question/answer and 24 Jan requester confirmation.	Existing unmarked-only filter solves the request. Correctly labeled resolved discoverability friction. Do not imply a missing filter or inspected moderator screenshot; current wording preserves both limits.

Boundaries to preserve

- A thread's relative age is unreliable in these rendered results. Exact dates above come from indexed timestamps, visible publication dates or date-specific footer links.

- Developer promises are not shipped fixes; a developer-reported release is not an independently tested binary.
- An unanswered forum request does not prove the requested feature is absent today.
- Keep the explicit disclaimer that this selected corpus supplies neither representative demand rankings nor measured retention effects. “Continue first,” reachable search and optional release inbox remain proposals to test.
- This recheck adds no new source count. It verifies six existing records across Reddit, SIMKL, Google Play and Anime-Planet; it does not repeat the 110-record market report.

19. Complete market and monetization research

The original revised market study follows in full so this is one self-contained decision document. Its market evidence, 18 product decisions and 10 monetization decisions remain proposals. Its earlier source-only Kuro mapping is historical and is superseded by the current-source/live evidence in sections 2–4 and 17–18 above. Record counts measure selected sources, not independent users or prevalence; 110 original records plus 11 later records are not asserted to be 121 unique URLs. Public programme terms are dated research, not Kuro acceptance.

19.1. What this research establishes

This is a comparative qualitative study of personal anime/manga tracking and discovery, covering evidence dated **1 January 2025–25 September 2026**, plus current official capability, commercial and policy references. It concerns watched/read history, progress, future titles, recommendations and related social behavior. Readers and streaming integrations appear only where they reveal relevant tracking needs. Their playback and content-access features are not Kuro requirements.

The revised register contains 110 selected source records across 37 distinct hosting domains. That measures breadth of material, not independent people, market share, complaint frequency, or proof of demand. Six sub-agents researched separate lanes: MAL; other platforms/forums; manga; native clients; affiliate programmes; and sponsorship/constraints. The lead researcher reconciled overlaps, reviewed their findings, checked high-impact claims and built the decisions. A further cross-check corrected a Bookshop terms date and verified a Kitsune review year through Apple's dated feed.

Coverage and the remaining access gaps

Research lane	Material actually inspected	Important boundary
MyAnimeList	Multiple r/MyAnimeList and r/animeapp threads with replies; Android and iOS store listings/reviews; corporate and commercial pages	MAL's own forum navigation was blocked by browser safety policy. This report does not claim those forum discussions were read.
AniList	Seven directly inspected forum threads in the first pass, including replies/dates; selected records retained here; current terms and ecosystem references	Existing filters, fixed bugs and planned work are distinguished. No signed-in task benchmark.
MyAniList and other clients	Apple public review feeds, dated release notes, Google Play and GitHub issues with maintainer replies	MyAniList is an independent AniList client. MALClient, AL-chan and MALSync have their own failure modes.
Anime-Planet	Ten recent forum threads read directly, including all visible replies; official support policy	Reports often reveal discoverability or mobile parity problems, not globally absent features.
MangaBaka / manga readers	Nine recent tracker-owned forum posts and visible statuses; Reddit reader/collector threads; Mihon documentation and changelogs	MangaBaka comments did not load; dates are approximate from relative labels. Done is not a verified shipped fix.
Kitsu / LiveChart / AniDB	Kitsu public issues and component status; LiveChart official release/notification documentation; AniDB documentation and a Shoko issue	Current documentation/status is distinct from 2025–2026 customer sentiment; AniDB evidence is narrower.
Other communities	Direct NeoGAF, Something Awful and MangaHelpers discussions; indexed complete ResetEra discussion	Different sites give independent contexts, but do not eliminate self-selection.
Monetization	Merchant/network programmes, publisher collaborations, platform terms and regulatory guidance	Public programme terms do not prove Kuro approval, native-app eligibility, signed partners, conversion or profit.

Direct MangaUpdates access was safety-blocked. Its strengths and weaknesses here are attributed to users and integration documentation, not a firsthand inspection of its forum. Anime News Network forum searches did not yield usable recent tracker-specific material. Blocked or low-detail pages were not counted as successfully read evidence. We did not create accounts, contact anyone, install competitors, test private libraries, or measure live market shares.

Naming matters: MyAnimeList = MAL, a database/community platform with official apps. AniList is a separate platform. MyAniList is a third-party iOS client for AniList. A comment saying “the AniList app” may mean a client or mobile website; attribution is retained wherever known. [P08, C02, M14]

How evidence was coded: One thread and its replies form one source record. Multiple selected reviews in one feed are aggregated. Store variants and duplicated agent finds were reconciled. We recorded date, access method, platform/client, reported job, workaround, staff/developer response, and whether the issue was fixed, disputed, unknown or merely proposed. Promotional launch posts, old featured reviews and undated statements were not counted as recent user complaints. Apple feed dates are review-update timestamps; edits can change them. Official undated pages were checked now but are not assigned invented 2026 publication dates.

Confidence: High means a repeated need across contexts or unusually clear primary confirmation. Medium means coherent but limited qualitative evidence. Low means an isolated request or commercial/product hypothesis. None means a measured prevalence. Priorities combine risk to history, recurrence in ordinary use, dependencies and Kuro relevance; they are analyst judgments, not a Reddit vote tally.

What changed from the first report

The first version leaned too heavily on AniList's accessible forums. This version adds substantial platform-specific MAL, Anime-Planet, MangaBaka, client and independent-forum evidence. It changes the recommendations in four ways: retain information density as a choice; separate library views from tracking status; distinguish manga position from actual consumption; and allow deeper recommendation exploration beyond a short default. Its paid-convenience proposal is withdrawn. The new business constraint is **zero fees to use Kuro**, with useful optional commerce and partner-funded benefits assessed separately.

19.2. The market is several overlapping jobs

The main competition is not a single app category. A person may keep a trusted MAL archive, use a native client for quick updates, consult LiveChart for schedules, and ask a community what to try. Kuro must replace enough effort to earn a place without requiring users to abandon valuable history or communities.

Layer	Examples	Asset users keep	Implication for Kuro
History and community platforms	MAL, AniList, Anime-Planet, Kitsu	Accumulated lists, reviews, relationships, metadata	Switching cost is personal history and habit, not just account creation.
Native and automation clients	MyAniList, ManGo, Kitsune, AL-chan, MALClient, MALSync	Fast updates, widgets, device fit, optional synchronization	Attractive cards and +1 controls are competitive basics. Reliability and recovery matter.
Manga specialist discovery	MangaUpdates, MangaBaka	Niche coverage, alternate titles, creators, publication/licensing context	Anime-style progress alone does not serve manga well.
Release specialists	LiveChart and seasonal tools	Relevant schedule distinctions, reminders, legal destinations	A Japanese airing countdown is weaker than a personal, language-aware release view.
Precision/library tooling	AniDB and Shoko	Exact episodes/files, local-library integration	Serve precise identities without exposing every underlying technical detail to casual users.
Personal substitutes	Notes, spreadsheets, book trackers, general TV trackers	Control, flexible categories, no unwanted feed	Fast defaults must coexist with optional notes and organization.

There is evidence of scale, but no defensible tracker-market revenue or share estimate in this study. A **May 2025 joint corporate release reports MAL had 19.5 million registered users**; that is self-reported historical registrations, not current monthly active users or paying demand. A 2025 Dentsu study covers 8,600 consumers across ten countries and discusses anime-related spending; Germany is not in that sample. Neither source estimates Kuro's reachable audience. Broad anime merchandise/streaming expenditure is not tracker revenue. [T01, T02]

Who to serve first

Segment	Recurring job	What matters most	Proposed role
Busy / returning fan	Where was I, and what fits tonight?	Immediate Continue, trustworthy history, a useful shortlist	Primary entry experience
Seasonal viewer	Which of my shows has something new?	Current-season view without changing status; relevant notifications	Primary retention loop
Manga-first reader	Which chapter, volume or translation am I on?	Independent units, unknown totals, publication distinctions	Co-primary, tested separately
Established archivist	Keep years of records intact	Import reconciliation, export, dates, exact adaptations	Essential migration audience
Explorer / enthusiast	Help me find something specific or obscure	Rich filters, selected seeds, deeper results, creator metadata	Optional depth, not mandatory complexity
Private group	Help us choose and stay spoiler-safe	Polls, reasons, shared pace, understandable privacy	Preserve Kuro clubs
Physical collector	Which exact edition is next, and did I preorder it?	Language, ISBN/edition, release source, duplicate avoidance	Narrow commerce pilot; defer full inventory

These are research-derived segments, not quantified shares. The strongest starting hypothesis is a returning or busy fan with existing history who wants continuity and relevant discovery. Manga readers should not be treated as a smaller copy of anime viewers. Interviews must include people who abandoned tracking, not only power users posting on forums.

19.3. What each major competitor teaches us

MyAnimeList: strong accumulated value, uneven access to it

MAL users value its connected information, reviews, interest stacks, friends and familiar lists. A recent client discussion explicitly asks for that information without requiring a public forum experience. [C14] A July 2025 iOS review specifically praises compact, easy list updating; a NeoGAF exchange prefers the website after app trouble. These are reasons not to assume that replacing the platform with a beautiful empty shell is enough. [M07, M11, F16]

The concrete friction is more useful than “old design.” In an August 2026 UI thread, a user discovers sorting after years while others defend the dense desktop layout. In July, another moved genuinely in-progress older shows to Dropped to isolate current-season watching. A returning fan with 500+ watched titles wants recognition-based batch capture rather than typing every title. Official Android reviews independently request better aliases, personal organization and season grouping and describe a loading failure. [M01, M04, M05, C01]

Decision: Improve collection control, findability and native continuity while retaining compact rows and access to rich title context. Distinguish website, official app and third-party client. Do not say MAL lacks all seasonal filters or cannot track manhwa based on one reviewer. No incidence or current global outage was established. [M08, M09]

AniList: powerful controls and ecosystem, but semantics and recovery still matter

AniList's forum shows that users can miss existing exclusion controls and medium filters. A developer explains why imports cannot reconstruct absent historical dates. A duplicate-entry cause was fixed in August 2026, while existing damage still required careful cleanup. The running bug thread and roadmap provide more specific context than a general “unreliable” label. [S08–S13, S26]

Decision: Make power visible, imports inspectable and repairs easy. Do not interpret missing source history as newly deleted data, or a roadmap item as shipped. Its API terms create a separate commercial/data dependency that must be resolved for Kuro; the report does not determine Kuro's agreement status. [B11]

MyAniList: native convenience can be undermined by monetization interruption

MyAniList is not MAL. Apple's public review feed includes a September 2026 complaint about repeated upgrade prompts and perceived loss of value from an old lifetime purchase. June sorting criticism has an explicit later correction saying sorting was fixed and free again. Other reviews praise organization and season-level precision; some dislike notification behavior or tag overload. [C02]

The changelog documents active work on synchronization, saved filters, notifications and accessibility. Those developer statements are not hands-on proof, but they prevent a misleading claim that every reported issue remains unresolved. [C03]

Decision: Never interrupt opening a collection or updating progress to seek revenue. Remember sort/filter preferences. Let users dismiss a campaign persistently. Group franchises without throwing away season-level detail that some users explicitly like. This evidence supports respecting attention; it does not prove nobody in the market pays for apps.

Anime-Planet: a feature can exist and still fail the user's task

Ten directly read threads show several distinct failures: desktop average-score sorting is not equally discoverable on mobile; a completed-airing filter already exists under Other; mobile width/zoom breaks a user's flow; and a requested manga submission form competes with limited developer/moderation capacity. The response is not automatically “add more features.” [F01, F02, F04, F08]

Its progress discussion exposes a deeper model issue: starting manga after an anime adaptation should not claim that every preceding chapter was read. Partial rereads and rewatches need a session concept. A missing-entry complaint was investigated without confirming deletion; another export-date concern was explained by fields never historically recorded. [F03, F05, F09]

Decision: Separate position from consumption, personal status from publication status, and save confirmation from visual feedback. Offer a simple correction form only with a workable review queue. The privacy discussion supports a clear friends-only middle ground, while its public support policy offers a useful clean-profile advertising precedent. [F07, F23]

MangaBaka and MangaUpdates: depth is an asset; simplification must be selective

MangaBaka's recent forum contains requests both to declutter home and to restore explicit progress totals. One user wants more recommendation results, another wants results based on a selected list rather than the entire history. Creator identity and follow alerts also matter. Import and volume-filter issues marked Done remain instructive but are not asserted to be current bugs. Comments were unavailable and September dates are approximate. [G01–G09]

Readers praise MangaUpdates for detailed tags, aliases, creator links and licensing context, but the sample also shows catalogue disagreements and people using it only for lookup without discovering personal-list sharing. Direct MangaUpdates forum access was blocked, so these are attributed community observations. [G14, G16, G21]

Decision: Keep a fast default and an intentional deeper path. Support chapter and volume input, creator identity and explicit coverage boundaries. A reader's missing webtoon is not proof of a universally small database; do a real coverage benchmark before claiming completeness. MangaBaka's own upstream-data documentation also shows aggregation does not remove provenance, freshness or rights dependencies. [G10, G23, G24]

Kitsu: don't confuse component problems with a dead service

Kitsu's public issues include filtered pagination and a More info control that reportedly reveals nothing. Its official status page separates an operational website/API from unavailable app-store components and records resolved incidents. These are concrete continuity and empty-state lessons; a status component is not a verdict on every installed client. [F11–F15]

Decision: Preserve readable history when one capability fails, distinguish no results from failed loading, and isolate optional sections from core tracking. The sample is narrower than for MAL/AniList/Anime-Planet; no general satisfaction ranking is warranted.

LiveChart: release awareness already has a serious benchmark

LiveChart documents per-title schedules, earliest/sub/dub distinctions, local-time countdowns, opt-in reminders and schedule-change information. It also distinguishes release notifications from partner giveaways. Its FAQ explains the limits of guest marks stored in a browser. These are current documented capabilities, not proof that Kuro's target users are dissatisfied with them. [F19–F21]

Decision: Build a personal catch-up view only if its data is useful for the user's country and language. Keep announcement, airing and actual availability distinct. Explain local-only guest saving. Do not claim an integration exists merely because a website has the needed information.

Native clients, AniDB and automation: maintenance is part of the product

ManGo's recent notes cover cache, login, widget and dual-service repairs. AL-chan reports show rate-limit friction, an auxiliary notification parsing failure with a reported repair, and a useful cached fallback. MALClient users report schedule/detail failures while still valuing the client. A MALSync matching issue received a partial change; a different automatic-progress complaint was withdrawn after extension-conflict testing. [C04, C07–C13]

AniDB's model distinguishes seen episodes and files. A Shoko issue shows a novice surprised by a huge synchronization queue, not an AniDB-wide failure. [F24, F25]

Decision: Optional automation needs an identity preview, queue visibility, conflict policy and undo. Broad integrations increase maintenance obligations. A generic green sync icon is insufficient, and a closed issue is not proof that the user's installed version is fixed.

19.4. Priorities, contradictions and design consequences

Priority	Consequential problem	Supporting contexts	Product response
P0	History or changes become untrustworthy	AniList import/repair; A-P saves/dates; MangaBaka import; client sync	Reconciliation, confirmed saves, export, repair, no fabricated history
P0	A daily update requires unnecessary navigation	MAL seasonal workaround; MyAniList interruption; home clutter	Continue, visible +1 and set-progress, retained views, no monetization in the task
P0	Manga data collapses different meanings	A-P start offsets; volume-only filtering; chapter/volume and translation distinctions	Bookmark vs consumption, chapters vs volumes, original vs available translation
P0	Search/filter power cannot be found or used consistently	MAL sort/aliases; A-P desktop/mobile; AniList exclusion; Kitsu pagination	Explicit include/exclude, clear labels, robust own-list search, device parity
P1	Franchise structure creates administrative work	MAL grouping requests; native precision praise; automation mapping	One visual journey with exact seasons/parts underneath
P1	Discovery lacks the right context or useful depth	MAL information demand; MangaBaka list seeds/more results; ResetEra resolution	Small default, useful reasons, selected seeds, deeper exploration
P1	Release information is irrelevant or ambiguous	Manga collectors; client reminders; LiveChart benchmark	Followed titles/creators, chosen schedule, language/region, opt-in alerts
P1	Revenue interrupts utility or biases choices	MyAniList review/correction; commerce and sponsorship mechanisms	Free core, user-requested commerce, separated and labelled sponsorship
P2	Social is either intrusive or too limited	Opposing MAL requests; A-P privacy; MangaHelpers club behavior	Optional private clubs, spoiler boundaries, simple outward sharing

P0 means first to evaluate and address if Kuro fails the task; it does not mean every proposed feature should be built immediately. Revenue-enabling data rights are a separate prerequisite, not a reason to halt unrelated local usability improvements.

The contradictions change the design. Some users want fewer sections; others want denser information. Some want hidden scores; others prominent scores. Some want a short recommendation shortlist; others more results. Some value individual seasons; others think in franchises. Those are not noise to average away. Use sensible defaults with a few meaningful controls, not an infinitely customizable interface. [M01, M06, C05, G01–G04]

What not to conclude: These sources do not prove that one tracker is best, that all users dislike social features, that AI recommendations outperform editorial/community picks, that automation should always be on, or that a paid feature is never commercially viable. Kuro's free-to-user strategy is the user's constraint and the proposal here; its profitability still needs proof.

19.5. Product decision register

All decisions are recommendations for validation. P0/P1/P2 indicate sequencing; confidence refers to the need, not guaranteed impact. Existing Kuro implementation must be inspected before any change.

D01 — Choose continuity as the central promise

P0 · High confidence in the job; audience positioning remains a hypothesis. Build the first experience around remembering progress, choosing the next worthwhile title and noticing relevant releases. The seasonal-status workaround, history-reconstruction request and notes-app workflows show a need for continuity that spans account creation, daily use and returning after a gap. [M04, M05, M07]

Design decision: Present Continue, Discover and Upcoming as clear task concepts. They need not become three new tabs. Preserve valuable review/title context and an obvious path back to a personal collection.

Tradeoff: This avoids building a public entertainment feed, but may appeal less to people primarily seeking public identity and debate. Keep clubs and outward sharing optional rather than removing social value.

Validation: Recruit returning, seasonal, manga-first and experienced tracker users. Ask them to locate their place, choose a suitable next title and explain an upcoming release. Measure success and return use; do not infer product-market fit from preference for a mockup.

D02 — Separate collection views from actual watch/read status

P0 · High confidence in the failure case; medium in its reach. A user changed Watching entries to Dropped simply to isolate current-season viewing. That distorts the meaning of a record to compensate for a missing or undiscoverable view. [M04]

Design decision: Offer Continue, This season, Caught up and Recently updated as views derived from data. Personal status remains Planning/Watching/Reading/Completed/Paused/Dropped as appropriate. A “Ready to binge” view combines personal planning state and finished publication state; label these separately. Remember the last chosen sort and provide visible reset. [F04]

Tradeoff: More views can become navigation clutter. Start with the smallest set that solves observed tasks, with compact rows alongside editorial cards. Do not add a dashboard editor by default.

Validation: Mix ongoing seasonal titles with older shows and manga. Users must find next actions without falsely completing, dropping or hiding real progress. Compare against current Kuro navigation before replacing it.

D03 — Make progress fast, explicit and recoverable

P0 · High confidence. Keep visible +1 and set-progress actions, with optional reviewed batch updates. Never require a rating, review, ad view or purchase to record consumption. Quick updates already exist in the market; the differentiator is confidence that they affected the right title once. [C04, M07]

Design decision: Confirm the exact episode/chapter/volume and provide immediate undo. Batch completion lists the affected entries. Automation is opt-in, with a visible match and a correction path; preserve specific mappings instead of rerunning uncertain guesses silently. [C12, C13]

Tradeoff: An undo window and a pending state add visual detail, but prevent the costlier burden of reconstructing history. Haptics or animation alone are not confirmation from the server.

Validation: Test rapid taps, app backgrounding, network failure, duplicate retries, mistaken season matches and rollback. Verify both stored state and visible state. A spinner disappearing is not proof of durable saving.

D04 — Keep history usable when a dependency fails

P0 · High confidence. AL-chan's cached list remained useful during one fetch failure; other client reports show optional metadata, notifications or schedule failures interrupting the experience. Rate limits also appear during rapid seasonal additions. [C07–C11]

Design decision: Distinguish saved, pending, failed and stale. Keep previously fetched personal history readable. If offline editing is supported, define conflict behavior and show queued changes; never promise offline writes merely

because a banner/cache exists. Isolate optional social/recommendation errors from core list access.

Tradeoff: Reliable offline mutation is meaningful engineering work. A clear read-only fallback is safer than silently losing queued edits while advertising full offline support. Reconnect/retry must avoid overwriting newer remote state.

Validation: Simulate loss of connectivity and individual provider capabilities, then reopen. A user should know what is saved, what needs action and what can still be done. Confirm retry correctness and account separation, not only pleasant error wording.

D05 — Treat migration as reconciliation, and make departure easy

P0 · High confidence. Multiple forums describe intermediary formats, missing coverage and lost meaning in imported dates. A-P's null-date example and AniList's recent-order explanation show why unknown history must remain unknown. [F09, F10, F17, S08, G08]

Design decision: Before applying an import, show source count, matches, duplicates, unresolved works, unsupported fields and proposed updates. Preserve source identifiers and date precision. Retain unmatched private records or a downloadable unresolved file rather than silently dropping them. Export a human-readable format and a documented machine format for the user's own Kuro history.

Tradeoff: Reconciliation takes longer than a success toast. Offer a short summary with detailed review only where needed; do not force users to inspect every exact match.

Validation: Round-trip representative libraries with absent/partial dates, notes, rewatches, manga volumes, aliases and missing titles. Target zero silent loss. Provider contracts and connector direction are independent prerequisites; a successful import is not continuous two-way sync. Reader-integration documentation and recent repairs corroborate that distinction. [G11, G18, G19]

D06 — Group franchises without merging away precision

P1 · High confidence in the tension; medium in the chosen UI. MAL users request whole-series grouping while MyAniList reviewers value individual seasons. Both needs are legitimate. [M02, C01, C02]

Design decision: Use a franchise overview showing entry point, completed parts and the next relevant part. Expand to exact seasons, split cours, films, specials and alternate adaptations. Keep separate scores and progress; label “series” versus “entries” in statistics. Never infer a universal watch order or copy a rating across adaptations without a choice.

Tradeoff: Relation data can be incomplete or ambiguous. Present sourced relationships and uncertainty. Do not let a simple umbrella card hide an unresolved match that an automated integration will later write to incorrectly. [C12]

Validation: Use complex examples, including split seasons, remakes and manga adaptations. Users should find where to continue and understand exactly which entry receives an update. Preserve imported IDs through grouping and export.

D07 — Separate manga position, units and actual consumption

P0 · High confidence. Starting at chapter 1000 after watching an adaptation does not mean reading 999 earlier chapters. Volume-only progress must not look unread because chapters remain zero. Weekly chapter reading, physical editions and partial rereads are distinct workflows. [F03, G09, G17]

Design decision: Store bookmark position separately from optional reading sessions. Support independent chapter and volume values, unknown totals and partial dates. Let advanced users specify a starting offset or reread range; keep ordinary +1 simple. Ownership and preorder status are separate from reading. Original publication completion, translated availability and personal caught-up state are different facts. [G13]

Tradeoff: Accurate consumption statistics require more data than a single counter. Do not invent precise hours or chapters read from an absolute bookmark. Avoid fixed chapter-to-volume conversion; use edition-specific mapping

only when verified.

Validation: Test ongoing unknown-total works, chapter-first readers, volume-only readers, omnibus editions, adaptation handoff and partial rereads. Filters, completion logic, imports and exports must all preserve the distinction.

D08 — Make releases personal, qualified and controllable

P1 · High confidence in distinctions; medium in priority. Release specialists already support precise schedules. Readers also want upcoming volumes and new creator work; client reviews warn that stale reminders become noise. [F06, F19, F20, G05, G12, C02]

Design decision: Show Today, This week and Later for followed titles/franchises and deliberately followed creators. Label announcement, Japanese broadcast, selected-language release and physical-edition publication separately. Include source/freshness where meaningful, delays and unknown dates. Stop appropriate reminders when a title is removed or dropped; allow the user to override intentionally.

Tradeoff: Reliable localized data and identity resolution cost money and maintenance. An honest partial view is preferable to false certainty. No sponsor notifications are bundled with utility reminders.

Validation: Measure useful alerts versus dismissals and mistaken destinations. Test late schedule changes, time zones, dropped titles and different languages. A push delivered on time is not useful if the episode or edition is unavailable to that user.

D09 — Make availability useful even when it earns nothing

P1 · Data-dependent. “Aired” does not mean legally available in the user's language, region or current subscriptions. Documentation already distinguishes these facts; verified availability is a data problem before a visual one. [P06, F19]

Design decision: Preserve official destinations, show qualified availability only when country, language, service and freshness support it, and prioritize access the user already has. Keep free/legal/library and non-affiliate alternatives where verified. A monetized merchant link is not evidence that a title can be watched there.

Tradeoff: More precise promises narrow coverage. Keep “unknown” explicit and allow a clean official-source link without pretending to know entitlement. Do not build a scraping-dependent availability business as an incidental monetization experiment.

Validation: Check a small set in real target countries/languages against official destinations. Track dead links, wrong territories and unexpected extra charges. Kuro's provider-availability work remains staged according to local documentation; this study did not change or live-verify it.

D10 — Make filters, aliases and sorting visible across devices

P0 · High confidence. MAL and Anime-Planet discussions expose hidden sort, confusing personal/community ratings and mobile parity; AniList users miss existing exclusions. Kitsu's filter-pagination report adds a reliability dimension. [M01, F02, F04, S11–S13, F11]

Design decision: Provide alias-aware title search with a preferred display language and a useful secondary title. Distinguish own-list search from catalogue search; my score from community score; personal status from publication status. Use explicit include/exclude/reset controls and retain selections. Essential sort/filter capability should survive grid/list, small screens and accessibility text sizes.

Tradeoff: Showing all specialist tags upfront overwhelms people. Start with common choices, make active criteria visible, and reveal deeper taxonomy intentionally. Never conflate demographic labels with story themes or protagonist attributes. [G07]

Validation: Test forgotten English titles, punctuation, excluded themes, completed-airing planned titles, mixed media and paginated filtered results. Observe finding and correcting criteria without explaining hidden gestures first.

D11 — Offer a useful shortlist and an escape from it

P1 · Medium confidence. People need contextual choosing, but some want deeper exploration. MangaBaka users request recommendations from a particular list and more than a limited result set; a ResetEra user ultimately finds an existing app satisfactory. [G03, G04, F18]

Design decision: Default to a few varied, well-explained choices based on stated mood, time, format, exclusions and taste. Offer “more like these,” a chosen-list seed and See more. A reason must reflect actual selection signals, not invented AI confidence. Keep editorial and community context available. Sponsored content never secretly alters these rankings.

Tradeoff: More controls can recreate administration. Ask only what helps the current decision; remember optional preferences and allow skipping. A whole-library taste profile should not override a deliberate different mood today.

Validation: Measure whether a user finds and later tries a suitable title, not just card clicks. Compare with current Kuro editorial/Taste Deck behavior. Include obscure-title seekers and people who actively reject similar-to-everything-they-already-know results.

D12 — Keep personal evaluation separate from public score

P1 · Medium confidence. Kitsune reviews from January 2025 and January 2026 ask for opposite score presentations. MAL notes-app comparisons show different scoring and journal preferences. [C05, M07]

Design decision: Always distinguish the user's rating from aggregate rating. Allow aggregate scores to be hidden without changing discovery access; retain a clear view for those who use them. Notes and short personal verdicts are optional. Do not force a score to mark progress or mix a recommendation explanation with an apparently scientific fit percentage.

Tradeoff: Excessive rating scales complicate interoperability. Preserve imported precision even if the default input is simple, and disclose any required conversion when syncing. Do not silently round historical values to fit a cosmetic control.

Validation: Ask users to explain whose score they see, record unrated completion and recover imported ratings. Test both score-led and score-averse users. The goal is comprehension and control, not a universal rating philosophy.

D13 — Support import, recognition and browsing at first use

P1 · Medium confidence in tasks; onboarding outcome unproven. File import serves established trackers; returning fans without a file need recognition-based reconstruction. A request to mark hundreds of remembered shows by genre exposes that distinct need. [M05]

Design decision: Offer Import, Mark what I know and Browse first. Use genre/era/title recognition with explicit exact-entry selection, reviewed batch completion and undo. Preserve unknown historical dates. Taste training remains optional and resumable; it should not block first value.

Tradeoff: A broad completion grid can introduce accidental matches and overwhelm new fans. Keep it a chosen path, use a final preview and avoid automatically selecting every season of a franchise.

Validation: Compare first useful action and later return for users with files, remembered history and no history. Check accidental bulk changes and task abandonment. Existing Kuro Taste Deck should be evaluated rather than discarded on the basis of forum inference.

D14 — Preserve private clubs and optional sharing

P2 · Medium confidence. A-P users ask for a middle ground between public and private. MangaHelpers' actual reading club uses nominations, reasons, polls and shared spoiler boundaries. Other users want rich reviews without a public feed. [F07, G20, C14]

Design decision: Keep understandable club sharing levels and show what others can see. Make a shared reading boundary visible. Use nomination reasons and spoiler-aware discussion; a simple outward share can serve some users without another public social network. Ordinary tracking never requires joining or posting.

Tradeoff: Sharing and moderation have real costs. Sponsor access must not be added to private clubs. Granular privacy that nobody understands is not safer than a small set of explicit levels.

Validation: Ask members to predict which name/status/progress/comment is shared and with whom. Test withdrawing sharing, leaving a club and spoiler protection beyond the agreed point. User comprehension must match server behavior before a privacy promise is made.

D15 — Keep the entire useful core free and uninterrupted

P0 · Explicit business constraint, supported by attention-friction evidence. Kuro will not charge for tracking, ordinary discovery, search/filter/sort, history, import/export or useful reminders. The first report's paid-convenience idea is superseded. MyAniList complaints about recurring prompts—and the correction when sorting became free again—show the practical harm of monetizing through routine friction. [C02]

Design decision: No upgrade modal, artificial daily tracking limit, reward-ad requirement, paid export or pay-to-sort mechanism. Optional purchases happen with an outside merchant because the user wants that product. Partner-funded content must deliver a distinct benefit and be visibly commercial.

Tradeoff: This removes a revenue option; it does not remove costs. Affiliate and sponsor income must earn their place through contribution margin and demonstrated usefulness. If the economics fail, reduce scope/cost or choose a different business, rather than quietly weakening the free-core promise.

Validation: Test the unmonetized core against each pilot; core task success, latency, trust and return use are guardrails, not variables to sacrifice for more clicks.

D16 — Resolve data permissions independently of monetization format

P0 prerequisite · High confidence in published constraints; Kuro agreements unknown. AniList's competing-tracker restrictions are separate from its commercial revenue threshold. MAL's agreement also has commercial, review, storage and advertising conditions. A switch of data supplier is not automatically a solution. [B11, B12]

Design decision: Record permitted use for catalogue text/images, user imports, storage, synchronization, redistribution, commercial display and partner reporting. Obtain scope appropriate to the actual app before revenue-dependent rollout. Retail affiliate approval cannot authorize catalogue usage. Aggregators retain upstream dependencies. [G24]

Tradeoff: Contractual clarity may limit or delay a revenue path. Continue unrelated usability work, but do not fund a business forecast on assumed permission or on remaining below a nominal threshold.

Validation: A written permission matrix with responsible owner, source/version, covered app/domain, geography and renewal/change requirements. This is a future implementation gate; this research makes no finding that Kuro has or lacks a private agreement.

D17 — Preserve editorial identity with functional density and accessible actions

P0 for controls/legibility; P1 for layout changes · High confidence in differing needs. Familiar density is actively defended on MAL; MangaBaka's simplified display prompts a request to restore totals; mobile layout trouble on A-P is concrete. [M01, M06, G01, F08]

Design decision: Keep Kuro's monochrome editorial direction for discovery. Give collection rows legible progress, persistent sort and obvious actions. Support meaningful text enlargement, screen-reader labels and narrow widths. Important controls must not rely only on long press, color or tiny decorative labels. Display optional detail progressively without stripping it out. Some readers actively enjoy organizing personal notes and categories. [G15]

Tradeoff: More visible controls reduce poster-led minimalism. Judge this by fewer errors and faster everyday use, not visual novelty. Do not add arbitrary themes or infinite layout choices to satisfy every isolated request.

Validation: Real update/search tasks on small screens and large text, plus a compact-versus-card comparison. Check empty states, loading, image-viewer return and discoverability before aesthetic polishing.

D18 — Make catalogue coverage and corrections honest

P1 · Medium-high confidence. Readers find differing coverage across trackers; an A-P correction request reveals that submission ease and moderation capacity must be designed together. Vendor catalogue counts do not measure overlap with a user's library. [G10, G21, F01]

Design decision: Define supported media/origins and measure a representative library benchmark. Provide private unresolved placeholders during import when needed. Offer a prefilled report-error/missing-title path with source URL, field and proposed correction, duplicate checks and visible status. Keep metadata provenance; do not replace nuanced publisher descriptions with untraceable generated copy. [G06]

Tradeoff: A submission form without review capacity creates false promises and catalogue pollution. Limit supported corrections, triage queues and communicate delay. Private placeholders should not instantly become public canon.

Validation: Separate title coverage from successful matching. Test obscure manga, one-shots, aliases, manhwa/manhua and deliberately included/excluded Western webtoons. Track correction turnaround, false merges and reopened errors, not submission volume alone.

19.6. Monetization that pays for a useful next step

The strategic constraint is **free access to the product**. Users may choose to buy a book, ticket or outside service, but Kuro does not charge them to remember, discover or keep up. A good commercial feature should still be useful if its commission were zero. That is the simplest test for distinguishing assistance from disguised advertising.

Recommended order

Model	Who pays / what they receive	Benefit to users	Recommendation
Exact-volume referrals	Retailer pays for an eligible referred purchase	Correct edition/language, local availability, fewer duplicate purchases	First small pilot, after data and app permissions
Licensed samples / sponsored discovery	Publisher funds a defined, labelled useful asset or campaign	Try an official work before buying; find a relevant translated edition	Second pilot with an actual partner and bounded cost
Optional club/event resources	Publisher, distributor or event organizer funds a kit/activity	Legal sample, reading guide, creator Q&A or relevant event	Later and opt-in; no private-club ad inventory
Publisher/retailer discovery tools	Business pays for a scoped tool on its own property	Better navigation through that partner's catalogue	Separate B2B hypothesis; secure paid demand before building
Voluntary publisher research	Publisher commissions disclosed feedback on selected samples	A chance to influence future legal translations/releases	Defer; distinct opt-in, no pooled history sale
Generic display/behavioral ads	Advertiser pays for impressions/targeting	Usually little direct benefit	Not the recommended base model; exclude intrusive/behavioral forms

The strongest initial combination is **a narrow book-referral utility plus, later, a genuinely useful sponsored sample**. It is not yet a proven sustainable business. The first is easier to test but can produce little revenue at small scale. The second may fund more work per agreement but introduces sales, rights, production and servicing costs. B2B tooling may earn earlier money from a business customer, but it is separate product/services work and can distract from Kuro.

Actual market precedents, with limits

MAL's 2026 Publisher & Bookseller Picks explicitly identifies sponsor selections; its MAL×Japan area hosts industry-linked discovery and participation. This establishes a commercial discovery surface, not neutral recommendation quality or disclosed fees. [B01, B02]

MAL's Manga Sampler Pilot offers authorized opening chapters and asks about future translation demand. Its July–September 2026 giveaway had already closed by this research date. The experimental service discloses unreviewed machine translation, availability limits and free-for-now status. It proves a mechanism, not publisher payment or profitability; Kuro should not copy its translation or broad data permissions by default. [B03]

NTT DOCOMO's March 2025 MANGA MIRAI announcement describes previews and purchases integrated into MAL. KADOKAWA's 2025 writing contest with MAL/Honeyfeed and its February 2026 result demonstrate a real creator opportunity. These collaborations show publishers will work through fandom platforms, but do not establish willingness to pay a small Kuro audience or prices for club sponsorship. [B04–B06]

LiveChart and Anime-Planet describe restrained advertising supporting operations. Anime-Planet says profiles are ad-free and rejects intrusive formats. Their statements are useful design precedents, not audited proof that ads alone fund a commercial team. MAL's advertised skins/banner/mobile inventory is a different approach and not a recommendation for Kuro. [B07, F23, B09]

Merchant feasibility: current public terms, not Kuro approvals

Germany is the initial worked example because of the user's operating context; the intended launch geography was not otherwise specified. Programme geography, audience and native-app permission must be verified before implementation. Rates below were checked on 25 September 2026 and can change.

Route	Verified public commercial detail	Territory / permission limit	Assessment
Thalia / Awin	Books 11%; non-price-bound books with a coupon 8%; 30-day attribution; daily product feed	German programme; accepted commission base and Kuro native-app placement still need confirmation. “App-Tracking” does not prove publisher-app approval.	Best DE first investigation; exact-volume utility. [A01, A02]
Amazon DE	Books currently 4.5%; ordinary session generally 24 hours, with conditional cart extension	Mobile app must be approved and meet original-content/free-access rules. No Amazon WebViews; permitted linking/price tools and relevant policy apply.	Additional neutral option, not a commission-driven default. [A03, A04]
Amazon US	Physical books 4.5%; some qualified Prime Video actions have published bounties	US programme only; eligible action/customer and storefront-specific digital-link rules matter	Do not transplant US bounties into Germany or budget unapproved streaming income. [A05]
Bookshop.org US	Standard media affiliate 10%, 48-hour last-touch window; enhanced offer carries audience/exclusivity conditions	US affiliate terms; US and UK sites have domestic fulfillment, not a German shipping route. UK programme terms require separate confirmation. Native app eligibility unknown.	Good potential indie option for a relevant market; reject exclusivity that removes neutral shop choice. [A06–A08]
Rakuten Kobo	Public page says up to 5% for ebooks/audio/devices and 14-day attribution	Exact accepted rate, German route and native-app placement unconfirmed; “up to” is not a promise	Secondary legal ebook route after approval and iOS review. [A09]
CDJapan	5–7% by prior sales tier; 30 days; paid/shipped orders qualify	Site definition does not establish native-app permission; shipping, cancellations and payout timing matter	Niche Japanese-edition/artbook/merch route when explicitly sought. [A10, A11]
EVENTIM DE	20% of net presale fee, not ticket price; page estimates that fee at 8–10% of net ticket	Programme has event exclusions; header says one-day attribution while text says session, requiring clarification; application/placement approval needed	Optional relevant-event experiment, not a dependable core forecast. [A12]
Crunchyroll streaming/store	No authoritative current public rate and acceptance terms were verified	Streaming/store, country, app traffic and contract remain unresolved	Accurate non-affiliate links remain valuable. Model zero revenue until accepted terms.

No programme was joined and nobody was contacted. A legal official link is not automatically an affiliate link. Programme acceptance is not permission to use a third party's catalogue, images or user data commercially.

19.7. Monetization decision register

These ten decisions make the free-core commitment operational. Commercial attractiveness is a hypothesis until Kuro has valid permissions, useful task outcomes and accepted revenue.

N01 — Pilot “Find the next volume,” not a general storefront

First commercial test · Medium confidence in utility; revenue unproven. Collector discussions describe difficulty finding upcoming editions and duplicate preorders. Chapter-first readers also need reliable edition mapping. Pair that need with a user-requested retailer choice, rather than showing random merchandise after a progress tap. [G12, G17]

Flow: The user opens Get this volume → chooses language/edition → sees verified local options and total-cost context → chooses a merchant → returns to the same Kuro state. Keep a clean official/non-affiliate option. Do not equate an outbound click with a purchase, ownership or reading progress.

Payer/value: An approved retailer pays for a qualifying attributed sale. The user benefits from finding the right item faster, not from an unverified claim that Kuro has the cheapest price or a discount.

Tradeoff and test: ISBN/edition mapping, stock, shipping and link maintenance cost money. Start with a limited manually verified set in one country. Stop on persistent wrong-edition or wrong-language recommendations. Expand only if usefulness and contribution justify maintenance.

N02 — Keep retailer and recommendation rankings independent of payment

Permanent design rule. Rank purchase options by exact item match, language/region, availability, reliable total cost and chosen preferences. Do not use commission as a relevance signal. Keep non-paying merchants and already-owned subscription access visible when useful. [A01–A12]

Commercial content: A paid placement, if offered, is a visually separate labelled module. A sponsor cannot purchase a hidden lift in “because you liked…” recommendations, suppress criticism or exclude competing stores. Reject an enhanced affiliate arrangement when its exclusivity would undermine neutral choice; Bookshop's enhanced terms illustrate why this needs contract-level checking. [A07]

Tradeoff: Kuro may earn less per click than an aggressive affiliate site. The strategy depends on repeated trusted use, not maximizing the payout of a single session.

Validation: Audit results with affiliate flags/rates removed: organic order should be unchanged. Ask users who paid for a sponsored item and why a shop is ranked first. A disclosure that exists but is misunderstood is insufficient.

N03 — Sell a licensed sample benefit, not an undisclosed recommendation

Second pilot · Primary precedent, unproven Kuro demand. A publisher funds a clearly labelled sample or useful discovery feature. MAL's sponsor picks, sampler and MANGA MIRAI integration establish relevant mechanisms; they do not disclose a transferable campaign rate. [B01, B03, B04]

Product: Start with a publisher-hosted, properly licensed sample link showing language, available chapters, territory and any expiry. It should help someone assess a work without buying. A separate sponsored card may introduce it in Discover, with hide/dismiss controls. Avoid an embedded reader initially unless the extra rights and maintenance are justified. No sponsor inside Continue, import/export, private notes or progress controls.

Commercial structure: Negotiate a bounded pilot fee for agreed placement/production, or a capped measurable engagement arrangement. Do not guarantee sales or positive reviews. Catalogue/image rights and translation quality need explicit treatment.

Validation: Measure useful sampling, voluntary saves after sampling, helpfulness, hides, load impact and contribution after labour. Clicks caused by confusing placement count against the experiment. A free sample available only in the wrong country is not a benefit.

N04 — Keep sponsorship optional, sparse and understandable

Permanent UX rule. A user can ignore or hide a campaign and retain the full tracker. Sponsor identity and the commercial relationship appear beside the content; the relevant local advertising label must be clear. Commercial disclosure is not buried in terms. [B14]

Design starting point: Test no more than one commercial module in a discovery session, with persistent campaign hiding and no repeated opening modal. That frequency is a proposed ceiling for a pilot, not an industry-validated optimum. Prefer contextual selection from the page's subject over profiling a person's history. Ordinary organic discovery remains usable when no sponsor is available.

Tradeoff: Lower exposure may constrain what sponsors will pay. If a campaign requires repeated nagging, accidental clicks or a blocked task to meet its target, reject the deal rather than alter the free-core promise.

Validation: Include an unexposed comparison group. Watch comprehension, task completion, latency, hides/reports and 7/28-day return alongside earned revenue. Do not interpret increased dwell time as success if users are struggling to exit.

N05 — Let partners fund an optional club benefit, never access to a club

Later · Medium evidence of collaboration, low evidence of Kuro demand. A publisher could fund a reading guide, authorized sample, creator discussion or relevant event. Actual industry contests demonstrate collaboration, while MangaHelpers shows what small reading groups do; neither proves a sponsor market for Kuro clubs. [B05, B06, G20]

Product: Offer a separate public event/resource page. A club organizer may choose a kit; members can decline. Sponsor identity, rights, age/territory limits and recording rules are explicit. Existing clubs keep ordinary polls, planning and discussion without participating. Sponsors receive no members' names, notes, private progress or membership lists.

Tradeoff: Events introduce coordination and moderation costs. Start with a simple static resource and publisher-hosted sample before a live event. No reward for positive ratings, public sharing or unwanted marketing consent.

Validation: Measure voluntary uptake, useful participation, repeat club use, moderator hours and net contribution. Cancel a pilot that trades privacy or participant attention for sponsor deliverables users did not choose.

N06 — Treat B2B discovery tools as a separate business hypothesis

Defer until paid demand · Low confidence. A publisher or bookstore might pay for a small recommendation questionnaire or curated widget using its own licensed catalogue. Existing MAL-linked marketing services show adjacent B2B activity, not proof that a neutral tracker can profitably sell reusable widgets. [B10]

Product: Clearly say “From this publisher's catalogue.” Use session preferences or the partner's own content, not access to Kuro histories. Keep the consumer app free regardless of the business relationship. Define integration, support and update responsibilities before building.

Tradeoff: Custom work may earn cash but consume the team in one-off services. A sponsor wanting a unique microsite is not recurring software demand. Do not build a multi-tenant platform in anticipation of buyers.

Validation: Obtain an actual paid pilot before substantial engineering, then measure discovery usefulness, support hours, contribution and willingness to renew. Scope and customer commitments would require a later authorized initiative; no outreach occurred in this research.

N07 — Avoid selling histories under an “insights” label

Permanent boundary; optional research is separate. No sale of personal lists, inferred tastes, private notes, club activity or event-level user identifiers. Removing names does not automatically make a dataset anonymous; small cohorts and combinations can still identify people. [B18]

Permissible proposal: Report broad campaign delivery totals with appropriate small-count suppression. A future publisher-feedback study would be a distinct opt-in task explaining the recipient, purpose, retention and reuse, with no effect on tracker access. The MAL sampler demonstrates a feedback mechanism, not permission to copy its data terms. [B03]

Tradeoff: This excludes targeting/longitudinal-data opportunities and may reduce advertiser interest. It protects the intended product relationship. Underlying measurement still needs a lawful basis even when the final output is aggregated. [B17]

Validation: Audit exactly what leaves Kuro and the merchant subtags; no private history or stable user identifier should be transmitted for affiliate reporting. Test opt-out and deletion against actual data flows before making privacy claims.

N08 — Keep utility reminders independent of commercial messages

Permanent design rule. A chosen title's release alert is a service the user requested. A partner giveaway or marketing message is a different purpose. LiveChart's separate categories provide a useful precedent; native reviews show how irrelevant reminders damage trust. [F20, C02, C14]

Design decision: A user can subscribe to a specific edition's availability without agreeing to broad promotions. Keep sponsor messages out of push/email in the initial model. Apple also restricts display advertising in notifications and widgets. [B13]

Tradeoff: A commercial partner may prefer push reach; that is not part of the initial inventory. Opting into a release reminder should never be treated as consent to behavioral advertising or a sponsor contact list.

Validation: Check removed/dropped titles, quiet hours, timezone changes and consent withdrawal. Track usefulness and unwanted-alert reports. An update to merchant inventory must not produce repeated promotional alerts or imply a release date is certain when it is not.

N09 — Review each channel's rights, geography and payment rules separately

Prerequisite · High confidence in constraints; implementation-specific assessment still required. A public website, a German iOS app and a US iOS app are not interchangeable distribution channels. Physical books consumed outside the app follow different Apple payment rules from digital content/service purchase links. A tracker does not automatically qualify for a reader-app entitlement. [B13]

Decision: Obtain merchant approval for the exact placement/app; verify attribution without intrusive tracking; confirm content/API commercial scope; and review the chosen storefront flow. German/EU disclosure, storage/access and personal-data requirements apply according to the actual implementation. Avoid behavioral ad targeting by design rather than treating every cookie or every DSA obligation as identical. [B11, B12, B14–B17]

Tradeoff: This is bounded implementation diligence, not a giant compliance feature project. Start with a simple approved physical-book route and clean links. Contextual placement reduces complexity but does not automatically eliminate privacy obligations.

Validation: A signed/accepted permission record and tested compliant route for each pilot. Public “App-Tracking” wording or a competitor's example is not evidence of Kuro authorization.

N10 — Require useful outcomes and positive contribution before scaling

Permanent commercial gate · Revenue hypothesis. Do not judge monetization by raw clicks, registrations or a nominal commission percentage. Approved attributable revenue must cover the incremental data, editorial, engineering, sales, servicing and support cost of the feature. Sponsor fees are not recurring revenue until renewal is demonstrated.

Decision: Keep a comparison group without the commercial treatment. Report correct-item success, sample usefulness, core tracking success, return use and complaints alongside approved revenue. Revenue generated by wrong-edition confusion, hidden paid rank or accidental taps fails the pilot regardless of gross results.

Tradeoff: At a small audience, the economically honest result may be “not enough yet.” Reduce scope or pursue separately justified partner services; do not add friction to make users click. Cost of running the free core remains a business obligation.

Validation: Review settled commissions and real labour over a defined pilot period. Set quantitative noninferiority margins after establishing a baseline and adequate sample size. The illustrative calculations below are sensitivity analysis, not a promise of profitability.

19.8. Economics: what would have to be true

There is no measured Kuro audience, conversion, accepted merchant rate or publisher budget in this research. Every scenario is illustrative. The point is to expose the scale and cost questions before investing, not to manufacture a business forecast.

Affiliate revenue formula

Approved monthly commission = monthly active users × share with a relevant shopping need × share of those users clicking × orders per click × commissionable basket × commission rate × approval after refunds × attribution survival.

“One click opportunity per relevant user” is the simplified assumption; repeat purchases are not added. Attribution survival represents such things as lost/cross-device attribution, another last-click affiliate or ineligible transactions. Its relationship with conversion is not necessarily independent. Use the merchant's accepted commissionable base, not an unexplained retail checkout total.

Illustrative month	Audience and funnel assumptions	Basket / commission / settlement assumptions	Result before costs
Small DE pilot	10,000 MAU; 40% relevant need; 20% of those click; 4% order	€20 commissionable basket; 11% rate; 90% approved; 80% attribution survival	**€50.69/month**
Larger manga audience	50,000 MAU; 50% relevant; 25% click; 6% order	€25 basket; 11%; 92% approved; 85% attribution	**€806.44/month**
Strong-intent example	100,000 MAU; 50% relevant; 25% click; 8% order	€30 basket; 11%; 95% approved; 90% attribution	**€2,821.50/month**

The 11% is a currently published Thalia book rate, conditional on applicable accepted terms. **Every other input is an assumption**, and accepted basket basis is still unresolved. These rows change both audience and behavior; they are not a measured growth curve. Under the first row's unchanged assumptions, about 197,286 MAU would be needed for €1,000 monthly commission. Actual results could be substantially lower or higher. [A01, A02]

The lesson is not that affiliates cannot work. It is that a high headline percentage applied to occasional small purchases may generate little per active user. It can offset costs or become meaningful with a concentrated purchasing audience, but it does not justify a sprawling marketplace. Infrastructure, licensed data, labour, taxes,

support and network costs are not deducted above.

Ticket example: At an assumed €50 net ticket and a 9% net presale fee, EVENTIM's 20%-of-fee rule yields €0.90, not €10. One hundred approved attributed sales would yield €90 under those assumptions. Actual event fees and eligibility control. [A12]

Sponsorship contribution, without invented market prices

No verified source disclosed what Kuro could charge for a sample campaign. Negotiate only after measuring relevant reach and showing a useful asset. Price must cover creative/rights costs, delivery/measurement, sales and servicing hours, and a reasonable contribution toward the free product.

Pure arithmetic example, not a market quote: If a pilot were priced at €1,000 and took eight total hours at an assumed fully loaded €50/hour plus €150 for permitted assets/hosting/other variable costs, contribution would be €450 before general overhead and tax. At sixteen hours it falls to €50. If no sponsor buys it, revenue is zero. This is why contract scope and servicing effort matter as much as a sponsor logo.

Do not assume one campaign per month, continuing renewals or an advertiser's budget. Publisher revenue is concentrated and sales-led. A free sample linked without a sponsor may still help retention, but it is not monetization until a commercial relationship actually exists.

Costs to record before a decision

For each pilot record catalogue/image rights, mapping/edition maintenance, feed/link checking, privacy/measurement implementation, accessibility, content production, account management, support, refunds/reconciliation, and ordinary infrastructure. Separate one-time setup from recurring costs. Report **incremental contribution** and overall product sustainability separately; a positive campaign does not mean Kuro as a whole is profitable.

19.9. Application to Kuro and a bounded sequence

This mapping uses local source and canonical documents read during this initiative and the immediately preceding pass. It is not a signed-in simulator test, remote database check, production audit or authorization to change live flags.

Existing local capability	What the evidence changes	First evaluation
Collection card/list +1 controls already exist	Native speed is table stakes; findability, confirmation and recovery are the questions	Update current titles, rapid taps, undo, background/reconnect and seasonal views
Concierge AniList import preview/confirmation	Do not duplicate a flow; assess preservation and explanation	Missing dates/coverage, duplicates, unresolved fields, export and connector direction
AddToListSheet progress logic uses a known maximum for visibility	Unknown-total manga is a specific source-review candidate	Validate actual ongoing manga journey before declaring a shipped defect
Taste Deck and editorial discovery	Keep identity; test optional onboarding, contextual seeds and more-results path	Time to first value and a title actually tried later
Adaptation/relationship tools and title detail	Grouping needs exact underlying identity	Complex franchises, inherited progress, false matches and continuation clarity
Search/Browse filters	Existing power may be hard to reach	Own-list versus catalogue search, include/exclude, alias and device parity

Existing local capability	What the evidence changes	First evaluation
Private clubs, polls and privacy levels	Preserve the useful social niche	Member understanding, shared boundary, spoiler safety; no sponsor access
Staged provider availability	Accurate region/language remains a data prerequisite	Keep staged status until source, cost and permission strategy is resolved
AniList-sourced catalogue	Monetization requires actual rights scope	Owner review of agreement and allowed storage/commercial distribution

The source locations reviewed include Kuro/Views/EditorialCollectionComponents.swift, Kuro/Views/AddToListSheet.swift, Kuro/Views/ConciergeView+AniListImport.swift, Kuro/Services/SupabaseService.swift, Kuro/ContentView.swift, and the three canonical state/plan documents. A source pattern is a candidate to test, not a claim about every live user's behavior. No application source was changed by this research.

Phase 1 — Validate the trust and daily-use foundation

Evaluate D02–D05, D07, D10 and D17 against the existing app. Prioritize failures that lose/misrepresent history, block unknown-total progress or require inaccurate status changes. Establish basic task and return-use baselines. Add no commercial interruptions. Run the data-permission review in parallel as a separate owner responsibility.

Phase 2 — Prove better choosing and keeping up

Test franchise grouping, useful contextual discovery with optional depth, first-use recognition, release relevance and catalogue correction intake. Preserve existing Taste Deck/editorial/clubs value. Avoid broad social expansion, general inventory and universal two-way sync before the core behavior and maintenance capacity are proven.

Phase 3 — One approved commerce pilot

Choose one country and a limited set of verified physical editions. Confirm exact merchant/app/data permissions. Offer neutral next-volume help only when requested; include clean non-affiliate choices. Measure correct-item task success and approved commission with a no-commerce comparison group. Do not extend the pilot simply because a merchant feed contains millions of products.

Phase 4 — One useful publisher-funded experiment

Only after measurable reach and a partner agreement, test a licensed sample or small resource kit with bounded production cost. Keep organic recommendations and personal surfaces independent. Pursue business widgets or research services only if a specific paying customer validates a repeatable need; neither is authorized implementation in this report.

19.10. Validation plan: fill the gaps forums cannot answer

Desk research identifies candidate problems and design constraints. It cannot establish Kuro's conversion, incidence of competitor bugs, target segment size, catalogue overlap, willingness to use commerce, or a partner's willingness to pay.

Recruit for differences, not only enthusiastic power users

Start with a proposed 16–20 moderated participants across returning/casual, seasonal, manga-first and established-library users, with overlap allowed. Include at least a few people who abandoned tracking and users with accessibility needs. Choose launch-country/language coverage deliberately. This is a practical discovery sample, not a statistically representative survey. Ask for real workflows and voluntary example titles; do not require private notes or account credentials.

Task	What to observe	What would change the decision
Resume and update a current title	Time, mistaken entry, confirmed saved state, ability to undo	Existing flow already succeeds; improve specific friction rather than rebuild navigation
Isolate this season without changing status	Understanding of collection view vs record state	Users still move shows to Dropped or cannot locate older in-progress titles
Restore a remembered history / import a file	Wrong matches, unknown dates, missing titles, effort to resolve	Loss of fields or silent omissions block migration rollout
Log volume-only or unknown-total manga	Unit comprehension, preserved values, correct filters	Anime-shaped assumptions still hide or mislabel progress
Start manga after an anime / partial reread	Bookmark versus actual consumption	Inflated statistics or confusing advanced controls require model/UX revision
Find a title by another name; exclude a theme	Alias matching, visible criteria, pagination and reset	Hidden controls or failed loads masquerade as absent catalogue coverage
Choose tonight from a selected mood/list	Explanation usefulness, need for deeper results, later trial	More clicks without a worthwhile title does not validate discovery
Follow a relevant release	Correct region/language/date, notification relevance	Wrong or duplicate alerts outweigh convenience
Choose the next physical volume	Correct ISBN/edition, total cost, neutral choice, return state	Wrong-edition risk or confusing disclosure stops commerce
Use a sponsored sample	Who paid, usefulness, ability to ignore, impact on core task	Users mistake it for an independent recommendation or core use worsens
Share inside a club	Prediction of what is shared and spoiler boundary	Misunderstood privacy blocks social or partner expansion

Small catalogue benchmark

With permitted source access and voluntary real-library examples, construct a proposed 100-title test set covering mainstream anime, split seasons, alternate adaptations, ongoing and obscure manga, one-shots, manhwa/manhua, alias-heavy works, volumes/omnibus editions and intentionally scoped Western webtoons. This benchmark was **not run** in this research. Report correct coverage, successful identity match and freshness separately; record why misses occur. A huge vendor count cannot substitute for this exercise.

Pilot measurement and stopping rules

For product changes: task success/error rate, time to durable update, successful reconciliation, false matches, recovery effort, and 7/28-day return. For discovery: a useful chosen title and later consumption, not endless swipes. For commerce: correct product found, broken/wrong links, user trust, settled attributable commission and incremental cost. For sponsors: useful asset engagement, explicit recognition of sponsorship, hides/complaints and contribution after servicing.

Do not invent significance from a small pilot. Set noninferiority margins and a sample plan after measuring baseline behavior. Stop immediately for silent history corruption, undisclosed paid ranking, unauthorized private-data disclosure or affiliate-induced wrong-progress changes. Investigate sustained increases in wrong-edition complaints, core-task failure or notification fatigue before expansion.

Open decisions for an implementation initiative

Confirm initial launch countries/languages, primary segment, available catalogue/image permissions, willingness to operate a corrections queue, edition-data sources and budget. Measure actual product costs and reach. Then validate merchant placement and a specific sponsor proposition. These are decision inputs; this report does not contact partners, enroll programmes, approve spending or start implementation.

19.11. Evidence register

All sources were accessed during this research on 25 September 2026 or retained from the directly preceding pass the same day. Source IDs are clickable in the PDF; titles link to the original page. Dates below identify publication, review update, explicit issue update or approximate displayed age. **Undated official references and older live agreements are not fresh user testimony.** No public post is a confirmed population statistic. Hosting domains count platforms such as Reddit once; many subreddits are not presented as different websites.

MyAnimeList communities and official app references

M01 — Is there any chance of UI rework?

Original source · **Date:** 2026-08-20 (indexed exact post date) · **Type/access:** Community discussion: r/MyAnimeList; Direct public thread and replies via web.open, accessed 2026-09-25

A user describes hidden seasonal sorting and overloaded mobile menus, but replies explicitly defend dense old-style desktop design and report no navigation difficulty. Another wants date displayed as a list column. Functional findability and compact presentation should be tested separately from aesthetic modernization. **Boundary:** Self-selected preferences, not a consensus or hands-on feature audit. No maintainer resolution observed.

M02 — What would your perfect anime app actually look like?

Original source · **Date:** 2026-05-15 to 2026-05-20 (indexed discussion dates) · **Type/access:** Community discussion: r/MyAnimeList; Direct public thread and replies via web.open, accessed 2026-09-25

Participants report MAL app redirects for cast/information and ask for a whole-series view instead of managing many season entries. Social preferences conflict: some want community, others less social, and one says AniList already satisfies their needs. Supports franchise grouping and optional social without flattening exact histories. **Boundary:** Prompted wishlist thread; possible builder bias. Streaming-player wishlists excluded. An AniList-app claim is ambiguous between website/PWA/community clients.

M04 — Recommendations for a FOSS anime tracking app?

Original source · **Date:** 2026-07-10 (indexed exact post date) · **Type/access:** Community discussion: r/animeapp; Direct public thread and replies via web.open, accessed 2026-09-25

A user requests current-season filtering and fast progress updates. To isolate this season they moved genuinely in-progress older titles to Dropped. This is a concrete workaround that distorts personal history, supporting seasonal collection views separate from canonical status. **Boundary:** Does not establish that every MAL interface lacks the filter. Replies recommend clients but no successful resolution is confirmed.

M05 — Anime website or app for a watchlist

Original source · **Date:** 2026-07-17 (indexed exact post date) · **Type/access:** Community discussion: r/animeapp; Direct public thread and replies via web.open, accessed 2026-09-25

A returning fan reporting over 500 watched anime wants genre-based visual selection and one-click completed status rather than typing every name or seeing only popular titles. Follow-up questions concern account connection and sync. Supports optional bulk history reconstruction beyond file import. **Boundary:** Single detailed workflow, not prevalence evidence. Suggested clients were not verified to solve the batch task.

M06 — It's 2025, and MAL still looks like it came out of the '80s

Original source · **Date:** 2025-02-14 to 2025-02-15 (indexed exact discussion dates) · **Type/access:** Community discussion: r/Animemes; Direct public thread and replies via web.open, accessed 2026-09-25

Replies under a design meme show strongly divergent preferences. Multiple users defend predictable sections, information density and performance; others prefer AniList or find MAL frustrating. This reinforces preserving a compact mode and testing actual task completion instead of assuming newer-looking design is better. **Boundary:** Humor community, selection bias and vote counts cannot establish market shares. Unverified API accusations excluded.

M07 — Started Watching Anime a Year Ago

Original source · **Date:** Post 2025-07-31; relevant replies 2025-08-01 (indexed exact dates) · **Type/access:** Community discussion: r/MyAnimeList; Direct public thread and replies via web.open, accessed 2026-09-25

Notes-app users want half-point scores, whole-series ratings, chronological memories and detailed next/later categories. A large-list user values MAL's linked saved state, friends, authors and reviews and enjoys quick progress increments. Different list sizes and motivations change what counts as simplicity. **Boundary:** Five statuses satisfy another participant. Randomizer is clarified as third-party. Do not convert perceived rating pressure into a claim that rating is technically mandatory.

M08 — What is wrong with mal app

Original source · **Date:** 2025-10-07 (indexed exact post date) · **Type/access:** Community discussion: r/MyAnimeList; Direct public thread and replies via web.open, accessed 2026-09-25

A Huawei user reports recurring connection errors; replies distinguish an SSL-type issue from a separate HTTP 500 report and discuss comparing browser access. Another person reports no problem. This supports accurate error states and preservation of useful cached history during failures. **Boundary:** No maintainer-confirmed root cause or global scope. Historic incident and no verified resolution; not evidence of a current outage.

M09 — myanimelist tracking search not working?

Original source · **Date:** 2026-09-09 to 2026-09-10 (indexed dates and relative ages) · **Type/access:** Community discussion: r/mangapiracy; Direct public thread and replies via web.open, accessed 2026-09-25

OP explicitly identifies Mihon stable: MAL search fails while updates to an already-linked series work; MangaUpdates search works and reauthentication did not help. Search, identity mapping and progress writes need independently visible health and recoverable errors. **Boundary:** Third-party integration, not official MAL app. Root cause unresolved and no reading or piracy workflow recommendation is made.

M11 — MyAnimeList Official — Romanian App Store

Original source · **Date:** Review 2025-07-13 · **Type/access:** Official store customer review; Direct store page read via web.open, accessed 2026-09-25

A recent reviewer values the official iOS app's compactness and easy list updating without needing the website. This is useful positive counterevidence: a utility-only segment already gets meaningful value from MAL, so new features should not obstruct that simple use. **Boundary:** One storefront review, not prevalence. Nearby 2021 manga-search complaint is out of period and excluded; do not compare regional overall star ratings as equivalent samples.

M14 — MyAnimeList Official — US App Store baseline

Original source · **Date:** Accessed 2026-09-25; displayed latest version dated 2024-03-08 · **Type/access:** Current official product listing; not recent sentiment evidence; Direct store listing via web.open

Official iOS listing advertises anime/manga progress, seasons, reviews, recommendations, friends and groups. Displayed privacy label includes tracking/advertising identifiers. Latest displayed iOS version is March 2024, while

Android listing has a May 2026 update; platform maintenance claims need qualification. **Boundary:** Developer disclosure is not a privacy audit. Prominent 2021/2024 reviews excluded from recent sentiment. Do not infer MAL is abandoned across platforms.

Anime-Planet, Kitsu, LiveChart, AniDB and independent forums

F01 — Manga corrections are hard to submit; moderation capacity is limiting

Original source · **Date:** 24 September 2026 (HTML time attributes; UI Yesterday on Sep25). Two posts, user + database moderator. · **Type/access:** community forum; full thread directly via Codex in-app browser DOM, including moderator response.

user requests a report-error action and structured missing-manga form rather than copying a pinned forum template. Moderator added a cleaner template, says form has not been prioritized because volunteer developer time and poor-source submissions reduce expected value. medium-high for specific friction; hypothesis for Kuro: prefill title/ID, require source URL and changed value, retain status. Human review remains essential. No claim every A-P user has difficulty. **Boundary:** a small format improvement was delivered; no structured form promised. Moderator's claimed proportion of poor requests is anecdotal, not a measured research statistic.

F02 — Site-rating sort exists on desktop but is hidden/absent on mobile

Original source · **Date:** 3–6 July 2026. Six posts. · **Type/access:** community forum; full direct browser thread.

user says Rating is misleading/useless on Want to Watch because it means personal rating. Reply points to AVG sorting in desktop table view. Another cannot find mobile version; third reports only exposing it by desktop-site plus landscape workaround. OP wants it in grid too. **Boundary:** not evidence feature entirely missing. Desktop capability is counterevidence; mobile inconsistency is reported, not independently interaction-tested.

F03 — Start at manga chapter X; partial rereads and rewatches

Original source · **Date:** 21 November 2025; replies 1 June and 5 July 2026. Three posts. · **Type/access:** community forum; full direct browser thread and machine dates.

OP loves time-spent stats but switched from One Piece anime to manga, reading one chapter after ~1000 caused past chapters to be counted. Reply wants reread tracking for only ~20 chapters/an arc. Another proposes reading/viewing instances and skipped-episode support, noting backend effort. **Boundary:** user request; no staff implementation commitment.

F04 — Binge-ready filter request is partly an existing hidden feature

Original source · **Date:** 14 May and 13 June 2026. Two posts. · **Type/access:** community forum; full direct browser thread.

OP wants completed-airing shows from Want to Watch; moderator explains Other already has Completed/Ongoing/Unreleased, combined with airing-date filters. Moderator says finished recently needs a defined time window. **Boundary:** high for confusion/counterevidence. Distinguish My status from Release status, surface Ready to binge as an optional shortcut built from precise filters. Do not report absent completed filter.

F05 — Apparent lost list entries: developer suspects failed saves

Original source · **Date:** 3–23 May 2026. Five posts. · **Type/access:** community forum; full direct browser thread.

user reports formerly watched/stalled entries now unwatched. Developer initially suggests merged duplicates, asks examples. After user supplies titles and re-adds them, developer checks feed and challenge tables and finds no prior mark; guesses timed-out save after tab switching/navigation might hide normal error alert. **Boundary: not confirmed deletion, not proved timeout root cause**, no deployed fix in thread.

F06 — Per-title countdown and consolidated new-episode summary

Original source · **Date:** 13 May 2026. One post, no reply. · **Type/access:** community forum; full direct browser thread.

requests exact next episode timer, alerts, and number of new episodes across tracked series since last check. **Boundary:** medium-low isolated request. Personal Upcoming with since-last-visit catch-up can reduce scanning. Benchmark LiveChart; do not invent region-specific availability from global airing.

F07 — More privacy than everyone/only me

Original source · **Date:** 18–21 April 2026. User + developer. · **Type/access:** community forum; full direct browser thread.

user wants friends/followers-specific fields; developer says following/followers privacy is something to investigate, per-status/list privacy not soon. **Boundary:** medium-high. Kuro club sharing fits need. Offer small number of understandable levels, not user's proposed sixty combinations. “Planned investigation” is not shipped.

F08 — Mobile width/zoom breaks layout

Original source · **Date:** 26 February–26 March 2026. Same user two posts. · **Type/access:** community forum; full direct browser thread.

user tested multiple Android phones/tablet and browsers, reports horizontal overflow, image viewer pinch/next state shrinks underlying page; later offers browser-config workaround with tablet tradeoff. medium. Validate narrow widths, image-viewer exit, large text and orientation. This is concrete mobile ergonomics rather than subjective visual taste. **Boundary:** no staff fix in thread; user report not reproduced here.

F09 — Exported missing dates are historic unknowns, not a bug

Original source · **Date:** 6–7 September 2026. User + developer + user. · **Type/access:** community forum; full direct browser thread, marked Not a bug.

user sees many null started/completed dates in JSON export. Developer explains those fields were not tracked before 2016. User accepts explanation and thanks fast reply. **Boundary:** high. Preserve Unknown dates with provenance; never fill import timestamps as original watch dates. Counterexample to blaming every missing field on current data loss.

F10 — Kitsu import requested because MAL manga coverage is incomplete

Original source · **Date:** 1 March 2026. One post. · **Type/access:** community forum; full direct browser thread.

user requests Kitsu import, says MAL lacks some manga entries, links Kitsu exports. No staff reply visible. **Boundary:** medium-low isolated demand. Report import coverage and unresolved titles; avoid forced intermediary formats that silently drop unsupported titles. Verify any integration's permissions separately.

F11 — Kitsu advanced search reportedly stops pagination

Original source · **Date:** 21 February 2025; still marked Open when read. · **Type/access:** primary developer issue; primary GitHub issue page directly via web; body read, no reply visible.

reporter says advanced filters stop further result loading, removing filters sometimes restores it, attaches video (not watched). **Boundary:** medium report, not reproduced. Test filter + load-more + empty/failure states together, distinguish exhausted results from request failure.

F12 — Kitsu “More info” can be a dead-end control

Original source · **Date:** 23 March 2026; Open when read. · **Type/access:** primary developer issue; primary GitHub issue directly.

enhancement asks not to render More info when no extra information exists; currently merely changes label to Less info according to reporter. **Boundary:** medium-low isolated report. Hide truly absent metadata sections or explain

unavailable data; avoid actions that only animate a state without useful result.

F13 — Kitsu password autofill leaks into search UI

Original source · **Date:** 24 January 2025; Open when read. · **Type/access:** primary developer issue; primary GitHub issue directly.

Firefox treats search bar as username after password page; autofill UI persists across navigation until reload. **Boundary:** medium-low unreplicated UI report. Test password-manager/browser autofill in real auth flows. Not evidence of credential disclosure or security breach.

F14 — Kitsu status separates functional website from unavailable app-store components

Original source · **Date:** page updated **25 September 2026 at 09:12 UTC**, read today. · **Type/access:** official service status; official current status page directly via web.

Kitsu Web operational; website/legacy API/GraphQL/CDN roughly 99.997–100% reported 30-day uptime. iOS App Store and Google Play components labelled downtime with 0% reported 30-day uptime. List Imports currently operational, AniList import component 91.542% reported 30-day uptime, MAL 99.915%. high official reported state, uncertain actual all-user experience. Keep component-specific health and independent access/export paths. **Boundary:** monitoring definitions not independently validated; do not equate app-store component downtime with all installed apps broken, entire Kitsu dead, or MAL's global uptime. Avoid quoting precision in executive brief.

F15 — Kitsu network incident resolved; historical importer interruptions

Original source · **Date:** incident opened 6 September 2025, aggregate updates through 2 April 2026; resolved network issue at 04:39 UTC April2. · **Type/access:** official incident timeline; official incident timeline directly via web.

short list-import component outages/recoveries and website/API network issue. Latest incident message says networking restored. **Boundary:** high official timeline, not ongoing incident. Queue/retry imports, expose provider-specific state, avoid discarding prior history when remote service fails.

F16 — NeoGAF: MAL app friction vs website satisfaction

Original source · **Date:** 14 January 2025, posts #7794–7795 (previous recommendation context read). · **Type/access:** independent community forum; full primary thread page directly via web; relevant posts and replies inspected.

one participant says MAL mobile app repeatedly causes issues; reply says they always use website and recommends seasonal/simulcast list. No concrete defect specified. **Boundary:** low for cause, useful counterevidence. Distinguish service from client. A premium native client may solve interface friction without requiring users to abandon incumbent history/community. Dynamic link previews show later seasons, **do not use preview year as post date**.

F17 — Something Awful: independent tracking and backups as refuge during host churn

Original source · **Date:** 27–29 March 2026 (earlier page context read too). · **Type/access:** independent community forum; full primary forum page directly via web, multi-person reply chain read.

participant exports history for safety; another creates Anime-Planet + MangaUpdates history alongside app backups. Asked whether MangaDex list can import to A-P, replies only MAL/AniDB options were available and manually adds manga. Another describes raw CSV plus compatible XML export. Discussion concerns reader-host churn but these passages are specifically tracker portability, not a recommendation of reading sources. **Boundary:** same page corrects an unsupported paywall rumor; do NOT repeat that rumor as true. Import claim is a user's March check, not current hands-on verified capability.

F18 — ResetEra: satisfied tracker user still needs discovery

Original source · **Date:** 18 September 2025, eleven posts visible in indexed thread. · **Type/access:** independent community forum; search engine returned full thread and replies; direct web open errored. Label **indexed full-thread**, not direct.

OP says Trakt check-ins on iPhone work very well but fragmented streaming makes anime discovery hard; asks seasonal/history-informed suggestions. Replies recommend MAL, LiveChart genre/season browsing, AniList, Anime-Planet manga lists, and human discussion. OP later says suggested app is perfect for past/current seasons/top rankings. **Boundary:** existing products can satisfy needs. Users' AI-company descriptions are perceptions, not verified corporate facts. Do not identify which AniList client was chosen—it isn't named.

F19 — LiveChart official Android capabilities

Original source · **Date:** official listing says updated 29 August 2025; read Sep25 2026. · **Type/access:** official app listing; official app listing directly via web.

local-time countdowns, earliest/sub/dub release schedules, per-anime schedule choice, legal links, multiple tracking statuses, hiding statuses, opt-in release reminders. Current what's-new mentions disabled external resource button appearance/UI fixes. high documented feature benchmark. Release tracking is already specialized; Kuro needs region/language/source clarity and restrained alerts. **Boundary:** featured reviews are 2023/2024 and **excluded from 2025–2026 sentiment evidence**. Store download count not active users/market share. Product claims not hands-on verified.

F20 — LiveChart notification granularity

Original source · **Date:** undated official reference accessed Sep25 2026 (not dated user evidence). · **Type/access:** official feature reference; official page directly.

release reminders follow selected schedules; separate scheduling notes include delay/premiere; new titles linked to favorites/franchises/list; partner giveaways have separate notification type. **Boundary:** distinguish utility notification consent from partner promotions. Do not bundle commercial alerts with episode reminders. Existing benchmark not unmet-gap proof.

F21 — LiveChart guest history and scope limits

Original source · **Date:** undated official reference accessed Sep25 2026; one catalogue section explicitly last updated 2018. · **Type/access:** official FAQ; official FAQ directly.

guest marks live in browser storage, cannot cross browsers and are lost on clearing; explicit import needed after signup. No public API offered, RSS available; FAQ says no iOS app. Older-anime coverage section may be stale. **Boundary:** guest mode must explain local save and offer account migration; do not promise a public API integration without permission. Treat iOS/coverage statements as documentation claims requiring launch-time check, not exhaustive app-store audit.

F23 — Anime-Planet support/revenue boundaries

Original source · **Date:** undated official page read Sep25 2026. · **Type/access:** official funding policy; full page direct in-app browser.

says few ad units and donations pay server/site costs; no ads in profiles, minimal elsewhere, no pop-under/in-text/intrusive formats. Volunteer development model is explicit. **Boundary:** high as published policy, not audited revenue. Protect personal library/edit/progress screens from monetization; contextual useful commercial discovery may fund part of service, but need real economics and no inference this funds a salaried team.

F24 — AniDB official precise episode/file tracking model

Original source · **Date:** undated reference indexed and read Sep25 2026, not recent complaint. · **Type/access:** official documentation; search source page text; colon variant direct failed, encoded URL indexed accessible.

MyList separates entries present, seen episodes, and file state; unknown episode total represented as zero, specials separate. Warns statistics can lag DB update. **Boundary:** official model background. Distinguish progress, completeness and freshness; do not present unknown total as completed or zero-length.

F25 — AniDB/Shoko bulk-sync default creates queue surprise

Original source · **Date:** 16 January 2026; marked Closed at read. · **Type/access:** primary developer issue; primary developer issue directly.

novice importing multi-TB library reports six-digit MyList job queue and requests opt-out/default deprioritization, already uses another scrobbler. Third-party Shoko configuration issue, not AniDB service outage. medium report. Optional automation should explain volume, priority and provider quotas; fast manual updates should not queue behind historical imports. **Boundary:** closed does not establish implemented/deployed fix; no closure rationale returned in retrieved text.

Native clients and tracking integrations

C01 — MyAnimeList Official, Google Play reviews

Original source · **Date:** 2026-08-14 (kyla); 2026-07-07 (I Love Anime 2); 2026-07-03 (Manish Pal). Page lists app updated 2026-05-25. · **Type/access:** User review / discussion; Direct full page, three visible user reviews; developer description distinguished from reviews.

Respectively: indefinite loading at first use; desire to organize a personal collection into groups and filter by genre; English-title search difficulty and desire for grouped seasons. Third reviewer also claims missing manhwa. **Boundary:** Two reviewers open positively. No developer reply or reproduction inspected. The manhwa claim is not verified and must not be stated as MAL’s factual catalogue policy. Do not use 2021/2024 iOS featured reviews as recent evidence.

C02 — MyAniList, Apple public review feed (iOS AniList client; NOT MyAnimeList)

Original source · **Date:** 2026-06-07 through 2026-09-20; Apple RSS updated timestamps (reviews may be edited) · **Type/access:** User review / discussion; Direct Apple JSON via HTTPS; 50 most-recent reviews, exact timestamps. Web text reader cannot parse feed; shell retrieved successfully.

Review sample includes recurring upgrade prompt, perceived lifetime entitlement dilution, sorting friction, unwanted dropped-title reminders, tag overload and sync complaints. **Boundary:** July26 review praises season-level tracking; many sampled reviews praise free tracking, organization and discovery. Individual claims are not reproduced. June sorting issue explicitly corrected, not an ongoing defect. Key Apple review IDs: 14572658199 (20 September upgrade-prompt complaint), 14229862861 (26 June sorting complaint), 14248083774 (30 June edited correction: fixed/free). Dates are updated timestamps.

C03 — MyAniList changelog, provider/client repair

Original source · **Date:** explicit 2025-11-26 / 2025-12-12; 2026 releases dated Jun30, Jul15, Jul31 and September within current-year history. · **Type/access:** Official developer listing/release notes; Direct official release notes and listing.

2025 notes changed launch sync to fetch fewer updates. 2026 notes report fixes to background sync, login, sorting persistence, airing cues, notification reliability and VoiceOver support. Listed product offers native tracking, offline sync and social feeds. **Boundary:** Developer-reported fixes, not hands-on verification; notes show active maintenance, so do not portray every complaint as unresolved. Premium features exist; not Kuro’s proposed model.

C04 — ManGo: manga coverage and portability

Original source · **Date:** 2025-05-17 review; 2025-02-10 review; current release notes Sep14 2026. · **Type/access:** User review / discussion; Direct app page plus Apple feed.

May reviewer wants dependable chapter totals and new-chapter notifications; February reviewer asks for public shareable profile and import/export. Sep14 developer notes report fixes for accidental widget completion, incomplete cached lists, secondary-service repeating status and expired login tokens. **Boundary:** Unknown total is not necessarily stale data for an ongoing manga. Current client advertises MAL/AniList support; do not repeat reviewer's 2025 MAL-only framing as current architecture. Notes describe fixed bugs, not current universal defects.

C05 — Kitsune for MyAnimeList: conflicting design preferences

Original source · **Date:** 2025-01-13 displayed review (RSS updated2025-01-12 PST); 2025-02-28; 2026-01-05 exact RSS timestamp confirmed · **Type/access:** User review / discussion; Direct App Store page and replies; Apple XML RSS independently confirms Jan5 review year. US JSON feed empty; XML successful.

January2025 reviewer wants hideable scores. January2026 reviewer wants scores prominent and status categories accessible by swiping. February2025 reviewer praises clean UI and voluntarily buying a small one-time customization upgrade. **Boundary:** Opposing score preferences refute a universal design answer. Developer requests details rather than confirming a delivered fix. Current listing says free AniList sync but some schedules/notifications are premium; "only cosmetics are paid" would be inaccurate. January 2026 score-visibility review verified in Apple XML feed, ID 13599357667, updated 5 January; this resolves the year ambiguity in the ordinary store display.

C07 — AL-chan #169: rapid actions hit rate limits

Original source · **Date:** 2026-07-04; open on access. · **Type/access:** User review / discussion; Full issue and empty comments via official GitHub API; direct webpage checked.

Reporter says adding many seasonal shows rapidly causes API throttling and stops subsequent data loading; requests queued writes. **Boundary:** One technically informed user report; suggested design is not implemented proof or verified root cause.

C08 — AL-chan #168: auxiliary notifications caused launch crashes, repaired

Original source · **Date:** 2026-03-26; replies through 2026-04-01; closed/completed, implemented label. · **Type/access:** User review / discussion; Full body and maintainer replies via GitHub API. Web page only exposes issue body.

Android15 crash tied to new notification types. Maintainer identifies fallback parsing/indexing fault and says crash is being fixed; fork developer reports release with repair. **Boundary:** Maintainer participated despite commenters alleging abandonment. Closed/implemented; do not call it a current unpatched bug or AniList database failure.

C09 — AL-chan #166: cached list survives data outage

Original source · **Date:** 2025-09-28; closed/completed on access. · **Type/access:** User review / discussion; Direct issue and GitHub API, no comments.

Reporter saw fetch error but explicitly says anime list loaded from cache. **Boundary:** Cause and fix not documented; cached fallback worked sufficiently to display the list. Closure alone does not identify shipped version.

C10 — MALClient #357: calendar empty

Original source · **Date:** 2025-06-20; replies 2025-07-08; open on access. · **Type/access:** User review / discussion; Full body/comments via official GitHub API; page checked.

Calendar no longer loads current anime. Reply offers force-stop workaround; reporter confirms it worked. **Boundary:** Workaround does not establish root cause or general fix. It is MALClient, not MAL's official app.

C11 — MALClient #360: broken title details and maintenance anxiety

Original source · **Date:** 2026-08-01; replies Aug4 and Aug28; open on access. · **Type/access:** User review / discussion; Full body/comments via GitHub API; page checked.

User reports images, synopsis and favorites missing while other app functions still work. Asks if app abandoned. A fork maintainer claims repair after changing data integration; another user thanks them. **Boundary:** Store removal and fork completeness are user claims, not independently validated. People still call the app their preferred MAL client.

C12 — MALSync #4148: wrong-title detection on Crunchyroll

Original source · **Date:** 2026-08-12; maintainer Aug18; further user report Sep1; open on access. · **Type/access:** User review / discussion; Full body/comments via official GitHub API; webpage checked.

Wrong-show matching and correction complaints. Another user identifies added “Season 1” search suffix. Maintainer reports removing the first-season label and notes inconsistent provider data. Subsequent Hulu report exists. **Boundary:** Partial code change reported; cannot claim all detection fixed or a universal backend outage. Initial reporter says manual matching can persist, another says reuse behavior depends on closing search.

C13 — MALSync #4137: auto-progress complaint withdrawn after further testing

Original source · **Date:** 2026-08-05–06; closed/completed. · **Type/access:** User review / discussion; Full GitHub API issue and comments.

Reporter originally alleges progress advances without watching; maintainer cannot reproduce. Reporter later attributes behavior to interaction with other extensions/adblockers and closes issue. **Boundary:** Explicitly withdrawn as a standalone tracker defect. This record exists to prevent overstating automation evidence.

C14 — MAL mobile information demand, not streaming

Original source · **Date:** search index explicitly 2026-08-06; direct thread uses relative age. · **Type/access:** User review / discussion; Full thread read; exact initial date from indexed record.

OP wants MAL reviews/recommendations/interest stacks in better native client; doesn't need community/forums. Reports irrelevant past-season notifications in DailyAL and rejects streaming-client substitution. Kiko developer says reviews added, stacks planned; OP acknowledges improvement. **Boundary:** Development claims not app-tested. OP preference is not all users’ preference. Shows improvement during discussion.

MangaBaka, manga readers and specialist workflows

G01 — Progress totals became harder to see after a redesign

Original source · **Date:** approximately 17 September 2026; displayed 1 week, 19 hours on access; screenshot filenames also contain 2026-09-17. · **Type/access:** tracker-owned forum; opening post and status directly read; comments not loaded.

contributor reports that the newer UI shows current/total only for in-progress works, whereas the prior UI also displayed zero/total and complete/total. Asks for an optional setting, acknowledging others may prefer the simpler display. Status: **Considering**. retain explicit progress information in accessible detail and test compact versus information-rich cards. Do not use decorative progress shapes as the only signal. **Boundary:** Opening post and status read; comments did not load. Dates approximate from relative labels on 2026-09-25. Request does not establish community consensus.

G02 — Home page clutter

Original source · **Date:** approximately 19 September 2026 (displayed 5 days, 24 hours). · **Type/access:** tracker-owned forum; opening post directly read; comments not loaded.

user wants to hide unneeded home sections, change row counts, and optionally bring useful stats or followed creators forward. Current workaround is moving between menus. Status: **Considering**. strong default “Continue” area plus optional visibility controls later; do not build a full dashboard editor before testing whether one prominent primary action solves most friction. **Boundary:** Opening post and status read; comments did not load. Dates approximate from relative labels on 2026-09-25. Request does not establish community consensus.

G03 — Recommendation results can be too limited

Original source · **Date:** approximately 23–24 September 2026 (displayed 1 day, 13 hours). · **Type/access:** tracker-owned forum; opening post directly read; comments not loaded.

Mix recommends from chosen seed series but returns at most 24 results; user asks for pagination or more results ordered by relevance. Status: **Considering**. small curated defaults should have an optional deeper exploration path. This is counterevidence to imposing a universal tiny recommendation ceiling. **Boundary:** Opening post and status read; comments did not load. Dates approximate from relative labels on 2026-09-25. Request does not establish community consensus.

G04 — Recommend from this list, not my entire history

Original source · **Date:** approximately 19 September 2026 (displayed 6 days, 6 hours). · **Type/access:** tracker-owned forum; opening post directly read; comments not loaded.

request to seed recommendations from a custom list and quickly add the list to Mix. Status: **Considering**. let users narrow intent to “more like these” or a chosen collection. A whole-library taste profile can mix unrelated moods, eras, and purposes. **Boundary:** Opening post and status read; comments did not load. Dates approximate from relative labels on 2026-09-25. Request does not establish community consensus.

G05 — Follow creators and distinguish their identities

Original source · **Date:** approximately 19 September 2026 (displayed 6 days). · **Type/access:** tracker-owned forum; opening post and status directly read; comments not loaded.

user describes mixed/unreliable author results and manually checking favorite creators for new works. Wants dedicated creator pages and new-work alerts. Status: **Planned**. creator follow alerts are a manga-relevant discovery hypothesis after identity quality is adequate. Do not send alerts from name-only matches; author pseudonyms and same-name people need canonical identities. **Boundary:** Opening post and status read; comments did not load. Dates approximate from relative labels on 2026-09-25. Request does not establish community consensus.

G06 — Different publisher descriptions and chapter notes

Original source · **Date:** approximately 20 September 2026 (displayed 5 days, 3 hours). · **Type/access:** tracker-owned forum; opening post directly read; comments not loaded.

request to store multiple official publisher descriptions because localized titles/descriptions differ, while preserving important chapter-count notes across the description switch. Status: **Planned**. retain provenance and edition/language distinctions. Do not overwrite source descriptions with a single untraceable generated summary. A concise display can sit above expandable source detail. **Boundary:** Opening post and status read; comments did not load. Dates approximate from relative labels on 2026-09-25. Request does not establish community consensus.

G07 — Missing protagonist filters despite many tags

Original source · **Date:** approximately 22 September 2026 (displayed 2 days, 17 hours). · **Type/access:** tracker-owned forum; opening post directly read; comments not loaded.

user cannot find direct male/female protagonist filtering despite related demographic/orientation tags. Status: **Pending**. use the language readers actually search with. Audience demographic, creator identity, protagonist identity, and story genre are not interchangeable. This is a candidate taxonomy audit, not a mandate to infer personal identity. **Boundary:** Opening post and status read; comments did not load. Dates approximate from relative labels on 2026-09-25. Request does not establish community consensus.

G08 — Partial import, issue marked done

Original source · **Date:** approximately 24 September 2026 (displayed 1 day, 3 hours; closed about 18 hours before access). · **Type/access:** tracker-owned forum; opening post and status directly read; resolution explanation unavailable.

user reports 53 of 119 works imported through an intermediate service, and is unsure whether the earlier intermediary import lost entries. Status: **Done**. compare source counts at every migration step and preserve unmatched rows. Do not blame MangaBaka definitively; cause is ambiguous and the issue is closed. A reconciliation receipt is more valuable than a bare “import successful” toast. **Boundary:** Opening post and status read; comments did not load. Dates approximate from relative labels on 2026-09-25. Issue marked Done, cause/fix not independently verified.

G09 — Chapter-only logic misread volume progress, issue marked done

Original source · **Date:** approximately 22 September 2026 (displayed 2 days, 19 hours; closed about 2 days before access). · **Type/access:** tracker-owned forum; opening post and status directly read; resolution explanation unavailable.

series with nonzero volume progress but zero chapter progress were included in a “zero progress” filter. Status: **Done**. all library filters, stats, completion logic, and exports must respect both tracking units. This is evidence of a class of modeling error, not a claim of an ongoing MangaBaka bug. **Boundary:** Opening post and status read; comments did not load. Dates approximate from relative labels on 2026-09-25. Issue marked Done, cause/fix not independently verified.

G10 — Missing Western webtoons and coverage-policy confusion

Original source · **Date:** 26 June 2026, with later replies. · **Type/access:** community thread; post and substantive replies directly read; exact original date from search result.

reader tried MAL, AniList, Kenmei and MangaBaka, but could not immediately track all desired webtoons. In replies, a Kenmei-associated account says it has most Webtoon Originals except original English-language works; the poster clarifies Western originals are precisely the gap. Other users suggest private spreadsheets and submissions. **Boundary:** Poster corrected scope to Western originals; do not repeat claim Kenmei lacks most Originals or assume an unrestricted Webtoon API exists.

G11 — One-way sync breaks a multi-device workflow

Original source · **Date:** 8–10 August 2026. · **Type/access:** community thread; full substantive text in search result; post and replies inspected there.

a reader updates MangaBaka after reading on PC and wants the Android reader to receive those changes. They report moving trackers to get their desired workflow. This is one user, not retention statistics. name the sync direction plainly and protect against overwriting a newer remote value. Two-way support needs conflict behavior and a repair path, not merely two API calls. **Boundary:** One user; cross-post in r/mangapiracy is the same incident, not an independent source. Official Mihon docs corroborate one-way behavior.

G12 — Upcoming physical releases: source discovery, not spreadsheet software

Original source · **Date:** 24 June 2026. · **Type/access:** community thread; post and substantive replies directly read; exact date from search.

collector already has a spreadsheet, but retailer publisher filtering and inconsistent publisher calendars make finding future editions laborious. They want months-ahead data to prepare preorder purchases. Replies point to distributor/retailer calendars; OP marks the question resolved. Other participants describe duplicate preorders. **Boundary:** OP marked issue resolved via source suggestions. Affiliate opportunity is inference, not verified programme availability or economics.

G13 — Completion is not the same as translation completion

Original source · **Date:** 26 March 2026. · **Type/access:** community thread; post and substantive replies directly read; date from search.

reader cannot tell whether a niche manga is finished, on hiatus, abandoned, or merely untranslated. They see inconsistent counts and stale copied metadata. Replies recommend MangaUpdates/MangaBaka and title-specific original-language sources. separately label original publication status, latest verified original unit, available translated unit and language, and timestamp/source. Unknown stays unknown; a lack of recent translated chapters is not automatic hiatus. **Boundary:** Qualitative, purposive sample; not prevalence or market-share evidence.

G14 — Rich metadata is a strength worth preserving

Original source · **Date:** 29–31 July 2025; one search-discoverability follow-up 28 March 2026. · **Type/access:** community thread; substantive post and replies in search result, not full page direct.

users praise MangaUpdates for highly specific tags, alternate/localized names, creator work lists, licensing information, sequel relationships and chapter/season details. One wants more nuanced peer reviews beyond content-warning tags. Later reply complains that general search engines surface unrelated sites instead. simplify entry paths without erasing specialist metadata. Alias-aware search, creator identity, licensing and explicit content notes solve different needs from a numeric rating. **Boundary:** Search-indexed substantive text; no full direct-page read. Positive qualitative evidence, not objective catalogue benchmark.

G15 — Personal organization is sometimes enjoyable

Original source · **Date:** 13 May 2026. · **Type/access:** community thread; post and replies directly read; exact date from search.

readers build rich spreadsheets, relationship/trope categories and personal summaries, and explicitly enjoy it. One asserts MangaUpdates cannot place a work in multiple lists, qualified “AFAIK”; this was not verified as a current product limitation. Others find maintaining tags tedious. default a fast path, but allow optional personal tags, notes, multiple collections and visual preference. Do not assume removing all organization effort improves every reader's experience. **Boundary:** Unverified commenter claim about MangaUpdates multiple-list constraint is not established product fact. Several users enjoy manual organization.

G16 — List sharing exists but users do not discover it

Original source · **Date:** 25 May 2026. · **Type/access:** community thread; post and replies directly read; exact date from search.

reader with hundreds of completed/planned/waiting-for-completion works wants a visual shareable library because Notion entry is too laborious. After being told existing trackers support public lists, they say they had used MangaUpdates only for lookup. make sharing a simple optional output of an existing collection rather than forcing duplicate manual entry. Privacy defaults and progress disclosure must remain explicit. This is a discoverability problem, not absence of public list features. **Boundary:** Qualitative, purposive sample; not prevalence or market-share evidence.

G17 — Chapter-first readers do manual volume conversion

Original source · **Date:** 24–25 December 2025; later follow-up present. · **Type/access:** community thread; substantive post/replies in search result; source also existed in earlier report.

readers of official chapter apps describe waiting for volume publication, finding chapter-to-volume boundaries and editions, and backdating entries; others keep manga in AniList/MAL instead of StoryGraph. chapters and volumes need separate progress entry. Offer reliable conversion only when the edition-specific mapping is known; never silently assume a fixed chapters-per-volume ratio. Preserve partial dates rather than fabricate reading days. **Boundary:** Search-indexed substantive text. Already included in prior report; deduplicate.

G18 — Official integration contract: one-way is intentional

Original source · **Date:** current documentation read 25 September 2026; page does not expose a stable publication date. · **Type/access:** official documentation; directly read primary documentation.

Mihon documents one-way updates to trackers, offline progress synchronization when connectivity returns, per-series tracker linking, privacy only for supported services, and a separate class of enhanced self-hosted services with two-way local chapter syncing. It suggests alternate titles or a direct MAL ID when search cannot find a work. **Boundary:** Current primary documentation, undated; not an independent user complaint or hands-on test.

G19 — Official recent fixes demonstrate edge-case importance

Original source · **Date:** 4 August 2025 through 5 August 2026, individual releases dated. · **Type/access:** official documentation; directly read primary changelog.

Official 2025–2026 release notes describe improvements to migration, notes, large libraries and identity matching, plus fixes for duplicate-chapter restore loss, MangaUpdates HTTP failures and partial MAL dates. MangaBaka tracker support was added in July 2026. These are documented implemented fixes and enhancements, not evidence of currently unresolved defects. **Boundary:** Keep combined paraphrase across all use of this source within 200 words. Fixed cases, not current bugs.

G20 — Community-led discovery has a clear process

Original source · **Date:** thread started 13 February 2025; round plan edited 2 March 2025. · **Type/access:** community thread; opening post and multiple discussion replies directly read.

A small collective reading club nominates lesser-known works with reasons, votes, reads at a shared minimum pace and protects discussion beyond that point with spoiler rules. The poll shows 11 voters. This observed workflow supports testing nomination, rationale, milestones and spoiler boundaries; it does not establish demand for a new public social network. **Boundary:** Small voluntary group; observed workflow, not market-wide demand. Thread starts February 13, not February 14.

G21 — Coverage comparisons disagree

Original source · **Date:** 19 February 2025 (indexed date and Reddit archive date). · **Type/access:** community thread; post and substantive replies directly read.

original poster values database breadth, organization, obscure works, one-shots and a modern UI. Replies favor different trackers: some praise AniList breadth, while another says MAL contains manga missing from their AniList migration. MangaUpdates is praised for niche metadata while criticized as dated. One commenter incorrectly doubts it has tracking; other observed use contradicts that. **Boundary:** Replies disagree about catalogue breadth; do not report subjective ranking or false claim MangaUpdates has no tracking as product facts.

G23 — MangaBaka proposition

Original source · **Date:** current product page read 25 September 2026; undated. · **Type/access:** official product page; directly read.

broad manga/manhwa/manhua/novel/OEL coverage, chapter and volume tracking, notes, imports and exports, flexible card/list layouts and library-based recommendations. Its “300,000+” count is a vendor catalogue claim, not our independently deduplicated benchmark. Sources retain ownership of upstream data. Kuro cannot differentiate just by saying clean, personalized and portable. Those promises already exist; execution and a chosen audience must be tested. **Boundary:** Vendor product claims; catalogue count not independently benchmarked. Undated current reference.

G24 — Aggregation does not eliminate freshness or rights dependencies

Original source · **Date:** current docs read 25 September 2026; undated. · **Type/access:** official documentation; directly read.

upstream metadata providers are named; per-title refresh is described as minimum one day and maximum seven days. Schema is not yet guaranteed stable, and metadata corrections are directed upstream. assess provenance, freshness, schema stability and permitted downstream use per provider before treating any aggregator as an

independent unrestricted data source. This is an integration diligence point; no legal compliance conclusion about Kuro or MangaBaka is made. **Boundary:** Current primary documentation, undated. Does not establish permission to reuse upstream metadata commercially.

Affiliate programmes and practical commercial terms

A01 — Thalia official partner program

Original source · **Date:** Undated live page; accessed 2026-09-25 · **Type/access:** official merchant program; Full text fetched

Books 11%; non-price-bound books with coupon 8%; 30-day attribution; application requires approval; daily product data. **Boundary:** Commissionable gross/net basis not established by public page. Publisher native-app traffic approval unknown. DE terms do not automatically cover AT/CH.

A02 — Thalia DE Awin merchant profile 14158

Original source · **Date:** Undated live page; accessed 2026-09-25 · **Type/access:** official affiliate network merchant profile; Full text fetched

Corroborates 11% books, 8% non-price-bound books with coupon, 30-day window, product feed and App-Tracking; cashback barred on coupon transactions. **Boundary:** App-Tracking can describe merchant-app attribution; it does not establish Kuro native-app publisher eligibility. Accepted commission base and exclusions require contract verification.

A03 — Amazon DE standard commission schedule

Original source · **Date:** Current published schedule; accessed 2026-09-25 · **Type/access:** official merchant rate schedule; Full text fetched

German books rate currently 4.5%, not old third-party 7%; Blu-ray/DVD 6%, toys 3%. Rate applies to contractual qualifying revenue. **Boundary:** Qualifying revenue deductions defined in policies; school-linked entity exceptions; category/territory and accepted account terms govern.

A04 — Amazon DE Associates program policies

Original source · **Date:** Updated 2026-04-14; accessed 2026-09-25 · **Type/access:** official contractual policies; Full text fetched

Qualifying revenue excludes shipping, taxes, fees, credits/rebates and other enumerated deductions. 24-hour session can terminate earlier; qualifying cart completion within 89 days. Approved free original-content mobile apps, permitted linking tools and no Amazon WebViews required; user-level subtags prohibited. **Boundary:** Kuro approval unverified; returns/refunds excluded. Native app eligibility does not by itself settle Apple rules for outbound digital purchases. Limit derivation from this page to 200 words in final report.

A05 — Amazon US standard rates and bounty events

Original source · **Date:** Current published schedule; accessed 2026-09-25 · **Type/access:** official merchant rate schedule; Full text fetched

Physical books 4.5%; eligible Prime Video free trial $2, add-on subscription trial $2.50 and paid $3. Eligibility-specific actions, not every click/subscriber. **Boundary:** US values only. Do not apply to Germany. No account approval or native digital-link compliance verified. Bounties optional secondary route, not modeled baseline.

A06 — Bookshop.org affiliate program explanation

Original source · **Date:** Displayed modified 11 February without year; accessed 2026-09-25 · **Type/access:** official merchant support; Full text fetched

Non-bookstore media affiliates have 48-hour attribution and last-touch assignment. Bookstore affiliation and storefront economics are a different program. **Boundary:** Do not present bookstore permanent affiliation or 30% bookstore share as Kuro economics. Native publisher-app approval not established.

A07 — Bookshop.org terms including affiliate program

Original source · **Date:** Affiliate Terms last updated 2025-07-10; page also contains older duplicated sections; accessed 2026-09-25 · **Type/access:** official contractual terms; Full text fetched

Base media affiliate 10%; revenue gross receipts received less applicable taxes, without listed operating-cost deductions; 48-hour tracking, $20 payout threshold. Enhanced additional 5% requires at least 100k audience and exclusive physical-book affiliate placements. **Boundary:** Use the later 2025 section. US terms do not establish UK rates or eligibility. Enhanced exclusivity conflicts with neutral comparison; Kuro native-app approval is unknown.

A08 — Bookshop.org international shipping support

Original source · **Date:** 2025-02-21; accessed 2026-09-25 · **Type/access:** official merchant support; Full text fetched

US orders ship within US and UK orders within UK; no direct Germany fulfillment route stated. **Boundary:** Physical shipping claim; do not conflate with ebook territorial licensing.

A09 — Rakuten Kobo US affiliate program

Original source · **Date:** Undated live page; accessed 2026-09-25 · **Type/access:** official merchant program; Full text fetched; localized AT page corroborated

Up to 5% ebooks, audiobooks and devices; up to 10% accessories; 14-day tracking; approval stated 1–2 weeks. **Boundary:** Up to is a ceiling, not accepted rate. Gross/net base, publisher native-app eligibility and exact German program terms not established. DE page fetch failed; AT localization is not proof of DE approval.

A10 — CDJapan affiliate commission FAQ

Original source · **Date:** Undated live page; accessed 2026-09-25 · **Type/access:** official merchant FAQ; Full text fetched

5–7% tiered by prior monthly qualifying product-price sales; 30-day window; paid and shipped orders count; cancellations/returns excluded. Default payout points, PayPal threshold JPY5000. **Boundary:** Commission based on qualifying product price, not total landed cost; taxes/net exact handling should be confirmed. Preorders can remain pending until shipped; app approval and destination shipping not established.

A11 — CDJapan affiliate program terms

Original source · **Date:** Undated live page; accessed 2026-09-25 · **Type/access:** official contractual terms; Full text fetched

Application approval; Site definition website/blog/mailinglist; registered accepted placements only; incentives and artificial click generation prohibited; limited product asset license for sales referrals. **Boundary:** No blanket native mobile-app permission; referral asset permission does not license general tracker catalog reuse.

A12 — EVENTIM DE Awin merchant profile 11388

Original source · **Date:** Undated live page; accessed 2026-09-25 · **Type/access:** official affiliate network merchant profile; Full text fetched

20% of net presale fee, not ticket face value. Page says fee averages 8–10% net ticket value; events may be excluded. Official Eventim partner page accepts website/app applications. **Boundary:** Header says one-day attribution but body says session-based; clarify before modeling. Native-app acceptance unknown. Relevant anime-event coverage unverified.

Publisher precedents, funding and distribution constraints

B01 — MyAnimeList — Publisher & Bookseller Picks 2026

Original source · **Date:** 2026 edition; accessed 2026-09-25 · **Type/access:** Official sponsored discovery page; Direct public HTTP 200; text read

MAL identifies selections by comici MANGA, SHUEISHA and Yen Press as sponsor picks, with reading destinations. This directly demonstrates a labelled commercial discovery area connected to useful title exploration. **Boundary:** No published placement prices, campaign conversion or audience satisfaction. The existence of sponsors does not prove Kuro can sell similar inventory profitably.

B02 — MAL×Japan — More than just anime

Original source · **Date:** Undated hub; includes 2025 and 2026 programmes; accessed 2026-09-25 · **Type/access:** Official programme directory; Direct public HTTP 200; text and linked programme URLs read

The hub lists licensed-sample projects, publisher picks, creator contests, quizzes, educational offerings and giveaways. It shows established tracker activity extending into industry-supported discovery and participation. **Boundary:** A programme listing does not establish cash sponsorship, profitability, present availability or audience benefit. Individual dated programmes are stronger evidence than this changing hub.

B03 — MyAnimeList — Manga Sampler Pilot Project

Original source · **Date:** Giveaway 2026-07-21 to 2026-09-15; accessed 2026-09-25 · **Type/access:** Official experimental programme and terms; Direct public HTTP 200; full page and terms read

Rights-holder-authorized opening chapters and disclosed unreviewed machine translation let overseas readers sample titles and give publishers feedback about translation demand. It is described as experimental and free for now. **Boundary:** The giveaway has ended. Region/language restrictions apply; commercial use of its content is prohibited. Publisher fees and financial results are not disclosed; Kuro must secure its own rights.

B04 — NTT DOCOMO announces U.S. rollout of MANGA MIRAI

Original source · **Date:** 2025-03-05 · **Type/access:** Official partner launch release; Web tool full text read

DOCOMO describes its partnership with MAL, Akatsuki and MEDIA DO. MAL integrates previews and purchase access into its site, providing a current-period example of discovery connecting to legitimate manga commerce. **Boundary:** This US rollout does not establish Germany availability or an affiliate rate. The release describes partner roles, not audited user outcomes, Kuro partner eligibility or revenue allocation.

B05 — KADOKAWA launches MAL × Honeyfeed Writing Contest 2025

Original source · **Date:** 2025-06-25 · **Type/access:** Official publisher programme announcement; Web tool full Japanese text read

KADOKAWA, qdopp/Honeyfeed and MyAnimeList jointly announced an English-language isekai contest to discover authors for possible Japanese publication. This is a concrete collaboration offering participants a creative opportunity. **Boundary:** The release does not disclose payments to MAL or prove that small private reading clubs attract sponsorship. Treat it as a programme precedent, not a Kuro sales benchmark.

B06 — KADOKAWA announces 2025 contest winners

Original source · **Date:** 2026-02-10 · **Type/access:** Official publisher programme result; Web tool full Japanese text read

The publisher reports 1,240 submissions, one grand-prize winner and two additional winners; the winning work is to be published in Japan. This confirms that the previously announced collaboration progressed to a concrete participant outcome. **Boundary:** Publisher-reported participation is not independent impact measurement. No platform revenue, participant satisfaction or sponsor-return figure is disclosed.

B07 — LiveChart.me — About

Original source · **Date:** Undated; accessed 2026-09-25 · **Type/access:** Official operating-model description; Direct public HTTP 200; text read

LiveChart describes itself as a hobby project using a small number of text and banner advertisements to cover operational costs. It invites advertising enquiries and emphasizes service simplicity and information accuracy. **Boundary:** This is an operator statement, not a financial report or a placement audit. No ad rates or evidence of profitability at Kuro's prospective scale are provided.

B09 — MyAnimeList — Advertising

Original source · **Date:** Undated; accessed 2026-09-25 · **Type/access:** Official advertising sales page; Direct public HTTP 200; text read after web-reader failure

MAL sells skin, IAB-standard and mobile ad formats and accepts campaign briefs. Germany is among selectable target countries. This verifies actual advertising inventory rather than relying on speculative business-model summaries. **Boundary:** The page's self-reported 12 million monthly unique visitors and 150 million pageviews lack a measurement date; do not present them as verified 2026 market sizing. No prices disclosed.

B10 — 33 USA — MAL33 global marketing support service

Original source · **Date:** Undated page, service marked established 2023; accessed 2026-09-25 · **Type/access:** Official agency service page; historical context; Web tool page read

Agency 33 describes an anime/manga global-marketing service jointly launched with MyAnimeList. It supports the existence of B2B services adjacent to an anime tracking community. **Boundary:** It does not validate white-label recommendation widgets or privacy-preserving campaign reporting specifically. Its old undated audience claims and undisclosed contract economics should not become a 2026 financial model.

B11 — AniList API Terms of Use

Original source · **Date:** Undated live terms; accessed 2026-09-25 · **Type/access:** Official API conditions; Web tool full terms read

Terms prohibit competing non-complementary trackers, hoarding and backup/storage use. Commercial services below $150 monthly revenue are described as free; above that threshold a commercial licence is required. **Boundary:** The commercial threshold does not override the separate competition restriction or grant redistribution rights. Exact $150 wording is imprecise. Kuro's own agreement and allowed scope were not established.

B12 — MyAnimeList API License and Developer Agreement

Original source · **Date:** Last modified 2019-08-08; live retrieved 2026-09-25 · **Type/access:** Official API agreement; current dependency reference; Direct public HTTP 200; full agreement read

Commercial-app revenue needs written authorization; release and major updates including ads require notice and possible review. The agreement restricts server-side MAL personal/user-generated content and minor-targeted ads. **Boundary:** Its noncommercial definition permits some nondisruptive click/view advertising, so avoid claiming all ads are forbidden. It is an old-dated current contract reference, not contemporary user evidence; Kuro's permission remains unknown.

B13 — Apple — App Review Guidelines

Original source · **Date:** Live guidelines; accessed 2026-09-25 · **Type/access:** Official platform rules; Web tool and direct public HTTP; sections 2.5.18, 3.1.1, 3.1.3 and 5.1.2 read

Rules distinguish physical commerce from digital purchases, vary external-link permissions by storefront, govern ad placement/reporting and require permission for tracking. Display advertising is barred from widgets and notifications. **Boundary:** A tracker is not automatically a reader app; exact purchase/referral flows need review. US rules cannot

be copied into Germany without checking applicable terms. These are dependencies, not a finding of Kuro noncompliance.

B14 — German media authorities — Online advertising disclosure guide

Original source · **Date:** May 2025 edition · **Type/access:** Official regulator practical guidance; Web tool indexed PDF text; edition corroborated by official advertising page

The media authorities explain transparent separation and labelling of commercial online content. This supports conspicuous sponsorship disclosure where a recommendation, sample or campaign appears, instead of hiding it in a terms page. **Boundary:** Specific wording and placement depend on the format and relationship. The guide is practical supervisory guidance, not a complete legal review of Kuro's proposed implementation.

B15 — European Commission — Digital Services Act questions and answers

Original source · **Date:** Current FAQ; accessed 2026-09-25 · **Type/access:** Official regulatory explanation; Web tool text read

The Commission explains restrictions on profiling-based advertisements to minors and sensitive-data targeting, plus child-protection responsibilities. Avoiding behavioural targeting is a practical design choice for an anime app that may attract younger users. **Boundary:** Kuro's service classification, size and applicable exemptions require assessment. Do not assert that every platform or very-large-platform obligation automatically applies to a small tracker.

B16 — Germany — TDDDG Section 25

Original source · **Date:** Current consolidated legislation; accessed 2026-09-25 · **Type/access:** Official legal text; Web tool text read

Section 25 generally requires consent for device storage/access, with narrow transmission and strictly necessary requested-service exceptions. Advertising cookies, tracking identifiers and SDK behaviour therefore need assessment before a monetization implementation. **Boundary:** Contextual sponsorship does not automatically mean every measurement method is exempt. Avoid broad claims that all analytics require consent or that a particular Kuro design has already been legally approved.

B17 — EDPB — Process personal data lawfully

Original source · **Date:** Current small-business guidance; accessed 2026-09-25 · **Type/access:** Official GDPR practical guidance; Web tool text read

The EDPB explains lawful bases, consent and children's consent when applicable; a service contract is not a blanket justification for unrelated marketing. It supports separating optional campaign research from the tracking service. **Boundary:** Do not reduce GDPR to 'all marketing always needs consent'. Exact lawful basis, age threshold and national rules depend on the processing and jurisdiction; no Kuro legal audit was performed.

B18 — EDPB — Anonymisation / pseudonymisation

Original source · **Date:** Current topic guidance; accessed 2026-09-25 · **Type/access:** Official privacy distinction; Web tool page text read

The EDPB distinguishes reduced linkability through pseudonymisation from true anonymisation. Simply removing user IDs from campaign or club reports is not enough to promise that recipients cannot identify anyone. **Boundary:** Aggregation thresholds are not a legal safe harbour. Underlying collection, rare cohorts and linkage risks still need assessment. Recommend broad campaign totals, no private history exports and no user-level sponsor datasets.

Selected AniList and schedule evidence retained from the first pass

S08 — Imported list not sorting correctly by Recently Added

Original source · **Date:** 2026-05-04 to 2026-05-05 · **Type/access:** platform forum/update; Direct in-app browser thread, including replies/dates, in preceding pass

Developer explains that an import cannot reconstruct missing watch dates/order from the source file. Distinguishes absent source history from a broken display. **Boundary:** Retained evidence; no signed-in or runtime test.

S09 — Duplicate entries on anime list

Original source · **Date:** 2026-08-05 to 2026-08-30 · **Type/access:** platform forum/update; Direct in-app browser thread, including replies/dates, in preceding pass

Developer reports a fix live on 30 August; existing duplicates need manual cleanup with preservation of information. Treat as a resolved cause with recovery burden, not an unqualified current unfixed bug. **Boundary:** Retained evidence; no signed-in or runtime test.

S10 — AniList Bugs and Issues

Original source · **Date:** Explicit updates July 2025 and August–September 2026; thread began 2024 · **Type/access:** platform forum/update; Direct in-app browser thread, including replies/dates, in preceding pass

Acknowledges rare custom-list shuffling on deletion, a following-feed issue, and delayed stats refresh; the stats issue has a likely fix deployed. Thread began in 2024. Only explicitly dated 2025–2026 entries support the recent analysis. **Boundary:** Retained evidence; no signed-in or runtime test.

S11 — Improve AniList Search Behavior

Original source · **Date:** 2026-08-17 · **Type/access:** platform forum/update; Direct in-app browser thread, including replies/dates, in preceding pass

Describes inconsistent partial-title results; reply points to search improvements on the roadmap. User report, not a reproduced search benchmark. **Boundary:** Retained evidence; no signed-in or runtime test.

S12 — How do I see Anime only Recommendations?

Original source · **Date:** 2026-04-19 · **Type/access:** platform forum/update; Direct in-app browser thread, including replies/dates, in preceding pass

User asks about unwanted medium mixing; reply suggests an existing list filter and disputes the premise. Evidence of filter comprehension, not proof of a missing capability. **Boundary:** Retained evidence; no signed-in or runtime test.

S13 — Is there a way to filter tags?

Original source · **Date:** 2026-08-06 · **Type/access:** platform forum/update; Direct in-app browser thread, including replies/dates, in preceding pass

Moderator explains an existing exclusion gesture; user acknowledges the help. Particularly strong example of discoverability rather than feature absence. **Boundary:** Retained evidence; no signed-in or runtime test.

S26 — AniList Roadmap & Changelog

Original source · **Date:** Roadmap updated 2026-08-20 · **Type/access:** platform forum/update; Direct in-app browser thread, including replies/dates, in preceding pass

Lists search/UI improvements, exports, volume-centered tracking and privacy among future work. Existing volume fields and volume-centered product behavior are not the same thing. Older changelog comments excluded. Roadmap items have no guaranteed delivery dates. **Boundary:** Retained evidence; no signed-in or runtime test.

P06 — LiveChart release schedules

Original source · **Date:** Undated official reference accessed 2026-09-25 · **Type/access:** official reference; Indexed official page text in preceding pass

Demonstrates separate broadcast, subtitled and dubbed schedules with a regional applicability warning. Used for data semantics, not current availability of the example title. **Boundary:** Retained evidence; no signed-in or runtime test.

P08 — Community apps

Original source · **Date:** Undated official reference accessed 2026-09-25 · **Type/access:** official reference; Indexed official page text in preceding pass

Identifies listed apps, including MyAniList, as not maintained by or affiliated with AniList staff. Market map and naming distinction. **Boundary:** Retained evidence; no signed-in or runtime test.

Market context, not tracker market-share estimates

T01 — Sony / Bandai Namco / Gaudiy strategic partnership

Original source · **Date:** 2025-05-08 · **Type/access:** primary joint corporate release; Full PDF text read

Reports MAL at 19.5 million registered users in 2025 and describes the strategic partnership. Establishes scale of an incumbent and corporate interest in the audience. **Boundary:** Self-reported registrations, not current MAU, tracker market share, Kuro reachable market or consumer demand for a new app.

T02 — Dentsu — Anime: A Growing Opportunity for Brands

Original source · **Date:** Data collected October 2024–March 2025; companion article 2025-07-25 · **Type/access:** primary research publisher summary; Official methodology page and companion article read

The stated study covers 8,600 consumers across ten named countries; Germany is absent. A companion Dentsu article discusses anime-related consumer spending and brand opportunities. **Boundary:** Commercial research context and broad fandom sample; not a tracker-user or Kuro monetization study. No percentages are extrapolated into Kuro economics. Companion article.

19.12. Limits and final decision boundaries

This is substantially broader research than the first edition, but it is not exhaustive access to every forum or a comparative usability trial. MAL and MangaUpdates forum access gaps remain explicit. MangaBaka comments were unavailable. Some Reddit dates come from exact search-index timestamps, some dates are approximate, and store feeds can change. No private libraries or installed competitor apps were tested. Official descriptions and status pages may differ from a particular user's experience.

User reports establish reported friction. Developer replies can clarify or report fixes, but do not independently prove a deployed repair. The report deliberately includes positive experiences, hidden existing features, corrected complaints and withdrawn allegations. It does not rank platforms by complaint counts: each platform has different access, audience, issue-reporting habits and sample coverage.

Commercial programme terms, regulations and store rules can change. Current published terms are evidence to assess a proposed route, not legal advice or confirmation of Kuro's private contracts. Merchant/app approval, data rights, precise country/channel rules and accepted commercial bases remain implementation prerequisites. A zero-revenue result remains possible.

The product direction is **trusted continuity with optional depth**. The business direction is **useful commerce and clearly separated partner-funded benefits**, with no user fee for Kuro's core. These are 18 product and 10 monetization decisions to validate—not shipped functionality, partner commitments or proven product-market fit.